



AURORA

Manual de uso e referência técnica

Versão 6.3.2

NIPSCERN

Núcleo de Inovação e Pesquisa em Sistemas Computacionais

Junho de 2026

Sumário

I	Primeiros passos	1
1	Escolha o fluxo de trabalho	2
1.1	Os fluxos disponíveis	2
1.2	Comparação rápida	2
1.3	Etapas compartilhadas	3
1.4	Qual fluxo escolher	3
2	O que é o SAPHO	4
2.1	O que é um soft-processor	4
2.2	As peças da plataforma	4
2.3	O fluxo de trabalho de ponta a ponta	5
2.4	Um exemplo condutor	6
2.5	Quem faz a plataforma	7
2.6	O próximo passo	7
3	Instalação e primeiro início	8
3.1	Antes de instalar	8
3.2	Baixar e instalar	8
3.3	O que é instalado, e onde	9
3.4	Primeira execução	9
3.5	Atualizações	10
3.6	Se a AURORA não iniciar	10
3.7	Desinstalação	10
3.8	O próximo passo	10
4	Tour pela interface	11
4.1	A barra superior	12
4.2	A árvore de arquivos	12
4.3	O editor	16
4.4	Os terminais	16
4.5	A barra de status	16
4.6	Paleta de comandos e notificações	17
4.7	Quando um botão estiver desabilitado	17
4.8	O próximo passo	17
II	Projetos e ferramentas	18
5	Projetos	19
5.1	Criar	19
5.2	Abrir	19
5.3	O que é o arquivo .spf	20
5.4	Navegar pela pasta do projeto	20
5.5	Filtrar a visualização Pastas com .inv	20
5.6	Importar arquivos	20
5.7	Fechar e reabrir	21
5.8	Backup	21
5.9	Renomear ou excluir	21
5.10	Versionamento	21

6	Editor, abas e painéis	22
6.1	Abrir arquivos	22
6.2	Abas	22
6.3	Dividir o editor	22
6.4	Localizar e navegar	23
6.5	Salvar	23
6.6	Usar a seleção com IA	23
6.7	Mudanças externas	23
7	Os terminais	24
7.1	Quem escreve onde	24
7.2	Cartões, filtros e o modo verboso	24
7.3	Ler os avisos de recurso instanciado	25
7.4	O TCMD, o shell integrado	25
7.5	Uma decisão de segurança que explica um limite	26
7.6	Exportar para relatar um problema	26
7.7	Leitura relacionada	26
8	Source Control e Git	27
8.1	Começar com o projeto aberto	27
8.2	Alterações e commits	28
8.3	Sincronização e branches	28
8.4	Conectar e clonar do GitHub	28
8.5	Usar .inv sem alterar o Git	29
8.6	Cuidados	29
III	Fluxo Verilog	30
9	Fluxo completo para projetos Verilog	31
9.1	1. Criar ou abrir o projeto	31
9.2	2. Adicionar os módulos RTL	32
9.3	3. Definir o Top Level	32
9.4	4. Adicionar o testbench	32
9.5	5. Validar o Verilog	32
9.6	6. Simular	32
9.7	7. Analisar formas de onda	33
9.8	8. Visualizar no PRISM	33
9.9	Resultado esperado	33
10	Arquivos Verilog e Testbenches	34
10.1	Tipos de arquivo	34
10.2	Adicionar arquivos	34
10.3	Definir o Top Level	34
10.4	Definir o Testbench Top	35
10.5	Usar a vista de hierarquia	35
10.6	Corrigir arquivos ausentes	35
10.7	Evitar erros comuns	36
IV	Fluxo SAPHO	37
11	Fluxo completo para processadores SAPHO	38
11.1	O caminho em uma olhada	39
11.2	1. Criar ou abrir o projeto	39
11.3	2. Criar o processador	40
11.4	3. Escrever o algoritmo C±	40
11.5	4. Gerar o hardware	40

11.6	5. Definir top-level e testbench	40
11.7	6. Escolher a forma de execução	41
11.8	7. Conferir entradas e saídas	41
11.9	8. Analisar o hardware gerado	41
11.10	Resultado esperado	41
11.11	Leitura relacionada	42
12	Primeiro projeto: um filtro de média móvel	43
12.1	Passo 1: criar o projeto	43
12.2	Passo 2: criar o processador	43
12.3	Passo 3: escrever o algoritmo	45
12.4	Passo 4: compilar	46
12.5	Passo 5: preparar o estímulo e simular	47
12.6	Passo 6: ler a forma de onda	48
12.7	Passo 7: ver o circuito por dentro	49
12.8	O que você aprendeu	49
12.9	Para onde ir agora	50
13	Criar e configurar processadores	51
13.1	O Hub de Processadores	51
13.2	O atalho \$cmm	52
13.3	O que é gerado	53
13.4	O processador ativo	53
13.5	Configurações de simulação	53
13.6	Gerar o hardware	54
13.7	Testar o processador isoladamente	55
13.8	Renomear e excluir	55
13.9	Vários processadores no mesmo projeto	56
14	Compilação: de C± a Verilog	57
14.1	O pipeline do YANC em três estágios	57
14.2	Os botões de compilação	58
14.3	Antes de clicar	58
14.4	Onde cada artefato aparece	58
14.5	Validar um projeto Verilog	59
14.6	Ações que preparam as suas dependências	59
14.7	Interromper uma execução	59
14.8	Uma decisão de segurança	60
14.9	Erros comuns	60
14.10	Leitura relacionada	60
V	A linguagem C±	61
15	A linguagem C±: fundamentos	62
15.1	A estrutura de um programa	62
15.2	O que ler primeiro	63
15.3	Um resumo de uma página	63
15.4	Onde o arquivo vive	63
15.5	Constantes com #define	64
15.6	E se eu precisar do que a C± não tem	64
16	Diretivas: configurando o processador no código	65
16.1	As diretivas de configuração	65
16.2	Como escolher cada valor	65
16.3	As diretivas comportamentais	66
16.4	Onde as diretivas aparecem depois	66

16.5	Erros comuns com diretivas	67
16.6	Referência rápida	67
17	Tipos, operadores e controle	68
17.1	Os quatro tipos	68
17.2	Operadores	69
17.3	Estruturas de controle	70
17.4	Funções	70
17.5	Vetores	71
17.6	Variáveis e escopo	72
17.7	O próximo passo	72
18	Entrada, saída e biblioteca padrão	73
18.1	As portas de entrada e saída	73
18.2	Funções especiais	73
18.3	Funções não lineares e de arredondamento	74
18.4	Funções para complexos	74
18.5	Álgebra linear em notação de Dirac	75
18.6	Referência rápida	76
18.7	O próximo passo	76
19	Recursos avançados	77
19.1	Interrupção: #PRACA	77
19.2	Sincronização com o hardware: #TOAQUI	77
19.3	FFT e o endereçamento bit-reverso	78
19.4	Quanto custa cada construção	78
19.5	O caminho C++	79
19.6	As mensagens do compilador	80
19.7	Leitura relacionada	81
VI	O processador SAPHO	82
20	O processador SAPHO por dentro	83
20.1	Visão geral	83
20.2	O caminho de dados	83
20.3	Os módulos gerados	84
20.4	Os parâmetros fundamentais	84
20.5	Entrada, saída e o mundo externo	85
20.6	Por que não há recursão	85
20.7	Um processador por tarefa	85
20.8	Leitura relacionada	86
21	O ponto flutuante do SAPHO	87
21.1	O formato	87
21.2	Por que não IEEE 754	87
21.3	O que esperar em precisão	87
21.4	Na prática, dentro do programa	88
21.5	Números complexos	88
21.6	Quando o float não vale a pena	89
21.7	Leitura relacionada	89
22	O conjunto de instruções	90
22.1	Como o conjunto é organizado	90
22.2	Convenções de prefixo e sufixo	90
22.3	As famílias	90
22.4	Lendo a trilha na forma de onda	91
22.5	Uma leitura comentada	92

22.6	Leitura relacionada	92
VII	Simular e analisar	93
23	Simular com Icarus, Verilator ou cocotb	94
23.1	Preparar a simulação	94
23.2	Escolher o simulador	94
23.3	Análise de forma de onda ou execução rápida	95
23.4	Testbench Verilog	95
23.5	Testbench cocotb	96
23.6	Confirmar o resultado	97
23.7	Problemas comuns	98
24	Galeria de projetos e testbenches	99
24.1	Como usar os exemplos	99
24.2	Porta AND em Verilog	99
24.3	Contador de 4 bits em Verilog	101
24.4	Soma de constantes em C $\pm$	104
24.5	Sequência de quadrados em C $\pm$	106
24.6	Escolher entre os dois tipos de testbench	109
25	Formas de onda: GTKWave e Surfer	111
25.1	O que a AURORA prepara	111
25.2	Gerar uma forma de onda	111
25.3	Escolher os sinais gravados	112
25.4	O seletor de layout	113
25.5	GTKWave	113
25.6	Surfer	113
25.7	Se o visualizador abrir sem sinais	114
25.8	Se o arquivo de ondas não for encontrado	114
25.9	Leitura relacionada	114
26	Visualizar e simular o RTL no PRISM	115
26.1	O que acontece quando você clica	115
26.2	Abrir o PRISM	115
26.3	Projeto Verilog puro	116
26.4	Processador SAPHO	116
26.5	Navegar no diagrama	116
26.6	Simulação interativa	117
26.7	O que o PRISM confirma	118
26.8	Quando o PRISM falhar	118
VIII	Aurora Intelligence	120
27	Aurora Intelligence	121
27.1	O que a distingue de uma assistente genérica	121
27.2	Como ela funciona por dentro	122
27.3	Abrir e usar o painel	122
27.4	Para que ela é boa no fluxo SAPHO	122
27.5	Como pedir bem	123
27.6	Um fluxo de trabalho seguro	123
27.7	Conversas e disponibilidade	124
27.8	Privacidade	124
27.9	Leitura relacionada	124

28 Configurar o provedor de IA	125
28.1 Escolher uma opção	125
28.2 Provedor por API	125
28.3 Ollama	126
28.4 Claude Code	126
28.5 ChatGPT via Codex CLI	126
28.6 Se o teste falhar	127
29 Tarefas com a Aurora Intelligence	128
29.1 Entender o projeto	128
29.2 Corrigir compilação	128
29.3 Editar com segurança	128
29.4 Trabalhar com processadores	128
29.5 Simular e analisar ondas	129
29.6 Organizar arquivos	129
29.7 Boas práticas	129
30 Usar Claude Code e ChatGPT via Codex CLI	130
30.1 Como a AURORA encontra os CLIs	130
30.2 Preparar o Claude Code	130
30.3 Preparar o ChatGPT via Codex CLI	130
30.4 Durante a conversa	131
30.5 Problemas comuns	131
IX Configuração e ajuda	132
31 Configurações do aplicativo	133
31.1 Geral	133
31.2 Aparência	133
31.3 Terminal	134
31.4 Atalhos	134
31.5 AI Assistant	134
31.6 About e atualização	134
31.7 Zoom e janela	134
32 Referência de diretivas	135
32.1 Diretivas de configuração	135
32.2 Diretivas comportamentais	135
32.3 Pré-processador	135
32.4 A restrição estrutural	136
33 Referência da biblioteca padrão	137
33.1 Entrada e saída	137
33.2 Funções especiais	137
33.3 Não lineares e arredondamento	137
33.4 Complexos	138
33.5 Notação de Dirac	138
33.6 Operadores	138
33.7 Palavras-chave	138
34 Arquivos do projeto e backup	139
34.1 O que deve ser preservado	139
34.2 Mover um projeto	139
34.3 O que não editar manualmente	139
34.4 Fazer backup	140
34.5 Arquivo .inv	140
34.6 Layouts de ondas	140

34.7	Verificar um backup	140
35	Atalhos de teclado	141
35.1	Arquivos e abas	141
35.2	Navegação e comandos	141
35.3	Edição	141
35.4	Ferramentas de linguagem	141
35.5	Configuração de ondas	142
35.6	Redefinir um atalho	142
36	Solução de problemas	143
36.1	Comece por aqui	143
36.2	Interface	143
36.3	Compilação	143
36.4	Simulação e ondas	144
36.5	PRISM	144
36.6	Aurora Intelligence	144
36.7	Pedir suporte	144
X	Apêndices	145
A	Glossário	146
B	O ecossistema SAPHO	149
B.1	O laboratório	149
B.2	Os componentes	149
B.3	Como as peças se encaixam	150
B.4	Onde a plataforma é usada	151
B.5	O que fica de fora	151
B.6	Para citar a plataforma	151
C	Escopo, versão e método	152
C.1	O que foi documentado	152
C.2	Fontes e precedência	152
C.3	Convenções	153
C.4	Limites	153
	Índice	154

Primeiros passos

Escolha o fluxo de trabalho

A AURORA é o ambiente de desenvolvimento de desktop que reúne, em uma única interface, a criação e a organização de projetos, o editor de código, a compilação, a simulação, a análise de formas de onda, a visualização RTL, o controle de versão e a Aurora Intelligence.

Ela pode ser usada tanto para trabalhar diretamente com circuitos escritos em Verilog quanto para gerar processadores SAPHO a partir de algoritmos C $\pm$. Por isso, AURORA e SAPHO não são nomes equivalentes: a AURORA é a aplicação que organiza o trabalho, enquanto o SAPHO é o ecossistema de geração de processadores disponível dentro dela.

As páginas deste manual são orientadas a tarefas e acompanham a ordem normal de uso. Depois desta introdução, você pode seguir o fluxo correspondente ao tipo de projeto que pretende desenvolver.

1.1 Os fluxos disponíveis

A AURORA atende a dois fluxos principais. Você pode desenvolver um projeto diretamente em Verilog ou gerar um processador dedicado do ecossistema SAPHO a partir de um algoritmo C $\pm$, armazenado em arquivo .cmm. Os dois caminhos usam o mesmo editor, a mesma estrutura de projeto e as mesmas ferramentas de validação e análise, mas começam de pontos diferentes.

Fluxo Verilog Crie ou importe módulos RTL, defina o Top Level, adicione um testbench e siga diretamente para validação, simulação, análise de ondas e PRISM.

Fluxo completo para projetos Verilog

Fluxo de processador SAPHO Configure um processador, escreva o algoritmo C $\pm$, gere o hardware Verilog com o YANC e depois simule ou visualize o RTL produzido.

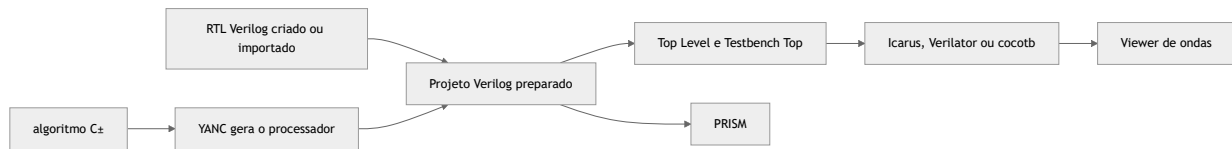
Fluxo completo para processadores SAPHO

1.2 Comparação rápida

Etapa	Fluxo Verilog	Fluxo de processador SAPHO
Entrada principal	Módulos .v ou .sv	algoritmo C $\pm$ em arquivo .cmm
Geração inicial	Escrita ou importação do RTL	Hub de Processadores e compilação C $\pm$
Hardware usado	RTL fornecido pelo usuário	RTL gerado na pasta Hardware
Top Level	Módulo principal do projeto	Módulo gerado do processador ou módulo que o instancia
Testbench Top	Testbench .v ou cocotb .py	Testbench gerado, personalizado .v ou cocotb .py
Validação Simulação	Botão Verilog Analisar Verilog (forma de onda) ou Execução rápida	Botão C$\pm$, seguido da validação Verilog quando necessária Analisar Verilog (forma de onda) , Execução rápida ou Teste do processador sintetizado
Análise	Viewer de ondas e PRISM	Viewer de ondas e PRISM

1.3 Etapas compartilhadas

Depois que o hardware Verilog está disponível, os dois fluxos convergem:



Em ambos os caminhos, o **Top Level** identifica o módulo principal do circuito e o **Testbench Top** identifica o teste que controla a simulação. A diferença está na origem do RTL: escrito ou importado no fluxo Verilog; gerado a partir de C± no fluxo SAPHO.

1.4 Qual fluxo escolher

Escolha *Fluxo completo para projetos Verilog* quando já possui módulos RTL, está aprendendo Verilog ou deseja testar um circuito digital sem gerar um processador SAPHO.

Escolha *Fluxo completo para processadores SAPHO* quando deseja transformar um algoritmo C± em uma arquitetura dedicada, configurar largura numérica e portas de entrada e saída ou trabalhar com o fluxo de geração de processadores do ecossistema SAPHO.

As páginas *Compilação: de C± a Verilog*, *Simular com Icarus, Verilator ou cocotb*, *Formas de onda: GTKWave e Surfer* e *Visualizar e simular o RTL no PRISM* detalham as ferramentas compartilhadas depois da preparação inicial.

O que é o SAPHO

O SAPHO (*Scalable-Architecture Processor for Hardware Optimization*) é uma plataforma completa para criar, programar, simular e visualizar *soft-processors*. Você escreve um programa em uma linguagem parecida com C, chamada C $\pm$ (lê-se “C mais-menos”), e a plataforma gera um processador dedicado em Verilog, feito sob medida para aquele programa e pronto para ser gravado em um FPGA. Tudo acontece dentro de um único aplicativo de desktop, a IDE AURORA, que roda localmente no Windows e não depende de nuvem.

Se a frase acima tem termos desconhecidos, esta página é o lugar certo para começar. Ela explica os conceitos na ordem em que eles importam.

2.1 O que é um soft-processor

O processador do seu computador é um circuito fixo. A arquitetura dele foi decidida na fábrica, gravada em silício, e não muda: a largura da palavra, o conjunto de instruções e o número de registradores são os que são.

Um *soft-processor* é diferente. Ele é um processador descrito em uma linguagem de descrição de *hardware* (HDL, do inglês *Hardware Description Language*), no caso o Verilog, e implementado na malha reconfigurável de um FPGA (*Field-Programmable Gate Array*), um circuito integrado cujas conexões internas podem ser reprogramadas. A arquitetura passa a ser uma escolha do projetista, que pode alterá-la e recompilá-la em minutos.

O SAPHO leva essa ideia ao extremo, com um princípio que vale a pena guardar desde já:

📌 Importante

O processador gerado contém apenas o *hardware* que o seu programa usa. Se o programa não divide, o divisor não existe no circuito final. Se não usa ponto flutuante, nenhuma lógica de ponto flutuante é gerada.

A consequência é relevante e um pouco surpreendente: o seu programa não é apenas executado sobre uma máquina pré-existente, ele *determina* a máquina. O resultado é um processador enxuto e previsível, feito para instrumentação científica e para processamento de sinais em tempo real, no qual cada elemento lógico economizado tem valor.

2.2 As peças da plataforma

O nome SAPHO designa tanto o processador quanto o guarda-chuva que reúne os componentes. Vale conhecer cada peça antes de abrir o aplicativo, porque os nomes aparecem o tempo todo na interface.

AURORA

A IDE de desktop, o programa que você instala e abre. Nela vivem o editor, o gerenciador de projetos, os botões de compilação e simulação, os terminais e os visualizadores. É a única interface gráfica do ecossistema.

YANC

Yet Another Compiler, a suíte de compiladores que trabalha por baixo dos panos. Ela traduz o programa C $\pm$ (ou C++) no processador em Verilog, nas imagens de memória e no *testbench*. Você nunca a chama diretamente.

O processador SAPHO

O circuito parametrizável que o YANC emite: acumulador único, arquitetura Harvard e *pipeline* de três estágios. Descrito em [O processador SAPHO por dentro](#).

A linguagem C±

O dialeto de C no qual você escreve o algoritmo, com números complexos como tipo nativo e álgebra linear em notação de Dirac. Arquivos com extensão `.cmm`. Descrita em [A linguagem C±: fundamentos](#).

Em volta desse núcleo orbitam as ferramentas de apoio, todas de código aberto e todas empacotadas junto com a instalação:

Icarus Verilog

O simulador padrão. Interpreta o Verilog e expõe todos os sinais internos do processador, o que faz dele o motor da depuração.

Verilator

Um simulador que converte o circuito em C++ e compila um executável nativo, de cinco a dez vezes mais rápido em simulações longas, ao custo de expor apenas os sinais do topo.

GTKWave e Surfer

Os dois visualizadores de formas de onda, nos quais você lê o comportamento do circuito ao longo do tempo. Veja [Formas de onda: GTKWave e Surfer](#).

Yosys e PRISM

O Yosys sintetiza o circuito e o PRISM (*Processor Rendering Interface for Schematic Models*) o desenha como um diagrama navegável de blocos e conexões. Veja [Visualizar e simular o RTL no PRISM](#).

cocotb

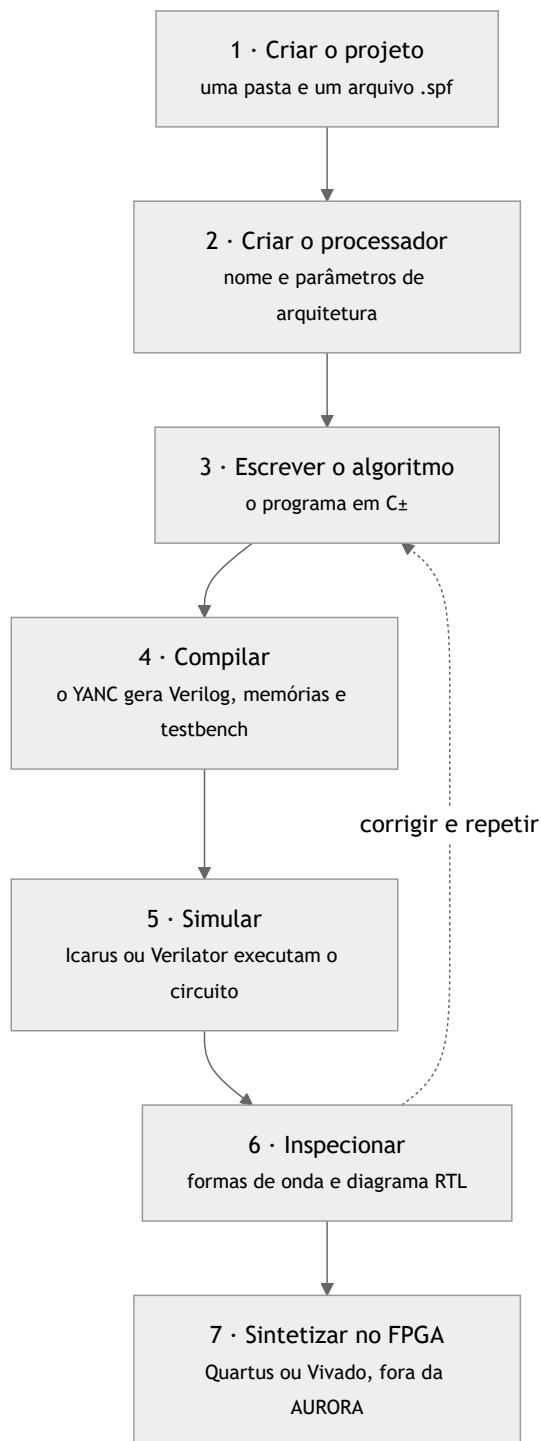
Um arcabouço que permite escrever o banco de testes inteiramente em Python, em vez de Verilog, aproveitando bibliotecas como NumPy e SciPy na verificação. Veja [Simular com Icarus, Verilator ou cocotb](#).

Aurora Intelligence

A assistente de inteligência artificial integrada, capaz de conversar sobre o projeto e de agir sobre a IDE por ferramentas com permissão controlada. Veja [Aurora Intelligence](#).

2.3 O fluxo de trabalho de ponta a ponta

O caminho de quem usa o SAPHO é sempre o mesmo, e este manual o percorre nessa ordem.



Repare na seta tracejada. Na prática você não percorre o caminho uma vez: você dá voltas nele. Escreve, compila, olha a onda, descobre que a saída satura no lugar errado, volta ao código. A agilidade dessa volta é justamente o que a AURORA existe para melhorar, reduzindo a seis ferramentas de linha de comando a um único botão.

2.4 Um exemplo condutor

Para que cada conceito apareça em contexto, este manual constrói um processador do zero e o acompanha até a forma de onda final: o `media_movel`, um filtro de média móvel que lê amostras de um sinal por uma porta de entrada, calcula a média das últimas quatro e escreve o resultado em uma porta de saída.

É um exemplo pequeno, de propósito, mas exercita o essencial: entrada e saída, aritmética, uso de vetor, laço principal e simulação com estímulo externo. Ele nasce no *Primeiro projeto: um filtro de média móvel* e reaparece nas páginas de linguagem, compilação e formas de onda.

2.5 Quem faz a plataforma

O SAPHO é desenvolvido e mantido pelo NIPS-CERN, o Núcleo de Instrumentação e Processamento de Sinais da Universidade Federal de Juiz de Fora (UFJF), em Minas Gerais. O laboratório atua em parceria com o CERN (*Conseil Européen pour la Recherche Nucléaire*) no experimento ATLAS do LHC (*Large Hadron Collider*), onde técnicas de processamento de sinais em FPGA, o habitat natural do SAPHO, são aplicadas à instrumentação dos calorímetros.

A plataforma também sustenta o ensino na disciplina de Dispositivos Lógicos Programáveis da UFJF, na qual os alunos projetam, simulam e inspecionam processadores de ponta a ponta sem depender de licenças proprietárias. Se você chegou aqui por essa disciplina, está no lugar certo.

Ver também

Para o histórico do ecossistema, os projetos que o usam e as publicações associadas, veja [O ecossistema SAPHO](#).

2.6 O próximo passo

Com o vocabulário assentado, siga para *Instalação e primeiro início*. A instalação é de um clique e traz a cadeia de ferramentas inteira embutida, de modo que nada além do instalador precisa ser baixado.

Instalação e primeiro início

A AURORA é hoje um aplicativo exclusivamente Windows, para sistemas de 64 bits. A instalação é de um clique e traz embutida toda a cadeia de compilação e simulação, de modo que nenhum outro programa precisa ser instalado antes ou depois.

3.1 Antes de instalar

Requisito	Detalhe
Sistema operacional	Windows 10 ou Windows 11, 64 bits
Espaço em disco	Cerca de 2 GB para o aplicativo e a cadeia de ferramentas, mais o espaço dos seus projetos
Permissões	Permissão de escrita na pasta escolhida para os projetos
Conexão	Necessária apenas para baixar o instalador e, depois, para as atualizações e a assistente de IA

Não é preciso instalar Python, compiladores C, simuladores Verilog nem GTKWave por fora. Tudo isso acompanha o instalador em versões validadas em conjunto.

3.2 Baixar e instalar

1. Acesse o canal oficial de distribuição em nipscern.com/sapho¹ e baixe o instalador, um arquivo chamado `sapho-aurora-Setup-vX.Y.Z.exe`, no qual X.Y.Z é a versão.
2. Dê dois cliques no executável.
3. Aguarde. O instalador é do tipo *one-click*: não há telas de opções nem escolha de pasta. Ele instala, cria os atalhos na área de trabalho e no menu Iniciar e abre o aplicativo.
4. Na primeira abertura, confira a versão instalada em *Configurações do Aurora* □ *Sobre*.

¹ <https://nipscern.com/sapho>

i Nota

O instalador ainda não é assinado digitalmente, e o Windows SmartScreen pode exibir um aviso de editor desconhecido na primeira execução. Nesse caso, clique em *Mais informações* e depois em *Executar assim mesmo*.

Durante a instalação, dois vínculos são registrados no sistema. A extensão `.spf`, dos arquivos de projeto do SAPHO, passa a abrir com a AURORA e ganha ícone próprio, de modo que dois cliques em um projeto no Explorador de Arquivos o abrem diretamente. Também é registrado o protocolo `sapho://`, reservado a integrações via navegador.

3.3 O que é instalado, e onde

Saber onde as coisas ficam ajuda tanto no diagnóstico quanto no *backup*.

Pasta do aplicativo

Recebe o executável da AURORA e a pasta `components`, que reúne a cadeia de ferramentas completa: os compiladores do YANC, os simuladores Icarus Verilog e Verilator, o Yosys, o GTKWave, o Python com cocotb e os moldes de HDL do processador.

Se essa pasta for movida ou tiver conteúdo removido, os botões de compilação passam a falhar com a mensagem de binário não encontrado. A correção é reinstalar pelo instalador oficial.

Dados do usuário

Ficam em `%APPDATA%\Aurora-IDE`:

- `logs/main.log`, o registro principal de execução, e `logs/main.old.log`, o anterior;
- a lista de projetos recentes;
- o registro de versão usado na confirmação após uma atualização;
- as conversas da Aurora Intelligence.

É o `main.log` que se anexa ao relatar um problema.

Credenciais

As chaves de API da Aurora Intelligence e o *token* do GitHub não vão para arquivo algum. São cifradas pelo cofre de credenciais do próprio Windows, a DPAPI, e usadas somente pelo processo principal da AURORA. A interface exibe apenas se existe uma chave configurada, nunca o valor.

Seus projetos

Ficam onde você quiser. A AURORA pergunta a pasta ao criar cada projeto e não impõe um diretório de trabalho. Para fazer *backup* do seu trabalho, copie a pasta completa que contém o `.spf`; veja [Projetos](#).

3.4 Primeira execução

Ao abrir, a AURORA exibe uma tela de abertura com o progresso real da inicialização e, em seguida, a tela de boas-vindas: o fundo animado de aurora boreal, os botões *Novo Projeto* e *Abrir Projeto* e a lista de projetos recentes, vazia na primeira vez.

Confirme que a instalação está saudável verificando estes cinco pontos:

- a janela principal abriu e não ficou presa na tela de carregamento;
- *Novo Projeto* e *Abrir Projeto* respondem ao clique;

- a área central do editor aparece;
- *Configurações do Aurora* abre e mostra a versão em *Sobre*;
- a barra inferior indica que a aplicação está pronta.

Cerca de seis segundos após abrir, a AURORA consulta silenciosamente se há uma versão mais nova. Se você já está na última, nada acontece. Havendo versão nova, a janela de atualização aparece, com o registro de mudanças e o tamanho do download; nada é baixado sem o seu aval.

3.5 Atualizações

Manter o SAPHO em dia não exige nada além de aceitar as atualizações quando oferecidas. Ao aceitar, a barra de progresso mostra percentual, velocidade e tempo restante, e é possível continuar trabalhando durante o download. Com o pacote pronto, o botão vira *Reiniciar e instalar*: a AURORA fecha, o instalador roda e a IDE reabre na versão nova.

A integridade do pacote é verificada por resumo criptográfico antes de instalar, e cada instalador carrega a versão da cadeia de ferramentas testada com aquela versão da IDE. Atualizar a AURORA atualiza o conjunto inteiro, em versões validadas juntas, sem nada para gerenciar manualmente.

i Dois repositórios, por desenho

O desenvolvimento acontece no repositório `nipscernlab/aurora`, e as versões estáveis são publicadas em `nipscernlab/sapho`, que é o canal consumido pelo instalador e pelo atualizador. Você sempre recebe versões estáveis, nunca o estado intermediário do desenvolvimento.

3.6 Se a AURORA não iniciar

1. Aguarde alguns segundos e confirme que a preparação inicial não está apenas em andamento.
2. Verifique se outra janela da AURORA já está aberta.
3. Reinicie o aplicativo pelo menu Iniciar.
4. Se o problema persistir, anote a versão do Windows e a etapa em que a abertura parou e anexe o `%APPDATA%\Aurora-IDE\logs\main.log` ao relato.

Consulte [Solução de problemas](#) para uma investigação orientada por sintomas.

3.7 Desinstalação

Use as Configurações do Windows, em Aplicativos, e desinstale o item Aurora-IDE. Os projetos criados por você não são apagados: eles pertencem às pastas que você escolheu.

3.8 O próximo passo

Com o aplicativo aberto, siga para [Tour pela interface](#) e conheça as regiões da janela, ou vá direto ao [Primeiro projeto: um filtro de média móvel](#) se preferir aprender construindo.

Tour pela interface

A janela da AURORA não tem a moldura padrão do Windows nem uma barra de menus tradicional. Toda a navegação vive em uma barra de título personalizada, no painel lateral da árvore de arquivos, no editor, nos terminais e na barra de status. Esta página apresenta cada região e o que esperar dela; o uso detalhado fica nas páginas dedicadas.

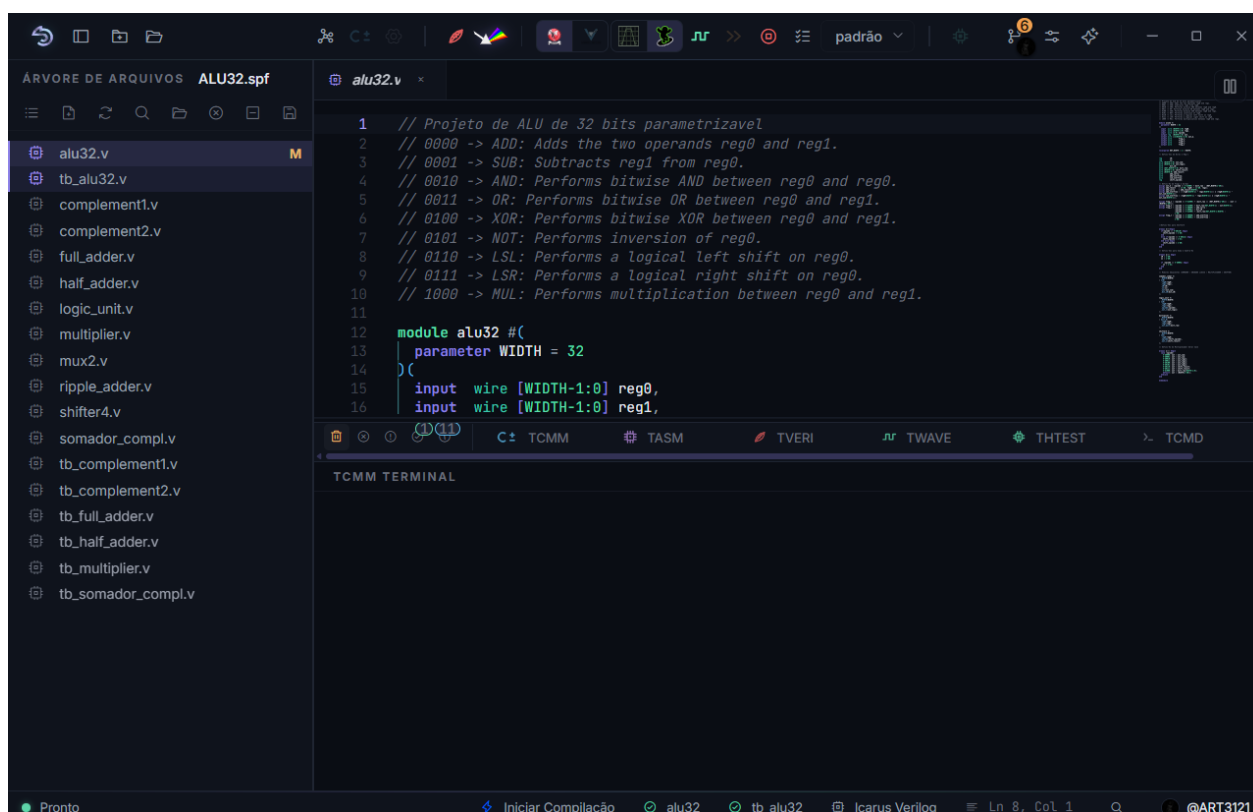


Figura4.1: As cinco regiões da janela principal: a barra superior, a árvore do projeto à esquerda, o editor ao centro, os terminais embaixo e a barra de status no rodapé.

Antes de percorrer cada uma, guarde o mapa geral:

Região	Para que serve
Barra superior	Criar e abrir projetos, criar processadores, compilar, simular, abrir o PRISM, acionar a IA e as configurações
Árvore lateral	Abrir arquivos, alternar entre três visões do projeto e definir os papéis de <i>top-level</i> e <i>test-bench</i>
Editor	Escrever C++, Verilog, SystemVerilog, Python e arquivos auxiliares
Terminais	Ler as mensagens de cada etapa da cadeia, uma aba por etapa
Barra de status	Conferir, antes de agir, qual processador está ativo e o que ainda falta

O fluxo normal começa na árvore ou no editor, continua por um botão da barra superior e produz mensagens nos terminais. Antes de executar uma ação, confira a barra de status para não compilar ou simular o arquivo errado.

4.1 A barra superior

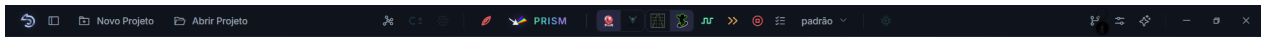


Figura4.2: A barra superior organiza-se em três zonas: identidade e projeto à esquerda, a cadeia do processador ao centro, apoios e controles de janela à direita.

À esquerda ficam o ícone do SAPHO, o botão que mostra ou esconde a barra lateral e os botões *Novo Projeto* e *Abrir Projeto*.

O centro concentra a cadeia do processador, disposta na ordem em que ela é usada. Vale ler a tabela abaixo uma vez com calma: ela responde à pergunta mais comum de quem está começando, que é por que um botão está apagado.

Tabela4.1: Botões da barra superior, na ordem em que aparecem

Botão	O que faz	Quando fica disponível
<i>Hub de Processadores</i>	Abre o formulário que cria um processador SAPHO	Com um projeto aberto
<i>Compilar C±</i>	Roda a cadeia YANC e gera o Verilog, as memórias e o <i>testbench</i>	Com um arquivo <i>.cmm</i> aberto no editor
<i>Configurações de simulação do processador</i>	Ajusta <i>clock</i> , número de ciclos e exibição de vetores nas ondas	Com um processador ativo
<i>Sintetizar Verilog</i>	Verifica sintaxe e elaboração dos arquivos Verilog	Com <i>top-level</i> definido
<i>Abrir PRISM</i>	Sintetiza com o Yosys e abre o diagrama RTL	Com <i>top-level</i> definido
Seletor de simulador	Alterna entre Icarus Verilog e Verilator, para o aplicativo inteiro	Sempre
Seletor de visualizador	Alterna entre GTKWave e Surfer	Sempre
<i>Analisar Verilog (forma de onda)</i>	Roda a simulação e abre o visualizador com o resultado	Com <i>testbench</i> definido
<i>Execução rápida</i>	Simula sem gravar ondas, para quando só interessam as saídas	Com <i>testbench .py</i> , ou <i>.v</i> com Verilator selecionado
<i>Teste do processador sintetizado</i>	Exercita apenas as portas de entrada e saída, como caixa-preta	Com um processador ativo e <i>hardware</i> já gerado
<i>Configuração de ondas</i>	Escolhe quais sinais a simulação vai gravar	Com um projeto aberto
<i>Cancelar</i>	Interrompe todos os processos em andamento, sem fechar a IDE	Durante uma compilação ou simulação

À direita ficam os apoios: *Controle de Versão*, que abre o painel Dagr; *Configurações do Aurora*; *Assistente IA*, a estrela que abre a Aurora Intelligence; e os botões de minimizar, maximizar e fechar. Um duplo clique na área vazia da barra também maximiza ou restaura a janela.

Dica

Todo botão tem uma dica detalhada ao passar o mouse. Se as dicas atrapalharem depois que a interface ficar familiar, desligue-as em *Configurações do Aurora* □ *Geral*.

4.2 A árvore de arquivos

À esquerda fica o painel da árvore, com o nome do projeto aberto junto ao rótulo. O mesmo painel abriga três visões do mesmo projeto, alternadas por um único botão no cabeçalho.

Um clique em um arquivo o abre no editor como aba de visualização; um clique duplo fixa a aba. Nas pastas, o clique expande ou recolhe.

O clique com o botão direito abre o menu de contexto, que é onde se atribuem os dois papéis mais importantes do projeto: *Definir como Top Level*, no módulo Verilog que integra o circuito, e *Marcar como Testbench*, no arquivo que estimula a simulação. Esses dois papéis são detalhados em [Arquivos Verilog e Testbenches](#).

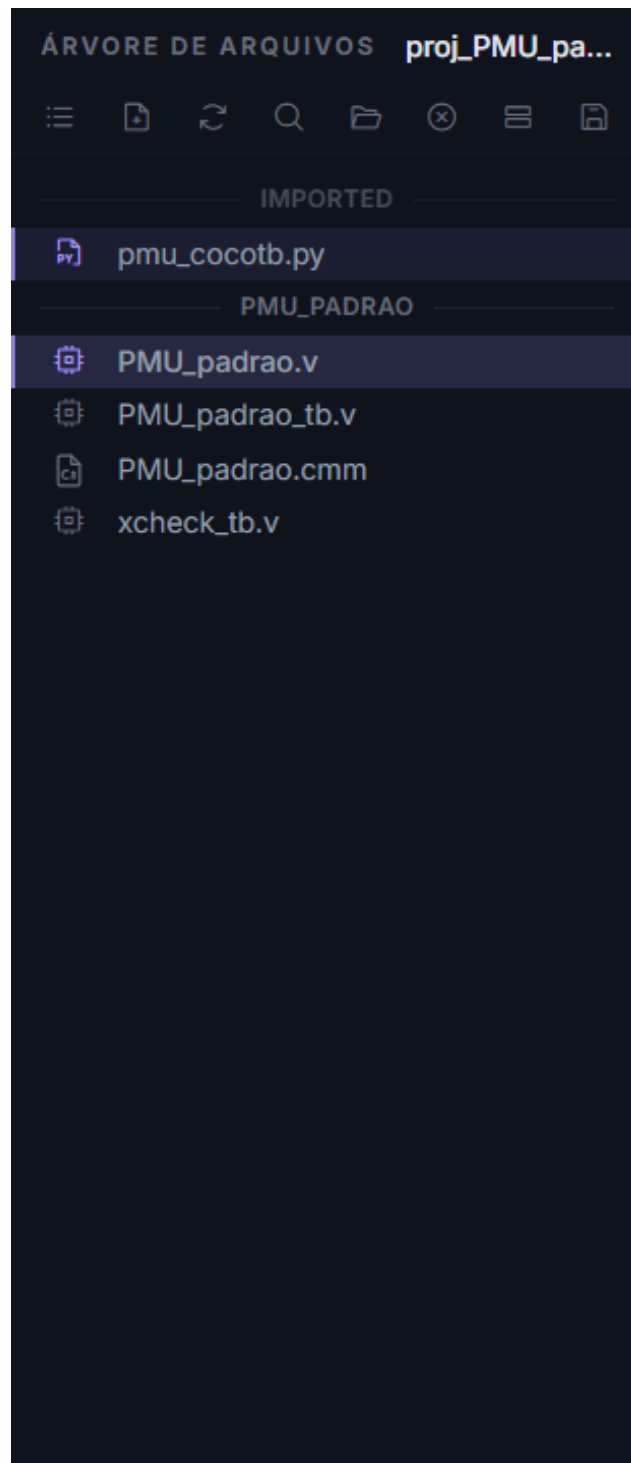


Figura4.3: Arquivos, a visão de trabalho: agrupada por processador, com os Verilog separados entre sintetizáveis e *testbenches*.



Figura4.4: Hierarquia, disponível após uma síntese: a árvore de instâncias de módulos como o Yosys a enxerga.

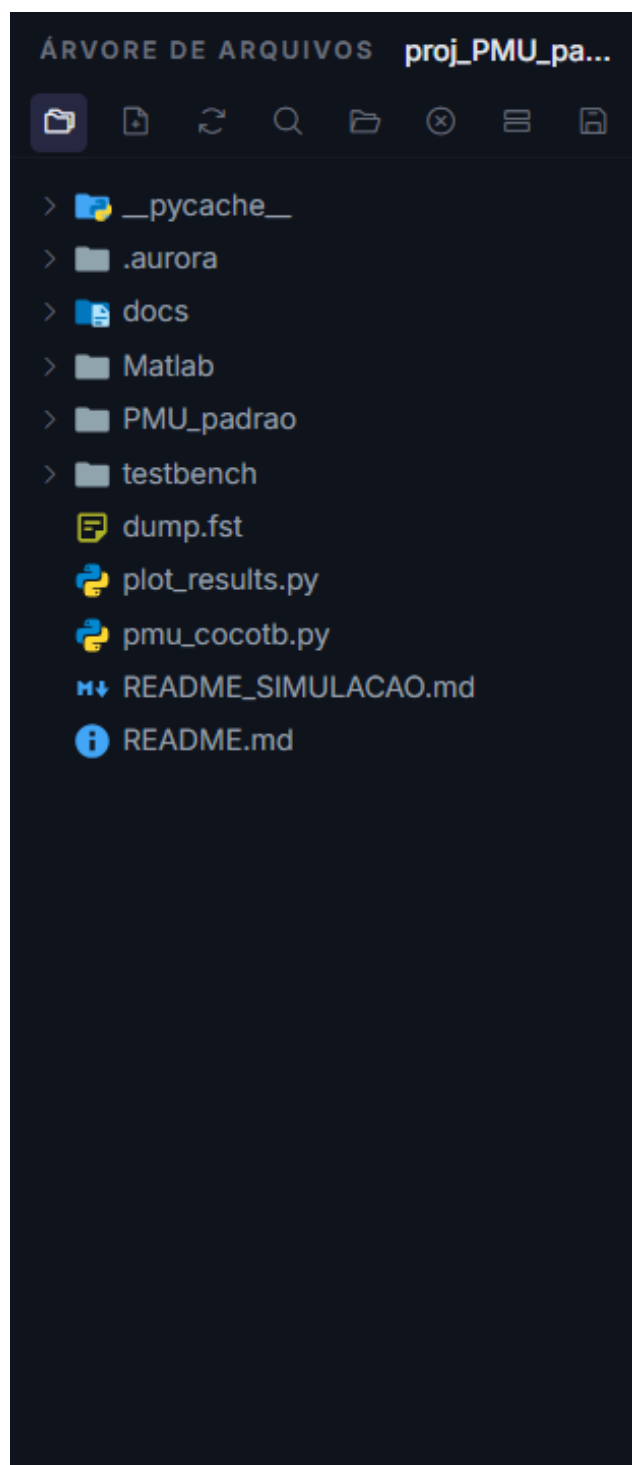


Figura4.5: Pastas, um explorador de diretórios completo, com menu de contexto para criar, renomear e excluir.

O cabeçalho da árvore reúne ainda os botões de novo arquivo, atualização, busca nos arquivos (**Ctrl+Shift+F**), abertura no explorador do sistema, *backup* do projeto em *.zip*, recolhimento da árvore e fechamento do projeto.

i Nota

A árvore aceita a importação de arquivos *.v*, *.sv*, *.vh* e *testbenches* cocotb *.py* por arrastar e soltar. Outros tipos são recusados com uma dica; arquivos *.gtkw*, por exemplo, entram pelo seletor próprio da barra superior.

4.3 O editor

O centro da janela pertence ao editor, construído sobre o Monaco, o mesmo motor do Visual Studio Code. Abas múltiplas, divisão em até três painéis, realce de sintaxe para C $\pm$, *assembly*, Verilog e SystemVerilog, diagnósticos em tempo real e formatação automática vêm de fábrica. A página [Editor, abas e painéis](#) é dedicada a ele.

4.4 Os terminais

A área inferior reúne seis terminais em abas. Cinco são consoles de saída, cada etapa da cadeia escrevendo no seu, o que torna previsível onde procurar cada mensagem. O sexto é um *shell* PowerShell interativo de verdade.

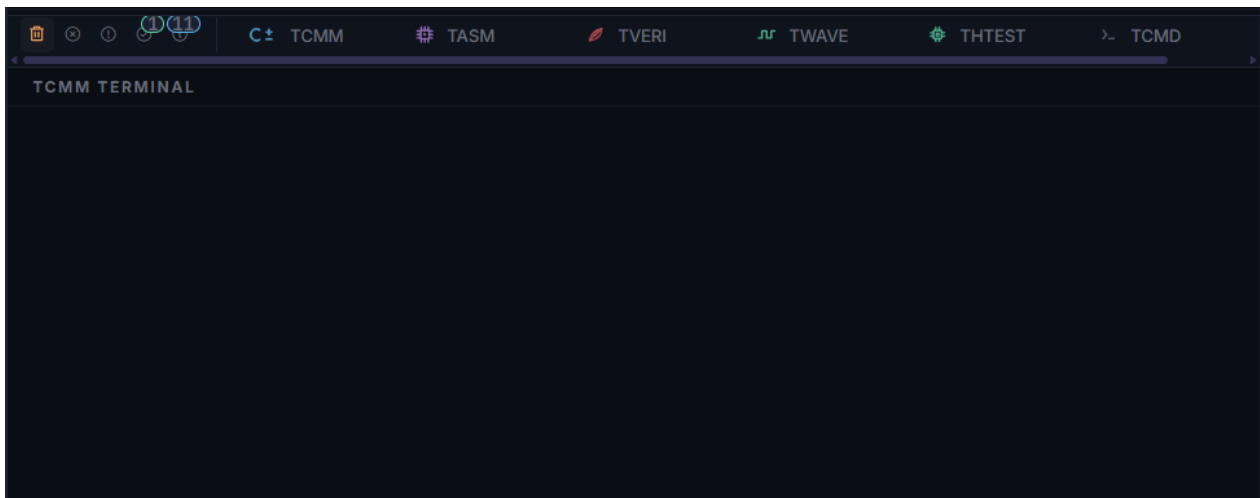


Figura4.6: Cada ação direciona a saída para o terminal correspondente, e a AURORA troca de aba sozinha conforme as etapas avançam.

Quem escreve onde: **TCMM** recebe o compilador C $\pm$; **TASM** recebe o montador, a geração do Verilog e os avisos de recurso instanciado; **TVERI** concentra o Icarus Verilog e o Yosys; **TWAVE** acompanha a simulação e a abertura dos visualizadores; **THTEST** é dedicado ao teste do processador sintetizado; e **TCMD** é o *shell* integrado. Detalhes em [Os terminais](#).

4.5 A barra de status

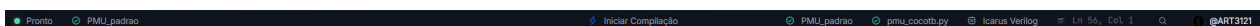


Figura4.7: A barra de status resume, em uma linha, tudo o que a próxima compilação ou simulação vai usar.

Da esquerda para a direita: o indicador Pronto ou Não Pronto, cujo ponto verde ou vermelho resume se o

projeto está em condições de compilar; o nome do processador ativo, deduzido do arquivo `.cmm` em foco no editor; ao centro, o botão *Iniciar Compilação*; e à direita os avisos de estrutura, o motor de simulação selecionado, a posição do cursor e o indicador do GitHub.

🔔 Importante

Consulte a barra de status antes de compilar, simular ou abrir o PRISM. A maior parte dos “não funcionou” de quem está começando é, na verdade, o arquivo errado em foco no editor.

4.6 Paleta de comandos e notificações

A paleta de comandos, aberta com `Ctrl+Shift+P`, oferece busca difusa sobre os comandos da IDE, organizados em grupos. É o caminho mais curto quando se sabe o nome da ação, mas não onde fica o botão.

Eventos como a criação de arquivos ou a troca de simulador aparecem como notificações discretas no canto da janela.

4.7 Quando um botão estiver desabilitado

Verifique nesta ordem, que é a ordem das dependências:

1. há um projeto aberto;
2. o arquivo certo está aberto e em foco no editor;
3. o processador ativo, na barra de status, é o que você quer compilar;
4. o *top-level* foi definido, se a ação exige síntese;
5. o *testbench* foi definido, se a ação exige simulação;
6. nenhuma execução anterior continua em andamento.

Na quase totalidade dos casos, um botão apagado significa que falta uma dessas seleções. Salve o arquivo ativo e aguarde a conclusão de qualquer operação anterior antes de tentar de novo.

4.8 O próximo passo

Agora que a janela não é mais estranha, construa algo nela: *Primeiro projeto: um filtro de média móvel*.

Projetos e ferramentas

Projetos

Um projeto reúne o arquivo de configuração `.spf`, os algoritmos C $\pm$, os módulos Verilog, os testbenches e os resultados gerados. Esta página explica como criar, abrir, mover e preservar esse conjunto sem quebrar referências.

5.1 Criar

1. Clique em **Novo Projeto**.
2. Informe um nome.
3. Escolha a pasta de destino.
4. Confirme.

O nome do projeto é usado na pasta e no arquivo `.spf`. Escolha um nome curto, descritivo e sem caracteres que possam ser rejeitados pelo Windows. Depois da confirmação, aguarde a árvore do projeto aparecer antes de criar processadores ou importar arquivos.

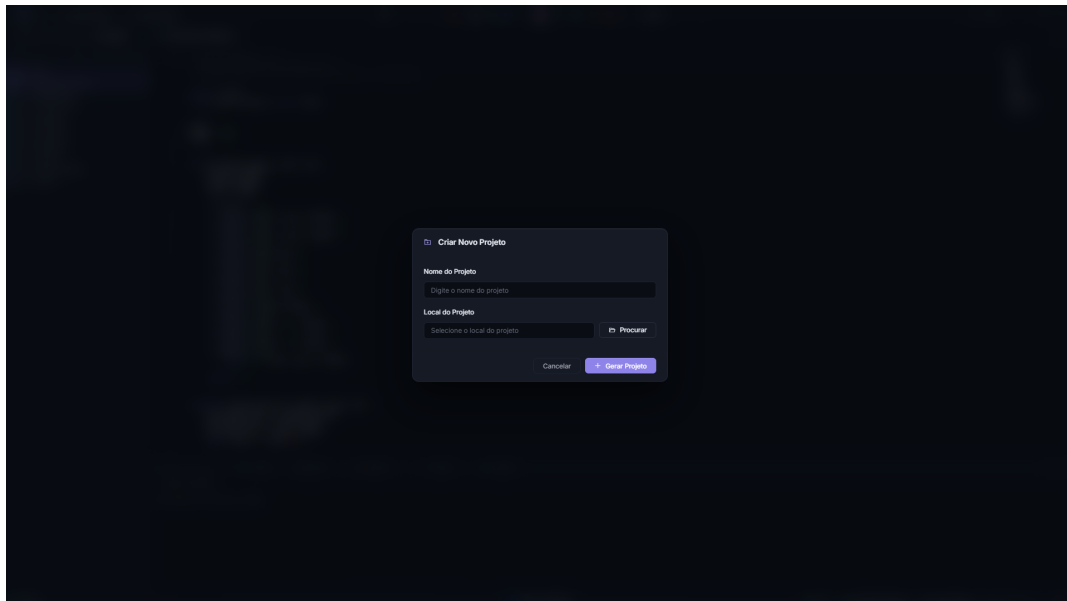


Figura5.1: Preencha o nome, escolha a pasta de destino em **Procurar** e confirme em **Gerar Projeto**.

5.2 Abrir

Clique em **Abrir Projeto** e selecione o arquivo `.spf`. Projetos usados recentemente também aparecem na tela inicial. Ao abrir, a AURORA restaura a estrutura registrada e verifica os caminhos dos arquivos associados.

Se um projeto recente foi movido ou excluído, abra-o novamente pela nova localização. Uma entrada recente não contém uma cópia do projeto; ela é apenas um atalho para o `.spf` existente no disco.

5.3 O que é o arquivo .spf

O .spf guarda a configuração do projeto, incluindo:

- Processadores.
- Arquivos Verilog e Testbenches.
- Top Level e Testbench Top.
- Caminhos de arquivos importados.

Não edite esse arquivo manualmente durante o uso normal. Para mover um projeto, feche-o na AURORA, copie a pasta completa e abra o .spf na nova localização. Depois, confirme se Top Level, Testbench Top e arquivos importados continuam válidos.

5.4 Navegar pela pasta do projeto

Use a visualização **Pastas** para conferir a estrutura real no disco. Ela mostra pastas e arquivos do projeto mesmo quando eles não fazem parte da lista Verilog registrada no .spf.

A visualização **Pastas** é útil para:

- Abrir arquivos auxiliares sem sair da AURORA.
- Conferir resultados gerados em Hardware/, Simulation/ e testbench/.
- Revelar ou expandir pastas do projeto.
- Ver badges Git junto dos arquivos no disco.

Arquivos ocultos, metadados internos e alguns arquivos de configuração histórica não aparecem nessa view para reduzir ruído.

5.5 Filtrar a visualização Pastas com .inv

Crie um arquivo .inv na raiz do projeto para esconder arquivos e pastas somente na navegação da AURORA. A sintaxe é inspirada em .gitignore: aceita comentários, linhas vazias, negação com !, padrões de diretório, *, ?, ** e comparação sem diferenciar maiúsculas de minúsculas.

Exemplo:

```
# Esconder resultados locais volumosos.  
Simulation/tmp/  
*.log  
  
# Manter um relatório visível.  
!Simulation/relatorio.log
```

O .inv não altera Git, não muda o .spf e não impede que um arquivo seja aberto por caminho direto. Ao salvar o .inv, a visualização **Pastas** é atualizada.

5.6 Importar arquivos

Você pode usar arquivos dentro ou fora da pasta do projeto.

1. Adicione o arquivo pelo comando de importação ou arraste e solte um ou vários arquivos sobre a árvore **Arquivos**.
2. Confirme se ele é fonte sintetizável ou testbench.
3. Defina Top Level ou Testbench Top quando necessário.

O recurso de arrastar e soltar aceita os formatos suportados pelo fluxo HDL: Verilog e SystemVerilog (.v, .sv e .vh) e testbenches Python/cocotb (.py). A AURORA registra os arquivos importados no .spf; ela não transforma automaticamente um formato desconhecido em fonte compilável.

Para facilitar backup e compartilhamento, prefira manter as fontes dentro da pasta do projeto.

Arquivos externos permanecem dependentes do caminho original. Se o projeto for enviado para outra máquina sem esses arquivos, a árvore indicará referências ausentes e as etapas de compilação poderão falhar.

5.7 Fechar e reabrir

Use a ação **Fechar Projeto** na árvore. Salve arquivos modificados antes de fechar.

Ao reabrir, confira na barra de status:

- Processador ativo.
- Top Level.
- Testbench Top.
- Simulador selecionado.

Abra também um ou dois arquivos importantes para confirmar que os caminhos foram restaurados. Essa verificação é especialmente importante depois de mover a pasta ou restaurar um backup.

5.8 Backup

Use a ação de backup antes de:

- Renomear ou excluir processadores.
- Importar muitos arquivos.
- Alterar a estrutura Verilog.
- Atualizar a AURORA.

O backup não substitui um sistema de versionamento, mas permite voltar rapidamente a um estado anterior. Identifique a cópia com uma data ou uma descrição da mudança para evitar confundi-la com a versão ativa.

5.9 Renomear ou excluir

Use os comandos da AURORA para renomear ou excluir projetos, processadores e arquivos registrados. Alterar apenas nomes de pastas pelo Explorer pode deixar referências inválidas no .spf.

5.10 Versionamento

Para acompanhar alterações ao longo do tempo, mantenha a pasta do projeto em Git. A AURORA pode exibir decorações nas árvores e operar status, diff, stage, commit, branch, stash, pull e push pelo painel de Source Control. Veja [Source Control e Git](#).

Editor, abas e painéis

O editor é a área de trabalho para algoritmos C $\pm$, módulos Verilog, testbenches e arquivos auxiliares. Esta página explica como organizar arquivos, salvar mudanças e comparar conteúdos sem perder o contexto do projeto.

6.1 Abrir arquivos

Clique duas vezes em um arquivo da árvore. Arquivos de texto são abertos em uma aba; imagens e conteúdos reconhecidos usam visualizadores próprios.

O editor oferece realce para C $\pm$, Assembly, Verilog, Python, JSON e outros formatos comuns. O realce ajuda na leitura, mas não garante que o conteúdo esteja correto; use as etapas de compilação e simulação para validar o arquivo.

Use **Ctrl+N** para criar um documento vazio chamado `Untitled-N`. A AURORA detecta o tipo pelo conteúdo e sugere a extensão correspondente quando o arquivo é salvo.

6.1.1 Criar um processador SAPHO com `$cmm`

1. Abra um documento vazio com **Ctrl+N**.
2. Digite somente `$cmm`.
3. Aguarde a expansão automática do modelo C $\pm$.
4. Use **Ctrl+S** e informe o nome do processador.

A AURORA substitui `$cmm` pelas diretivas padrão e pelo corpo inicial de um algoritmo C $\pm$. Ao salvar dentro de um projeto aberto, ela usa o nome escolhido em `#PRNAME`, registra o processador no `.spf` e cria a estrutura `<processador>/Software`, `<processador>/Hardware` e `<processador>/Simulation`. O arquivo `.cmm` é gravado em `Software` e passa a aparecer como processador do projeto.

6.2 Abas

- Clique em uma aba para ativá-la.
- Use **Ctrl+W** para fechar a aba atual.
- Use **Ctrl+Shift+T** para reabrir a última aba fechada.
- O marcador na aba indica alterações ainda não salvas.

Ao fechar uma aba modificada, leia a confirmação antes de descartar o conteúdo. Fechar a aba não remove o arquivo do projeto nem do disco.

6.3 Dividir o editor

Use o botão de divisão para abrir até três painéis. Isso é útil para comparar módulos, manter o testbench ao lado do RTL ou consultar um arquivo enquanto edita outro.

Se o mesmo arquivo estiver aberto em dois painéis, a alteração aparece nos dois porque ambos representam o mesmo documento. Use a divisão para consultar regiões diferentes ou comparar arquivos relacionados, não para criar versões independentes do mesmo conteúdo.

6.4 Localizar e navegar

Atalho	Ação
Ctrl+F	Localizar no arquivo
Ctrl+H	Localizar e substituir
Ctrl+G	Ir para uma linha
F1	Abrir a paleta de comandos do Monaco
Ctrl+Shift+P	Abrir a paleta de comandos da AURORA
F12	Ir para definição quando disponível

6.5 Salvar

- Atalho Ctrl+S: Salva o arquivo ativo.
- Atalho Ctrl+Shift+S: Salva todos os arquivos modificados.

Antes de compilar, confirme que o marcador de alteração desapareceu da aba.

Salvar antes da compilação é importante porque as ferramentas externas leem o conteúdo gravado no disco. Uma alteração visível no editor, mas ainda não salva, não fará parte da execução.

6.6 Usar a seleção com IA

Selecione um trecho de código para enviá-lo à Aurora Intelligence com o caminho e a linguagem do arquivo. Um pedido específico, como “explique por que este sinal nunca muda”, produz um contexto melhor do que uma solicitação genérica. Peça primeiro uma explicação ou revisão. Alterações propostas pela IA exigem confirmação antes de serem aplicadas.

6.7 Mudanças externas

Quando o arquivo muda no disco enquanto possui alterações locais, escolha conscientemente qual versão manter. Em caso de dúvida, copie o trecho local antes de recarregar.

Depois de resolver o conflito, salve o arquivo e verifique se a aba deixou de indicar modificação. Em seguida, compile novamente para garantir que a versão correta foi utilizada.

Os terminais

A área inferior da janela reúne seis terminais em abas. Cinco são consoles de saída, cada etapa da cadeia escrevendo no seu, o que torna previsível onde procurar cada mensagem. O sexto é um *shell* interativo de verdade.

Quando você dispara uma ação na barra superior, a AURORA troca automaticamente para a aba certa conforme as etapas avançam. Saber quem escreve onde é o que transforma uma falha opaca em um diagnóstico de trinta segundos.

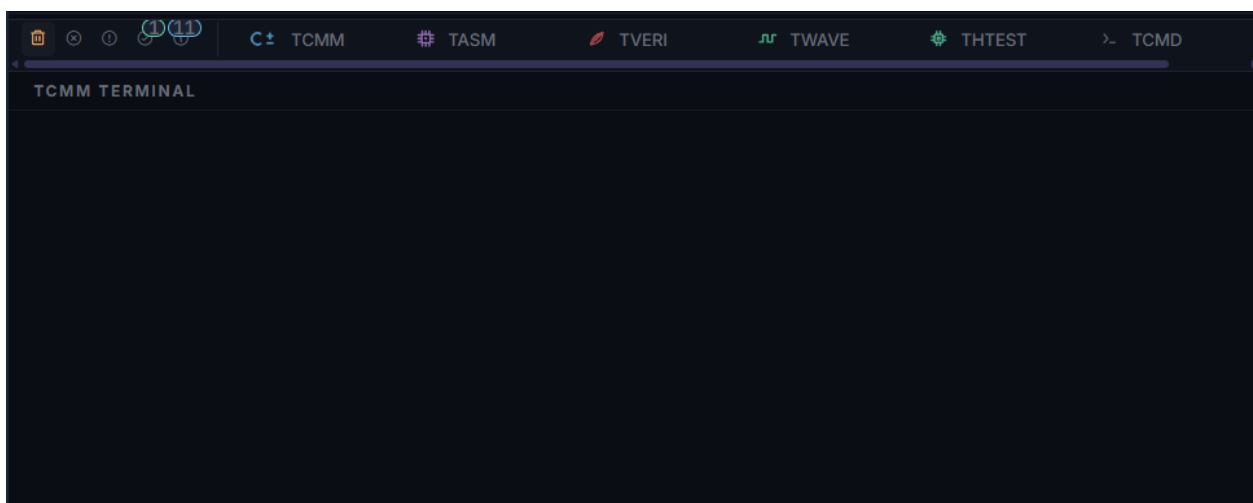


Figura 7.1: Os seis terminais. Cada etapa escreve no seu, e a AURORA seleciona a aba correspondente sozinha durante a execução.

7.1 Quem escreve onde

Aba	Recebe
TCMM	A saída do compilador C±: erros, avisos e o progresso da tradução para <i>assembly</i>
TASM	A saída do montador, a geração do Verilog e das memórias e os avisos de recurso instanciado
TVERI	O Icarus Verilog e a síntese do Yosys, com os diagnósticos de Verilog
TWAVE	A execução da simulação, seja pelo vvp, pelo Verilator ou pelo cocotb, e a abertura dos visualizadores
THTEST	O teste do processador sintetizado, com barra de progresso própria
TCMD	Um terminal PowerShell interativo completo

A ordem acima é também a ordem temporal de uma execução completa. Se algo falhou, procure o primeiro terminal da sequência que registrou erro: os seguintes costumam mostrar apenas consequências.

7.2 Cartões, filtros e o modo verboso

As mensagens chegam como cartões classificados em erros, avisos, sucessos e dicas, no mesmo código de cores da IDE. No topo da área ficam os filtros por tipo, com contadores, além dos botões de limpar o terminal atual, exportar o conteúdo para arquivo e recarregar a interface.

Os filtros atuam quando o Modo Verboso está desligado nas configurações. Com o modo ligado, que é o padrão, tudo aparece. Cada terminal retém até cinco mil entradas.

Dica

Linhas clicáveis Erros que citam arquivo e linha, como `arquivo.v:42:` do Icarus ou “linha 17” do compilador C $\pm$, viram *links*. O clique abre o arquivo no editor, na linha exata. É o caminho mais rápido do erro à correção.

7.3 Ler os avisos de recurso instanciado

O terminal TASM tem uma função que nenhum outro ambiente oferece: ele anuncia, durante a montagem, cada bloco de *hardware* que o seu programa acabou de ligar no circuito.

À medida que os *opcodes* aparecem pela primeira vez, o TASM informa a instância correspondente: o divisor inteiro, o bloco de módulo, os blocos de ponto flutuante, o circuito de endereçamento indexado. Ao final, resume o percentual do conjunto de instruções e da unidade lógica e aritmética efetivamente usados.

Importante

Leia esse resumo com atenção depois de cada mudança relevante no algoritmo. Ele é a medida mais direta do custo em área do seu programa, e a diferença entre duas compilações mostra exatamente o que uma alteração de uma linha custou.

O assunto está desenvolvido em [Recursos avançados](#).

7.4 O TCMD, o shell integrado

A aba TCMD é um terminal PowerShell real, com um *prompt* próprio da AURORA. Digite qualquer comando, como `git`, `python` ou `cd`, e tecla Enter. O diretório atual e as variáveis de ambiente persistem entre comandos, como em um terminal comum.

Ele abre no diretório do projeto e acompanha o contexto do projeto ou processador ativo. O menu de contexto da árvore de arquivos oferece *Abrir no Terminal Integrado*, que muda a sessão para a pasta escolhida.

É também esse *shell* que a Aurora Intelligence usa quando você autoriza a ferramenta de execução de comandos, conforme [Tarefas com a Aurora Intelligence](#).

Nota

Use o TCMD para tarefas auxiliares, como Git avançado ou *scripts* Python que analisem os `output_*.txt`. Para compilar e simular, prefira os botões da barra superior, que cuidam de argumentos, caminhos e ordem das etapas por você.

⚠ Aviso

Os comandos são executados com as suas permissões de usuário. Confira o diretório exibido no *prompt* antes de alterar ou remover arquivos.

7.5 Uma decisão de segurança que explica um limite

A AURORA só executa binários da própria cadeia de ferramentas, validados contra uma lista fechada, e os botões não passam por um *shell*. É por isso que a IDE não oferece a execução de comandos arbitrários fora do TCMD.

Se você esperava um campo de opções de linha de comando em algum botão e não o encontrou, esta é a razão.

7.6 Exportar para relatar um problema

O botão de exportação salva o conteúdo do terminal atual em arquivo. Junto com o registro principal em %APPDATA%\Aurora-IDE\logs\main.log, é o que se anexa ao relatar um problema nos repositórios da organização nipscernlab.

7.7 Leitura relacionada

- *Compilação: de C $\pm$ a Verilog* explica o que cada etapa da cadeia produz.
- *Solução de problemas* reúne sintomas e soluções por sintoma.
- *Configurações do aplicativo* mostra onde ligar e desligar o Modo Verboso.

Source Control e Git

O painel de **Controle de Versão** integra as operações Git mais frequentes ao projeto aberto na AURORA. Ele permite iniciar um repositório, revisar alterações, criar commits, sincronizar com o GitHub e clonar projetos associados à conta conectada.

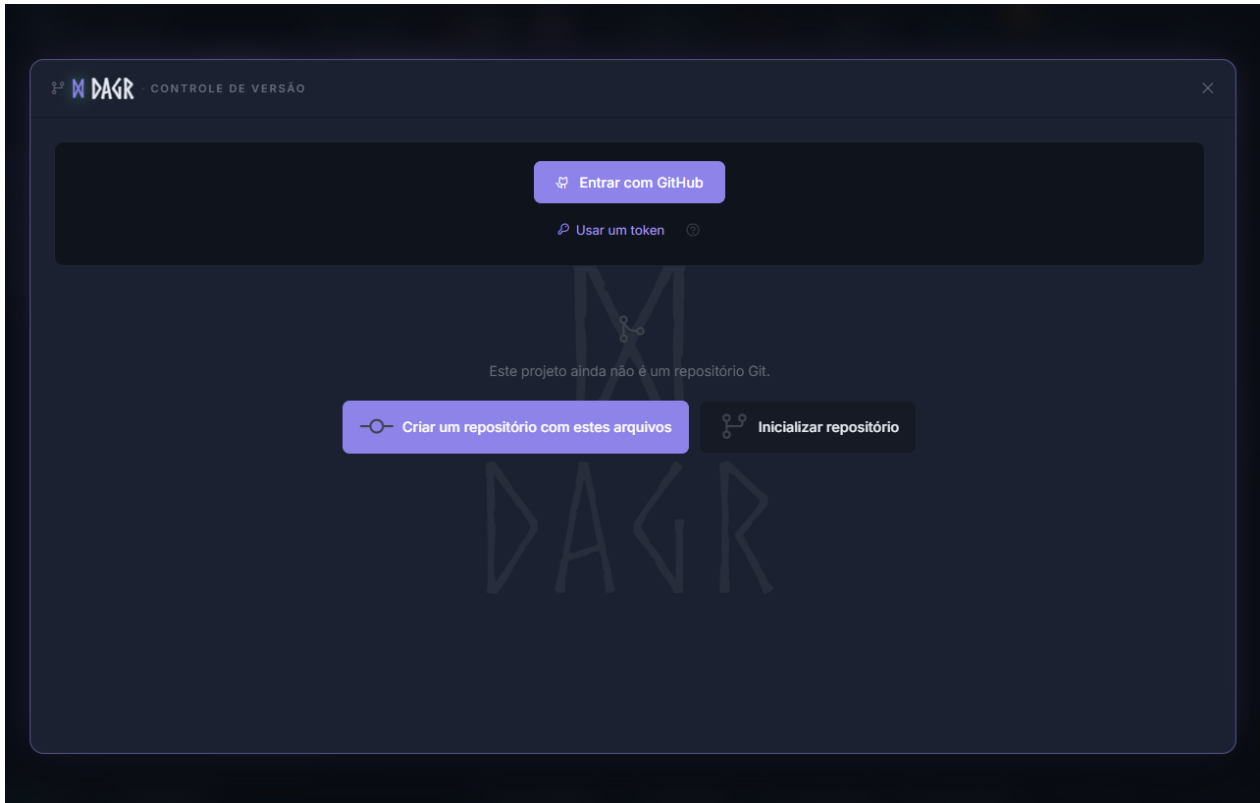


Figura8.1: Sem uma conta conectada e antes da inicialização local, o painel oferece **Criar um repositório com estes arquivos** e **Inicializar repositório**, sem exibir nomes, avatares ou repositórios pessoais.

8.1 Começar com o projeto aberto

Quando a pasta do projeto ainda não contém um repositório Git, o painel oferece duas ações:

Criar um repositório com estes arquivos

Inicializa o Git na pasta do projeto, prepara os arquivos, cria o commit inicial e conduz a publicação em um novo repositório remoto quando a conta do GitHub está conectada.

Inicializar repositório

Cria apenas o repositório Git local. Use esta opção para versionar o projeto sem publicá-lo imediatamente ou para configurar o remoto depois.

O repositório usa a pasta que contém o `.spf`; portanto, fontes, testbenches e demais arquivos do projeto podem ser acompanhados em conjunto. Revise a lista antes do primeiro commit para evitar incluir resultados volumosos ou credenciais.

8.2 Alterações e commits

Na aba **Alterações**, a AURORA separa arquivos modificados, novos e preparados. Clique em um arquivo para abrir o diff e use as caixas de seleção para controlar o próximo commit.

Controle	O que faz
Preparar / Tirar do stage	Inclui ou remove um arquivo do próximo commit
Preparar tudo / Despreparar tudo	Aplica a seleção a todos os arquivos listados
Resumo do commit	Define a mensagem que identifica a alteração
Commit	Registra os arquivos preparados no histórico local
Amend	Atualiza o commit mais recente com a seleção e a mensagem atuais
Desfazer último commit	Retorna o último commit para alterações locais, conforme a confirmação exibida
Atualizar	Recarrega status, branch e diferenças do repositório

Use a aba **Histórico** para consultar commits, autores e arquivos alterados. Ao abrir um commit, a interface carrega o diff de cada arquivo sob demanda.

8.3 Sincronização e branches

Controle	O que faz
Fetch	Consulta o estado do remoto sem integrar as mudanças
Pull	Traz e integra os commits do remoto à branch atual
Push	Envia os commits locais ao remoto configurado
Publicar	Cria ou associa o repositório remoto quando ele ainda não existe
Branches	Troca de branch, cria uma branch, acompanha branches remotas e faz merge
Alterações guardadas	Restaura ou descarta um stash criado durante uma troca de branch

Se uma troca de branch sobrescreveria alterações locais, a AURORA pode guardá-las em stash antes da troca. Leia as confirmações antes de merge, restauração ou descarte, pois essas operações alteram o conteúdo do projeto no disco.

8.4 Conectar e clonar do GitHub

Conecte uma conta do GitHub no próprio painel para listar os repositórios aos quais ela tem acesso. Na aba **Clonar**:

1. Escolha um repositório da conta ou das organizações disponíveis.
2. Selecione a pasta de destino.
3. Clique em **Clonar**.
4. Aguarde a AURORA procurar arquivos `.spf` no conteúdo baixado.
5. Abra o projeto encontrado ou consulte-o em **Projetos**.

A lista **Projetos** oferece ações para abrir o projeto na AURORA, acessar o repositório no GitHub, abrir o terminal, mostrar a pasta no Explorer ou remover a entrada da lista. Remover a entrada não deve ser confundido com excluir o repositório do disco; leia a confirmação apresentada pela interface.

A integração pode usar token armazenado de forma segura ou OAuth, conforme a configuração da instalação. Nunca inclua tokens em arquivos do projeto, capturas de tela ou commits.

8.5 Usar .inv sem alterar o Git

O `.inv` é um filtro exclusivo da visualização **Pastas**. Ele não remove arquivos, não muda o `.spf`, não altera o status Git e não substitui `.gitignore`.

O exemplo abaixo foi aplicado ao projeto `porta_AND`:

```
# Oculta artefatos locais apenas na visualização Pastas da AURORA.  
__pycache__/  
testbench/  
arquivo.inv
```

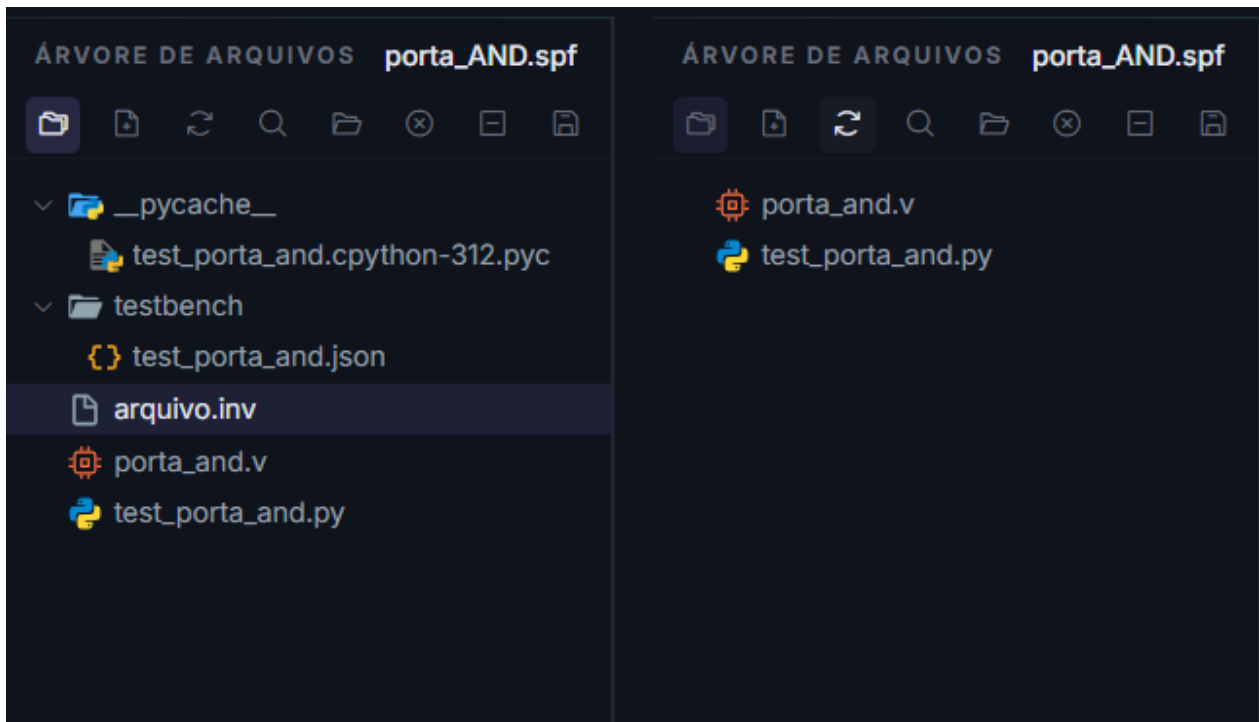


Figura8.2: À esquerda, pastas auxiliares e `arquivo.inv` aparecem na árvore. À direita, depois de salvar o `.inv`, permanecem somente os arquivos não filtrados.

Use `.inv` para reduzir o ruído visual na AURORA e `.gitignore` para impedir que o Git rastreie arquivos gerados, temporários ou locais. Um item escondido por `.inv` ainda pode aparecer normalmente no painel de **Controle de Versão**.

8.6 Cuidados

- Revise o diff antes de preparar ou confirmar arquivos.
- Salve as abas abertas antes de trocar de branch.
- Faça Fetch antes de sincronizar um projeto compartilhado.
- Não publique saídas geradas ou credenciais sem necessidade.
- Preserve uma cópia do projeto antes de operações estruturais importantes.

Fluxo Verilog

Fluxo completo para projetos Verilog

Este roteiro apresenta a AURORA como ambiente de desenvolvimento Verilog independente do fluxo C±. Ao final, você terá um projeto com fontes RTL, Top Level, testbench, validação, simulação, formas de onda e visualização no PRISM.

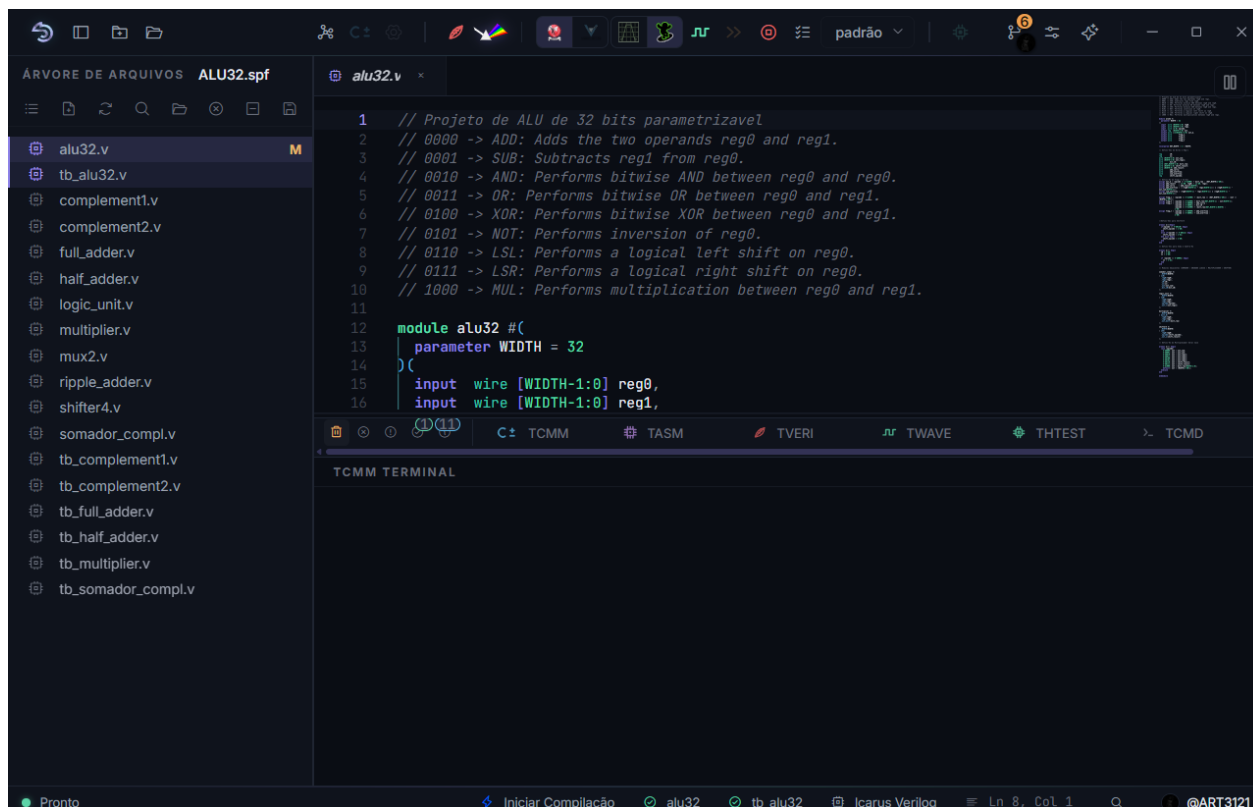


Figura9.1: O projeto ALU32 aberto em alu32.v, com os módulos RTL e os testbenches registrados na árvore **Arquivos**.

9.1 1. Criar ou abrir o projeto

1. Clique em **Novo Projeto**.
2. Informe um nome para o projeto.
3. Escolha uma pasta com permissão de escrita.
4. Confirme a criação.

A AURORA cria a pasta do projeto e um arquivo .spf, usado para registrar os arquivos e as seleções do fluxo. Não é necessário criar um processador no **Hub de Processadores** para trabalhar com Verilog.

9.2 2. Adicionar os módulos RTL

Crie os arquivos no editor ou importe módulos existentes. Você também pode arrastar e soltar um ou vários arquivos sobre a árvore **Arquivos**. Esse recurso aceita módulos Verilog e SystemVerilog (.v, .sv e .vh) e testbenches cocotb (.py).

A AURORA classifica o conteúdo importado como fonte sintetizável ou testbench. Confirme a classificação na árvore e mantenha como fontes sintetizáveis todos os módulos que participam do circuito, inclusive os módulos instanciados pelo módulo principal.

Para um primeiro teste, você pode usar o circuito `porta_and.v` disponível na [Galeria de projetos e testbenches](#).

9.3 3. Definir o Top Level

Na árvore **Arquivos**, localize o arquivo que contém o módulo principal. Clique nele com o botão direito do mouse e selecione **Definir como Top Level**.

O Top Level é a raiz usada para:

- Validar a elaboração do projeto.
- Gerar a visualização **Hierarquia** da árvore.
- Associar o circuito aos testbenches Verilog ou Python/cocotb.
- Gerar a visualização do PRISM.

A bandeira ao lado do arquivo e o nome exibido na barra de status confirmam a seleção. Se o módulo principal instancia outros módulos, todos eles também devem estar adicionados como fontes sintetizáveis.

9.4 4. Adicionar o testbench

Use uma das alternativas:

- Um testbench Verilog .v, que instancia o circuito, gera estímulos e pode incluir as verificações no próprio HDL.
- Um testbench Python .py com cocotb, que controla o Top Level pelo simulador.

Adicione o arquivo à árvore, clique nele com o botão direito do mouse e escolha **Marcar como Testbench**. Essa ação define o Testbench Top usado na simulação. O testbench fica separado das fontes sintetizáveis.

9.5 5. Validar o Verilog

1. Salve os arquivos.
2. Clique em **Compilar Verilog**.
3. Leia o terminal **Verilog**.
4. Corrija a primeira mensagem de erro, se houver.

A validação confirma sintaxe, módulos e conexões necessárias para elaborar o Top Level. Ela não substitui o testbench: um circuito pode ser elaborado corretamente e ainda produzir resultados funcionais incorretos.

9.6 6. Simular

1. Escolha Icarus Verilog ou Verilator. A **Execução rápida** com testbench Verilog exige Verilator; testbenches cocotb podem usar o simulador selecionado.
2. Confirme o Top Level e o Testbench Top na barra de status.
3. Use **Execução rápida** para executar o teste sem abrir formas de onda.

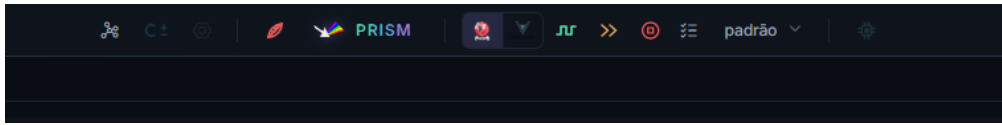


Figura9.2: Na barra superior, escolha o simulador e use **Execução rápida** ou **Analisar Verilog (forma de onda)** conforme o resultado desejado.

4. Use **Analisar Verilog (forma de onda)** para executar o Testbench Top, gerar a saída temporal e abrir o viewer selecionado.
5. Confirme no terminal se o testbench terminou e se todas as verificações passaram.

Icarus Verilog é uma boa escolha para a primeira execução e para observar sinais internos. Verilator é indicado quando a simulação exige mais desempenho e o projeto utiliza construções compatíveis.

9.7 7. Analisar formas de onda

Abra **Configuração de Ondas** para escolher os sinais e execute **Analisar Verilog (forma de onda)**. O viewer selecionado abre o VCD ou FST produzido pela simulação.

Use a forma de onda para conferir clock, reset, entradas, saídas e estados internos relevantes. A passagem do testbench continua sendo o critério funcional; a forma de onda ajuda a explicar por que um caso passou ou falhou.

9.8 8. Visualizar no PRISM

Com o Top Level definido e o Verilog válido, clique em **Abrir PRISM**. Use a visualização para conferir a hierarquia e as conexões estruturais do circuito.

O PRISM oferece o modo **Esquemático**, para inspecionar a estrutura, e o modo **Simular**, para alterar entradas lógicas e observar saídas diretamente. Essa interação não substitui um testbench nem a análise temporal no viewer de ondas.

9.9 Resultado esperado

O fluxo Verilog está concluído quando:

- Todos os módulos necessários estão registrados como fontes sintetizáveis.
- O Top Level e o Testbench Top aparecem corretamente na barra de status.
- A validação Verilog termina sem erro.
- O testbench Verilog ou cocotb conclui suas verificações.
- A análise de forma de onda abre os sinais esperados, quando solicitada.
- O PRISM apresenta a raiz correta do circuito.

Para exemplos completos em `.v` e cocotb, consulte [Galeria de projetos e testbenches](#). Para detalhes de classificação dos arquivos, consulte [Arquivos Verilog e Testbenches](#).

Arquivos Verilog e Testbenches

Os arquivos Verilog descrevem o circuito e o ambiente usado para testá-lo. Esta página mostra como classificá-los, definir os pontos de entrada e evitar seleções que levem a uma compilação incompleta.

10.1 Tipos de arquivo

A AURORA separa os arquivos em dois grupos:

Fontes sintetizáveis

Módulos RTL que formam o circuito.

Testbenches

Arquivos Verilog ou Python/cocotb usados somente para simulação.

Confirme a categoria ao importar. Um testbench não deve ser incluído como fonte sintetizável, pois contém estímulos e construções destinadas apenas à simulação. Da mesma forma, os módulos instanciados pelo circuito precisam estar entre as fontes selecionadas.

10.2 Adicionar arquivos

1. Localize o arquivo na árvore ou use a ação de importação.
2. Marque-o na categoria correta.
3. Repita para todas as dependências do projeto.

Arquivos externos funcionam, mas tornam o projeto mais difícil de mover. Sempre que possível, mantenha-os dentro da pasta do projeto. Depois da importação, abra o arquivo pela árvore para confirmar que o caminho registrado está acessível.

10.3 Definir o Top Level

Escolha o arquivo que contém o módulo principal do circuito, clique nele com o botão direito do mouse e selecione **Definir como Top Level**.

O Top Level deve declarar ou conter o módulo que representa o circuito completo. Ele é necessário para:

- Validar o Verilog.
- Abrir o PRISM.
- Montar a hierarquia do projeto.

Definir um módulo intermediário como Top Level pode produzir uma validação parcial e uma árvore incompleta. Confirme o nome do módulo principal antes de executar o fluxo.

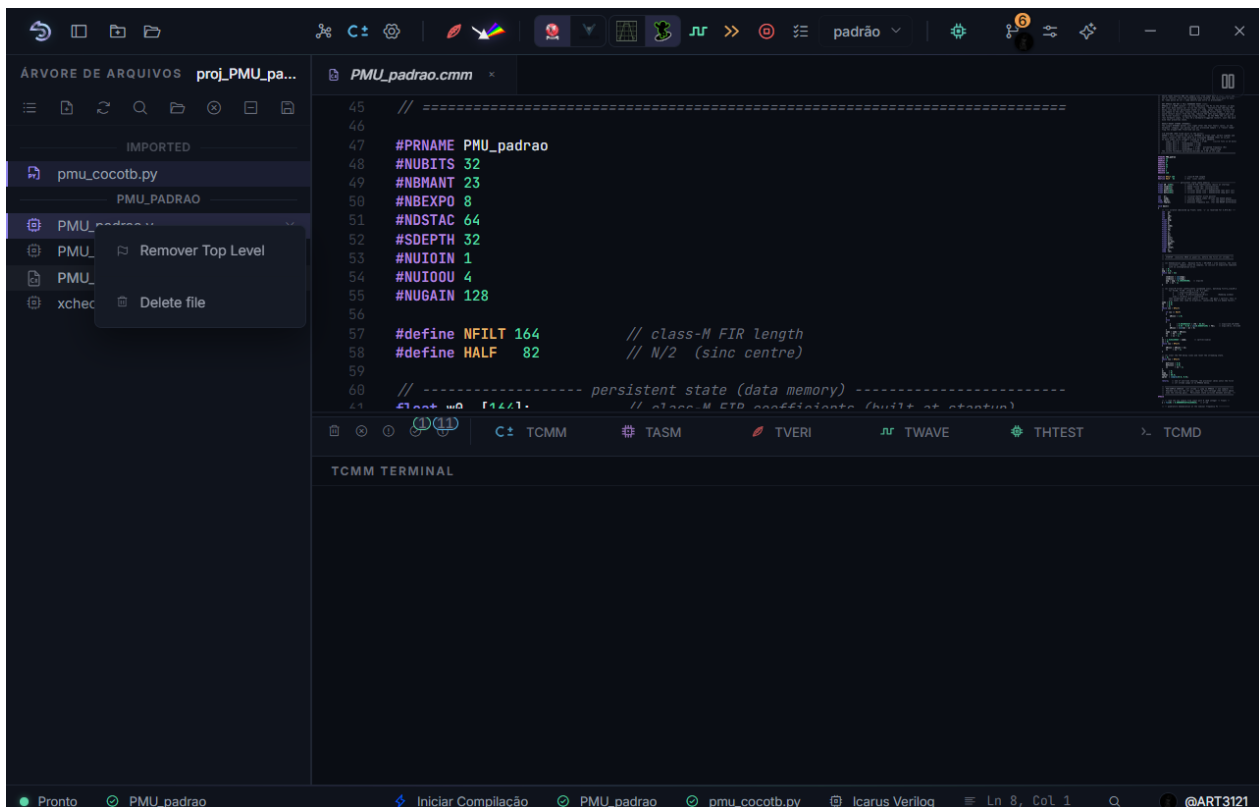


Figura 10.1: No projeto `proj_PMU_padrao`, clique com o botão direito em `PMU_padrao.v` e use a ação de Top Level. A bandeira na árvore e o nome na barra de status confirmam a seleção; quando o arquivo já está selecionado, o menu oferece **Remove Top Level**.

10.4 Definir o Testbench Top

Escolha o testbench que será executado, clique nele com o botão direito do mouse e selecione **Marcar como Testbench**. Essa ação define o Testbench Top usado pela simulação.

- Arquivos `.v` usam testbench Verilog.
- Arquivos `.py` usam cocotb.

O Testbench Top é necessário para **Analisar Verilog (forma de onda)** e outras ações de simulação.

O arquivo selecionado deve conseguir instanciar o circuito e gerar os estímulos do teste. Em cocotb, o arquivo Python contém o módulo de teste, enquanto o Top Level continua apontando para o circuito Verilog.

10.5 Usar a vista de hierarquia

Depois de uma análise bem-sucedida, alterne a árvore para a vista de hierarquia. Ela mostra como os módulos se relacionam e ajuda a localizar instâncias. Use essa vista para verificar se todas as dependências esperadas foram encontradas.

A vista de hierarquia não altera os arquivos selecionados no projeto.

10.6 Corrigir arquivos ausentes

Quando uma referência não existe mais:

1. Restaure o arquivo no caminho anterior.
2. Remova a referência antiga e importe a nova localização.

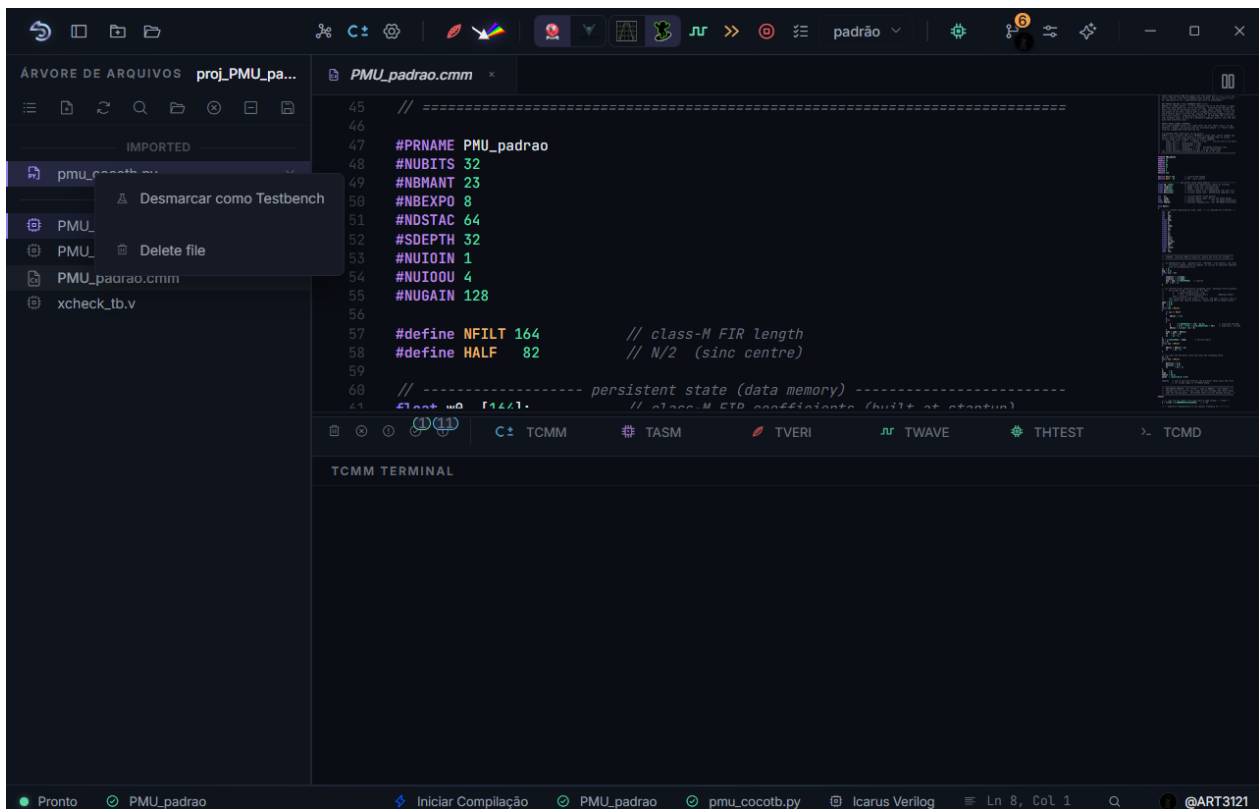


Figura10.2: No projeto proj_PMU_padrão, use **Marcar como Testbench** no arquivo .v ou .py que será executado. O ícone de frasco identifica o testbench; quando ele já está selecionado, o menu oferece **Desmarcar como Testbench**.

10.7 Evitar erros comuns

- Não inclua duas cópias que declarem o mesmo módulo.
- Mantenha todas as dependências do Top Level selecionadas.
- Defina Top Level e Testbench Top antes de simular.
- Valide com **Verilog** antes de abrir o PRISM.

Quando houver erro de módulo desconhecido, confira primeiro se o arquivo que declara esse módulo foi adicionado como fonte sintetizável. Quando houver declaração duplicada, procure cópias do mesmo módulo em arquivos diferentes.

Para praticar essas seleções com circuitos completos, consulte [Galeria de projetos e testbenches](#). A galeria contém fontes Verilog e C++ acompanhadas por testbenches Verilog e cocotb equivalentes.

Fluxo SAPHO

Fluxo completo para processadores SAPHO

Esta página é o roteiro de referência do fluxo SAPHO, para consulta quando você já conhece o caminho e precisa conferir uma etapa ou tratar um caso menos comum. Se esta é a sua primeira vez, faça antes o *Primeiro projeto: um filtro de média móvel*, que percorre o mesmo caminho ensinando cada conceito.

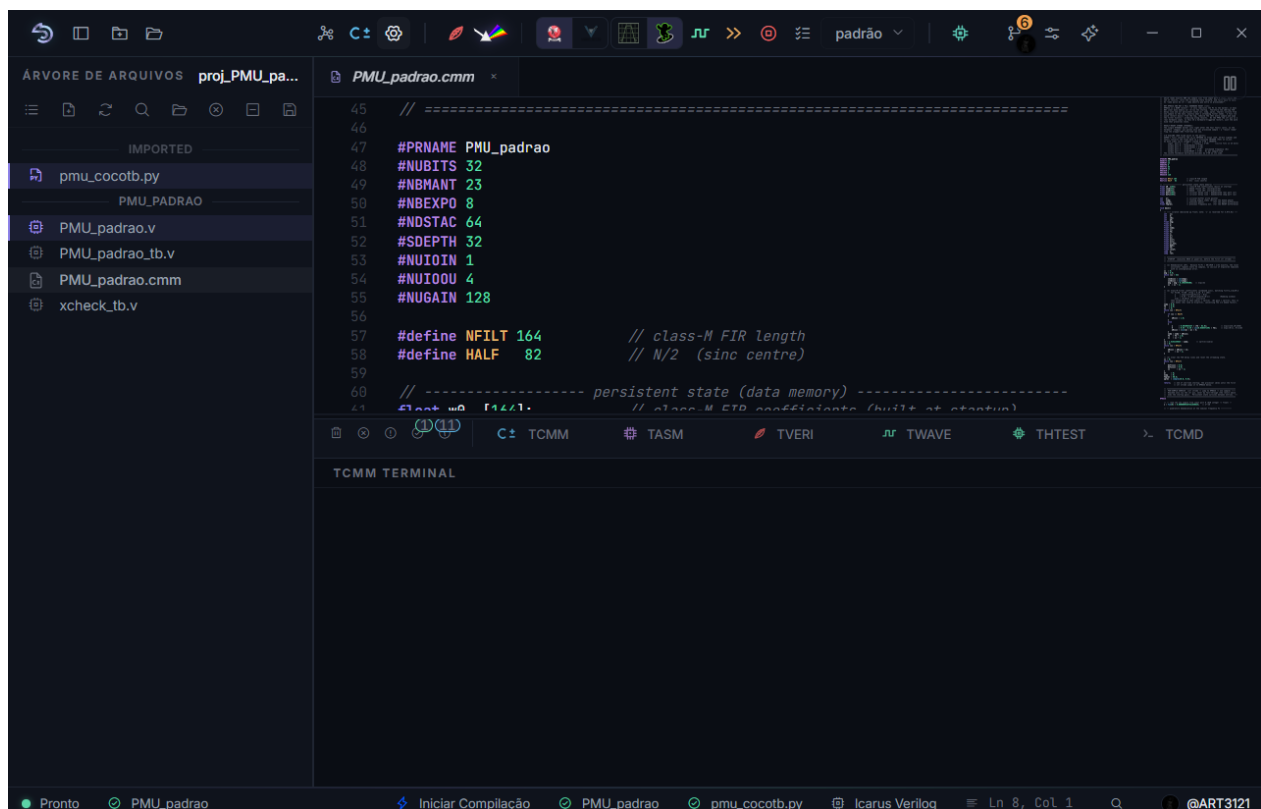
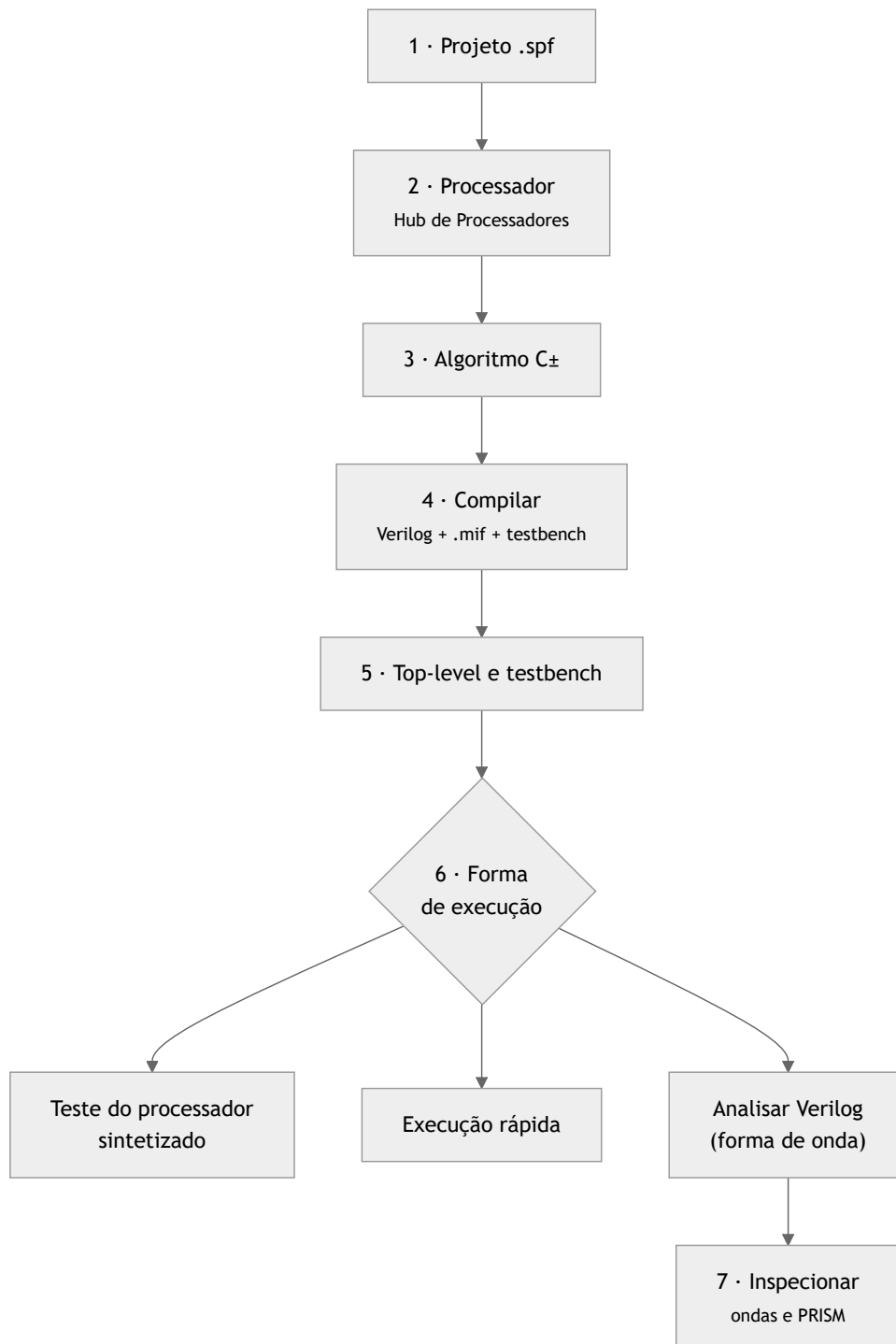


Figura 11.1: Ao abrir um algoritmo C $\pm$, o processador correspondente fica ativo e as ações relacionadas a ele são habilitadas na barra superior.

11.1 O caminho em uma olhada



11.2 1. Criar ou abrir o projeto

Clique em *Novo Projeto*, informe um nome sem espaços nem acentos e escolha uma pasta com permissão de escrita.

O arquivo .spf criado pela AURORA registra os processadores, as fontes e as seleções do projeto. Ele é JSON legível e amigável a *diffs*, mas a AURORA é a sua única escritora: não o edite à mão no fluxo normal. Detalhes em [Projetos](#).

11.3 2. Criar o processador

Abra o *Hub de Processadores*, informe o nome e os parâmetros e confirme em *Gerar Processador*. No primeiro projeto, mantenha os valores de fábrica, que já formam uma configuração válida.

A referência de cada campo, das validações e do que é gerado está em [Criar e configurar processadores](#); o significado de cada parâmetro em termos de *hardware*, em [O processador SAPHO por dentro](#).

Depois da confirmação, o processador aparece na árvore com as três subpastas:

```
<processador>/
├── Software/      o algoritmo C± editável
├── Hardware/      o Verilog e as memórias, gerados
└── Simulation/    o testbench, os estímulos e as saídas
```

11.4 3. Escrever o algoritmo C±

Abra `Software/<processador>.cmm`, preserve as diretivas geradas e escreva o algoritmo. Salve antes de compilar: os compiladores leem o conteúdo gravado no disco, não o que está no editor.

A linguagem inteira está documentada em [A linguagem C±: fundamentos](#). Para testes que precisam identificar o término do algoritmo, use `#TOAQUI` no ponto adequado.

11.5 4. Gerar o hardware

1. Mantenha o arquivo `.cmm` aberto e em foco.
2. Clique em *Compilar C±*.
3. Acompanhe os terminais **TCMM** e **TASM**.
4. Aguarde a criação ou atualização dos arquivos em *Hardware*.

O YANC transforma o algoritmo em *assembly*, depois gera o módulo Verilog, as imagens de memória e o *testbench* padrão. O detalhamento dos três estágios está em [Compilação: de C± a Verilog](#).

⚠ Aviso

Um arquivo antigo em *Hardware* não comprova que a compilação atual passou. Confirme o resultado nos terminais.

11.6 5. Definir top-level e testbench

O módulo em `Hardware/<processador>.v` pode servir como *top-level* quando o projeto testa apenas esse processador. Clique nele com o botão direito e escolha *Definir como Top Level*.

Em sistemas maiores, o *top-level* é outro módulo Verilog, escrito por você, que instancia um ou mais processadores gerados junto com a infraestrutura convencional de *hardware*. Esse é o padrão de projeto do SAPHO, discutido em [O processador SAPHO por dentro](#).

Como *testbench*, escolha uma das três opções:

O gerado

O `Simulation/<processador>_tb.v` produzido pela compilação. Gera *clock* e *reset*, lê os estímulos de `input_<i>.txt` e grava as saídas em `output_<i>.txt`. Basta para a maioria dos projetos de um único processador.

Um Verilog seu

Um `.v` escrito por você, quando o estímulo exige lógica que os arquivos de texto não expressam, ou quando o teste envolve vários blocos.

Um cocotb em Python

Um `.py` com a diretiva `# aurora-toplevel: <processador>` em comentário, ou herdada da configuração do projeto. Indicado quando a verificação se beneficia de NumPy, SciPy e asserções legíveis. Veja [Simular com Icarus, Verilator ou cocotb](#).

Clique com o botão direito no arquivo escolhido e selecione *Marcar como Testbench*. Confirme os dois papéis na barra de status antes de simular.

11.7 6. Escolher a forma de execução

Ação	Quando usar
<i>Teste do processador sintetizado</i>	Validação rápida de entradas, saídas e término, sem inspeção visual. Usa um <i>harness</i> do Verilator e reporta no terminal THTEST
<i>Execução rápida</i>	Verificações automatizadas e regressões curtas, sem abrir formas de onda
<i>Analisar Verilog (forma de onda)</i>	Quando você precisa observar sinais, portas e estados internos ao longo do tempo

11.8 7. Conferir entradas e saídas

Nos processadores gerados, `out` transporta o valor e `out_en` identifica a porta de saída escrita. O sinal `cheguei` indica que a execução alcançou o marcador `#TOAQUI`.

Um teste adequado verifica:

- se cada porta recebeu os valores esperados;
- se a quantidade e a ordem das saídas estão corretas;
- se o processador alcançou o ponto de término;
- se existe um limite de ciclos que evita espera infinita.

11.9 8. Analisar o hardware gerado

Use *Analisar Verilog (forma de onda)* para observar o comportamento temporal e *Abrir PRISM* para examinar a estrutura RTL.

🔔 Importante

O algoritmo `.cmm` continua sendo a única fonte editável. Os arquivos da pasta `Hardware` são resultado da geração e serão substituídos na próxima compilação: qualquer edição manual neles se perde.

11.10 Resultado esperado

O fluxo está concluído quando:

- o Hub de Processadores criou a estrutura do processador;
- o algoritmo `.cmm` compila sem erro;
- o Verilog e as imagens de memória estão atualizados;

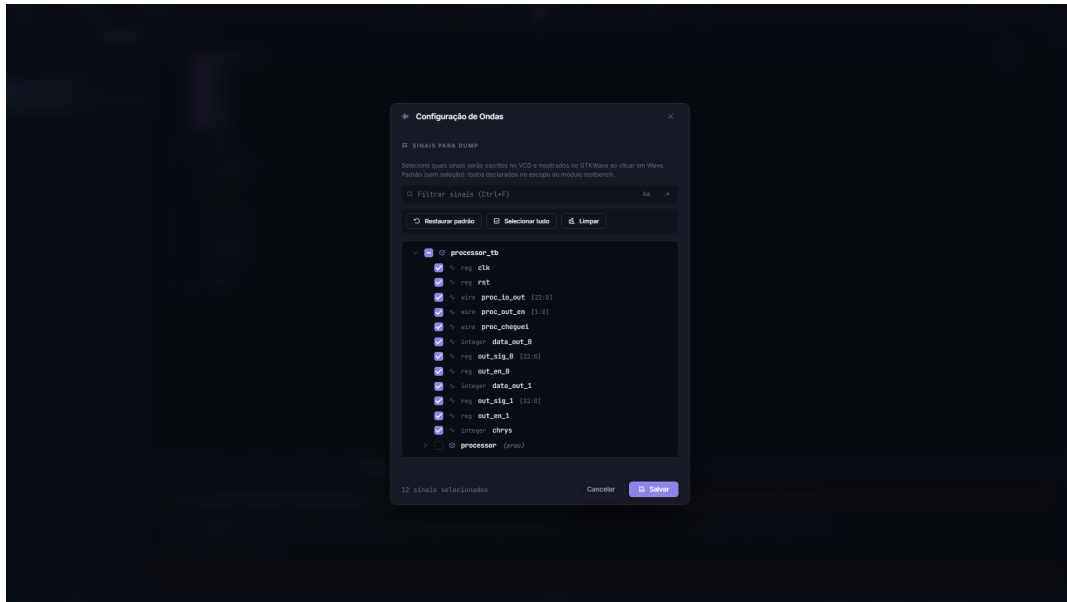


Figura11.2: No fluxo SAPHO, comece por `clk`, `rst`, as portas de saída e o sinal de término. Expanda o processador somente quando precisar investigar sinais internos.

- o *top-level* e o *testbench* aparecem corretamente na barra de status;
- a execução produz as saídas esperadas e alcança o término;
- o visualizador de ondas ou o PRISM apresenta o resultado desejado, quando utilizado.

11.11 Leitura relacionada

- *A linguagem C±: fundamentos* para escrever o algoritmo.
- *Criar e configurar processadores* para os campos do Hub e as configurações de simulação.
- *Galeria de projetos e testbenches* para exemplos C± com *testbenches* Verilog e cocotb.
- *Fluxo completo para projetos Verilog* se o seu projeto não gera processadores e trabalha só com RTL.

Primeiro projeto: um filtro de média móvel

Este é o tutorial condutor do manual. Ao final dele você terá criado um projeto, gerado um processador SAPHO sob medida, escrito um algoritmo em C $\pm$, compilado até Verilog, simulado o circuito e lido o resultado na forma de onda. Tudo dentro da AURORA, sem tocar em uma linha de comando.

Reserve cerca de trinta minutos. Se algo não sair como o descrito, cada seção termina com o que conferir.

O que vamos construir

Um filtro de média móvel de quatro amostras. Ele lê um valor por uma porta de entrada, guarda as quatro últimas leituras, soma e devolve a média por uma porta de saída. É o filtro mais simples que existe em processamento de sinais, e serve para suavizar um sinal ruidoso.

Escolhemos esse exemplo porque ele exercita, em vinte linhas, tudo o que um projeto SAPHO tem: entrada e saída, um vetor na memória de dados, aritmética e um laço infinito.

12.1 Passo 1: criar o projeto

Tudo no SAPHO acontece dentro de um projeto, que é uma pasta no disco governada por um arquivo `.spf` (*SAPHO Project File*).

1. Clique em *Novo Projeto*, na barra superior ou na tela de boas-vindas.
2. Em *Nome do Projeto*, digite `MeuFiltro`.
3. Clique em *Procurar* e escolha uma pasta na qual você tenha permissão de escrita.
4. Clique em *Gerar Projeto*.

Aviso

O nome aceita apenas letras, números, sublinhado e hífen. Sem espaços, sem acentos e sem símbolos. Se algum campo violar a regra, um diálogo explica o problema antes de qualquer coisa ser criada no disco.

Confira: a árvore lateral deve mostrar `MeuFiltro`, praticamente vazia. No disco surgiu `<local>/MeuFiltro/MeuFiltro.spf`.

12.2 Passo 2: criar o processador

Um projeto recém-criado ainda não tem processadores. Vamos criar o nosso.

1. Clique em *Hub de Processadores*, na barra superior.
2. Em *Nome do Processador*, digite `media_movel`.
3. Preencha os parâmetros conforme a tabela abaixo.
4. Clique em *Gerar Processador*.

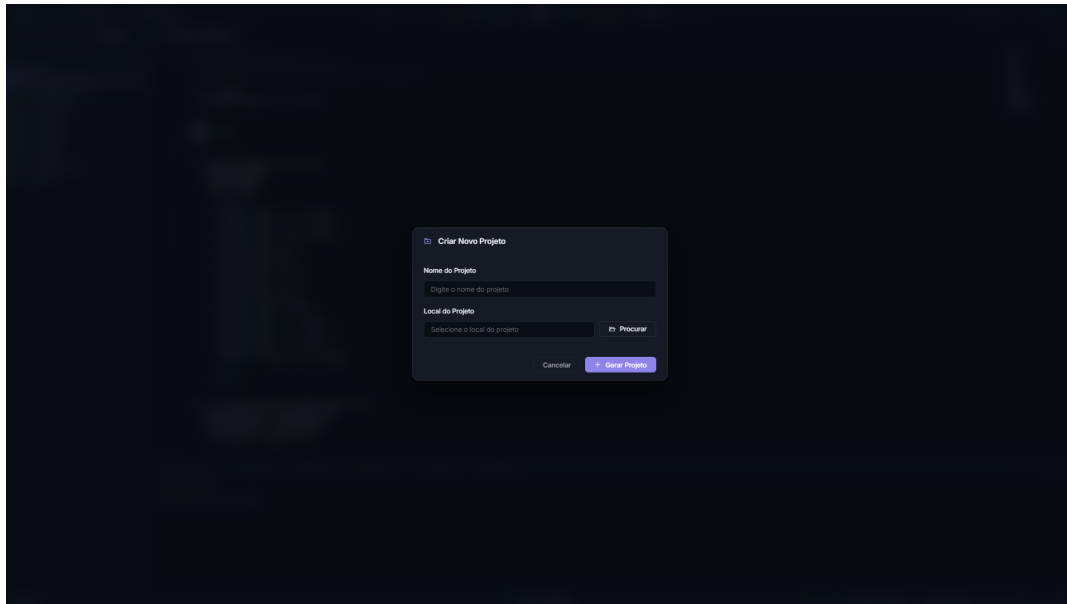


Figura12.1: O formulário de criação pede apenas nome e local. O campo de local é somente leitura e é preenchido pelo botão *Procurar*.

Tabela12.1: Valores para este tutorial

Campo	Valor	Por que este valor
Nome do Processador	<code>media_movel</code>	Vira o nome da pasta, do arquivo e da diretiva <code>#PRNAME</code>
Total de Bits	16	Largura da palavra; 16 bits bastam para sinais de instrumentação
Bits da Mantissa	10	Precisão do ponto flutuante, cerca de três dígitos decimais
Bits do Expoente	5	Faixa do ponto flutuante
Ganho	128	Só importa se o programa usar <code>norm()</code> ; deve ser potência de dois
Pilha de Instruções	2	Profundidade de chamadas de função aninhadas
Pilha de Dados	4	Complexidade das expressões
Portas de Entrada	1	Uma porta para receber as amostras
Portas de Saída	1	Uma porta para publicar a média

⏸ Importante

A regra que mais reprova o formulário O total de bits deve ser exatamente a soma da mantissa, do expoente e do bit de sinal. Aqui, $16 = 10 + 5 + 1$. Se o botão não habilitar, confira essa igualdade primeiro; ela é uma exigência estrutural do *hardware* de ponto flutuante, explicada em [O ponto flutuante do SAPHO](#).

Confira: a árvore deve mostrar o processador `media_movel` com três subpastas.

```

MeuFiltro/
├── MeuFiltro.spf
├── media_movel/
│   ├── Software/media_movel.cmm    o seu código
│   ├── Hardware/                  vazia por enquanto
│   └── Simulation/                 vazia por enquanto

```

O arquivo `media_movel.cmm` já nasce com o cabeçalho preenchido pelo formulário e um `main()` vazio à sua espera.

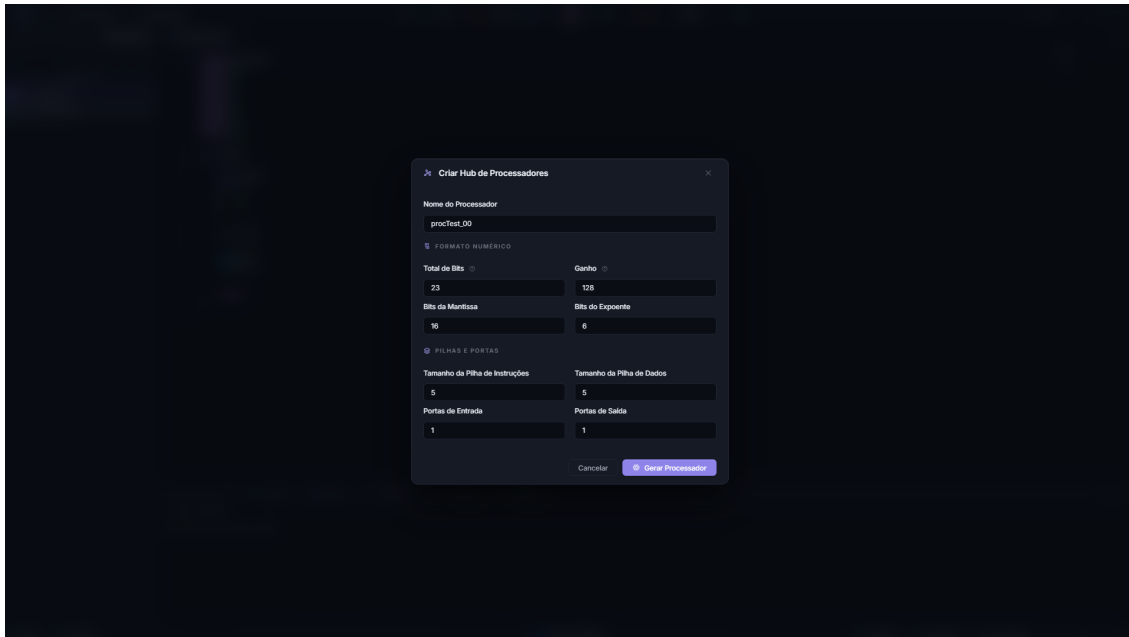


Figura12.2: O Hub concentra os parâmetros de arquitetura. A validação acontece enquanto você digita, e *Gerar Processador* só habilita quando tudo está consistente.

12.3 Passo 3: escrever o algoritmo

Abra `Software/media_movel.cmm` com um clique duplo na árvore. Substitua o corpo do arquivo por este programa:

Listing 12.1: `media_movel.cmm`, o filtro completo

```

1  #PRNAME media_movel
2  #NUBITS 16
3  #NDSTAC 4
4  #SDEPTH 2
5  #NUIOIN 1
6  #NUIOOU 1
7  #NBMANT 10
8  #NBEXPO 5
9  #NUGAIN 128
10
11 void main()
12 {
13     int x[4];    // historico das ultimas 4 amostras
14     int soma;
15
16     while (1)
17     {
18         x[3] = x[2];        // desloca o historico
19         x[2] = x[1];
20         x[1] = x[0];
21         x[0] = in(0);        // le nova amostra da porta 0
22
23         soma = x[0] + x[1] + x[2] + x[3];
24         out(0, soma >> 2);    // media = soma/4, sem usar divisor
25     }
26 }
```

Salve com `Ctrl+S`. O ponto na aba deve desaparecer.

12.3.1 Lendo o programa linha a linha

As nove primeiras linhas são **diretivas**, e não código executável: elas configuram o *hardware* que será gerado. Foi o formulário do passo anterior que as escreveu, e a partir de agora o arquivo é a fonte da verdade. Editar uma diretiva muda o processador na próxima compilação. A lista completa está em [Referência de diretivas](#).

O `while` (1) não é um descuido. O programa modela um circuito, e circuitos não terminam: eles processam amostras para sempre. Essa é a forma canônica de um programa SAPHO.

Duas escolhas do corpo do laço merecem atenção, porque são idiomáticas e explicam muito sobre a plataforma:

O histórico é deslocado à mão

Não há alocação dinâmica nem ponteiros. O vetor `x` ocupa quatro endereços fixos da memória de dados, e deslocar o histórico é copiar valor a valor. Em troca, cada uma dessas quatro posições aparece pelo nome na forma de onda, o que você verá no passo 6.

A divisão por quatro é um deslocamento de bits

`soma >> 2` produz o mesmo resultado de `soma / 4`, mas com uma diferença decisiva: usar `/` instanciaría um divisor inteiro no circuito, um dos blocos mais caros da unidade lógica e aritmética. O deslocamento é quase de graça. Esse é o princípio de pagar apenas pelo que se usa, detalhado em [Recursos avançados](#).

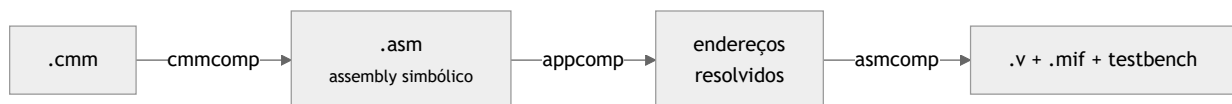
➔ Ver também

A linguagem inteira está documentada em [A linguagem C±: fundamentos](#). Se você já conhece C, a leitura mais útil é a das ausências: não existem `--`, `+=`, operador ternário, ponteiros nem `struct`.

12.4 Passo 4: compilar

Com o `.cmm` salvo e em foco no editor, clique em *Compilar C±*.

Três compiladores rodam em sequência, e você acompanha os dois primeiros terminais:



O terminal **TCMM** mostra a tradução para *assembly*. O terminal **TASM** mostra a montagem, a geração do Verilog e, o mais interessante, os avisos de recurso instanciado: cada bloco de *hardware* que o seu programa acabou de ligar no circuito.

Confira: a pasta `Hardware` deve ter ganhado três arquivos.

```

media_movel/
├── Software/
│   ├── media_movel.cmm      o seu fonte
│   ├── media_movel.asm      assembly gerado
│   └── pc_media_movel_mem.txt  tabela PC -> linha, usada nas ondas
├── Hardware/
│   ├── media_movel.v        o processador em Verilog
│   ├── media_movel_inst.mif  imagem da memória de programa
│   └── media_movel_data.mif  imagem da memória de dados
└── Simulation/
    └── media_movel_tb.v      testbench gerado
  
```

Esse `.v` e esses `.mif` são exatamente o que se leva ao Quartus ou ao Vivado na hora de gravar o FPGA de verdade.

Aviso

A existência de um arquivo em `Hardware` não prova que a compilação atual passou: ele pode ser de uma tentativa anterior. Confirme sempre nos terminais que as duas etapas terminaram sem erro.

Se a compilação falhou, vá à primeira mensagem de erro, não à última. O terminal transforma a referência de linha em um *link*: o clique leva o editor ao ponto exato. As mensagens do compilador são explicadas em [Recursos avançados](#).

12.5 Passo 5: preparar o estímulo e simular

O *testbench* gerado alimenta cada porta de entrada com o conteúdo de um arquivo de texto, um valor por linha, e grava cada porta de saída em outro. Vamos criar o estímulo.

1. Na visão *Pastas* da árvore, clique com o botão direito em `media_move1/Simulation` e escolha *Novo Arquivo...*
2. Nomeie o arquivo `input_0.txt`, correspondente à porta de entrada 0.
3. Escreva um degrau: alguns zeros seguidos de um valor alto, para ver o filtro suavizar a transição.

Listing 12.2: `Simulation/input_0.txt`, um degrau de 0 para 100

```
0
0
0
0
0
100
100
100
100
100
100
100
100
100
100
100
```

4. Confirme, na barra superior, que o simulador selecionado é o **Icarus Verilog** e o visualizador é o **GTKWave**.
5. Clique em *Analisar Verilog (forma de onda)*.

A AURORA compila o que for preciso, executa a simulação e abre o visualizador. Acompanhe o progresso no terminal **TWAVE**.

i Se a simulação terminar antes de o resultado aparecer

A causa quase sempre é o número de ciclos. Clique na engrenagem ao lado do botão de compilar e aumente o valor de ciclos, que por padrão é 2000. O mesmo painel ajusta a frequência de *clock* e mostra o tempo simulado estimado.

12.6 Passo 6: ler a forma de onda

O visualizador abre com um *layout* já curado pela AURORA, e essa curadoria é uma das coisas mais úteis da plataforma. Abrir um despejo bruto em um visualizador qualquer significa começar do zero, sem nenhum sinal selecionado e tudo em binário. Aqui, os sinais já vêm agrupados, nomeados e coloridos.

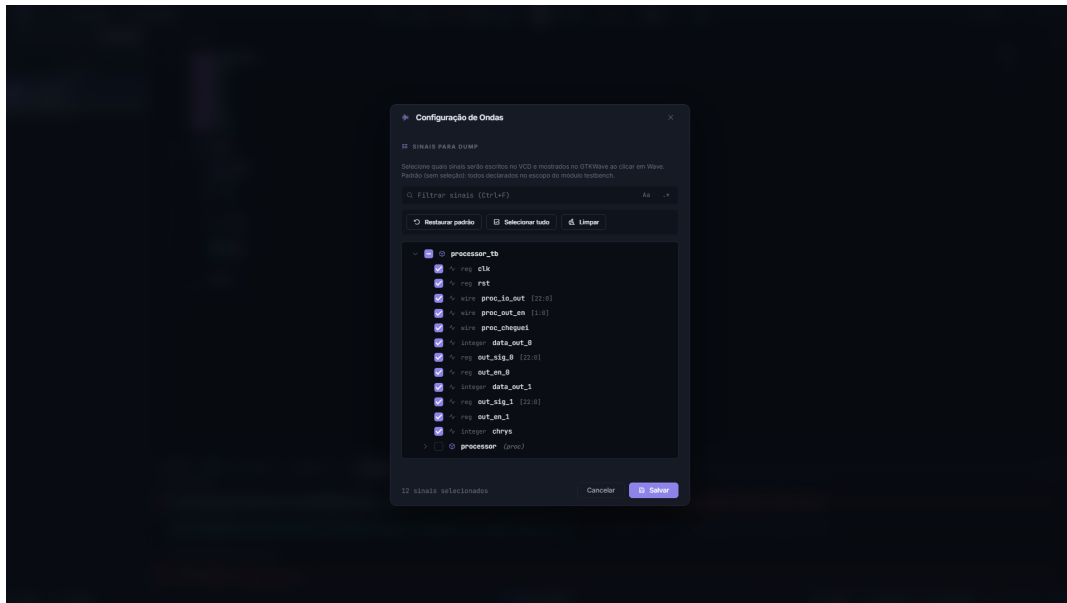


Figura 12.3: O modal *Configuração de ondas* escolhe quais sinais a simulação grava. Comece por *clk*, *rst*, as portas e o sinal de término; expanda o processador só quando precisar investigar por dentro.

Procure, na janela do visualizador:

- as variáveis `x[0]` a `x[3]`, que mostram o histórico deslizando a cada amostra;
- a variável `soma`, que sobe em degraus de 100 conforme o histórico se preenche;
- a porta de saída, na qual o degrau aparece suavizado em quatro passos, exatamente o comportamento esperado de uma média móvel de quatro amostras;
- as trilhas de texto que exibem, a cada ciclo de *clock*, a instrução *assembly* executada e a linha do seu C± correspondente.

Essa última trilha merece um instante de atenção: é o seu programa rodando dentro do *hardware*, linha a linha, em sincronia com o *clock*. É o vínculo mais direto entre o código que você escreveu e o circuito que ele virou.

Os valores de saída também ficam gravados em `Simulation/output_0.txt`, prontos para conferência em uma planilha ou em um *script*.

12.7 Passo 7: ver o circuito por dentro

Falta olhar o que o seu programa virou em termos de estrutura.

1. Na árvore, clique com o botão direito em `Hardware/media_model.v` e escolha *Definir como Top Level*.
2. Clique em *Abrir PRISM*.

O PRISM sintetiza o projeto com o Yosys e desenha o circuito como um diagrama navegável. Um clique em um módulo entra nele; um duplo clique em uma célula abre o código-fonte correspondente no editor.

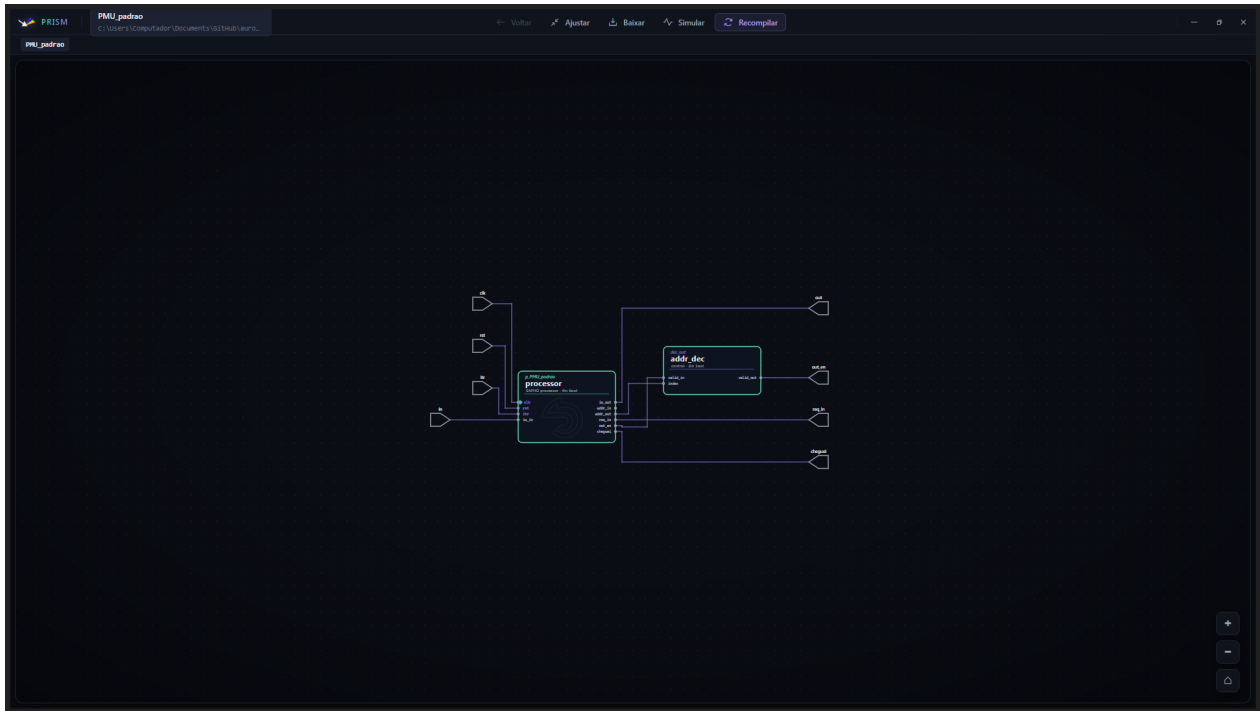


Figura 12.4: No PRISM, o processador SAPHO aparece com símbolo próprio ao lado dos demais blocos. Os blocos internos, como a unidade lógica e aritmética e as memórias, também têm desenho dedicado.

Dica

O experimento que ensina mais rápido Volte ao `.cmm`, troque `out(0, soma >> 2);` por `out(0, soma / 4);`, recompile e clique em *Recompile* no PRISM. Um divisor inteiro aparece no diagrama, e o terminal TASM anuncia o novo bloco instanciado. Desfaça a mudança e ele some.

Esse ida e volta torna palpável a relação entre uma construção da linguagem e o custo em *hardware*, que é o assunto central de [Recursos avançados](#).

12.8 O que você aprendeu

- Um projeto é uma pasta com um `.spf`, e um processador é uma pasta com *Software*, *Hardware* e *Simulation*.
- As diretivas no topo do `.cmm` definem o *hardware*; o corpo define o comportamento.
- Compilar produz Verilog, imagens de memória e um *testbench*, nessa ordem, por três compiladores encadeados.
- A simulação lê estímulos de `input_<i>.txt` e grava resultados em `output_<i>.txt`.
- A forma de onda mostra as suas variáveis pelo nome e a linha do seu código a cada ciclo.
- Escolhas de linguagem têm custo em *hardware*, e o PRISM mostra esse custo.

12.9 Para onde ir agora

Aprofundar a linguagem Tipos, complexos, notação de Dirac e a biblioteca completa.

A linguagem C $\pm$: fundamentos
executam o seu programa.

Entender a máquina Como o acumulador, o *pipeline* e as memórias

O processador SAPHO por dentro
com NumPy na verificação.

Testar em Python Trocar o *testbench* Verilog por um em cocotb,

Simular com Icarus, Verilator ou cocotb

Se, em vez de gerar um processador, o seu objetivo é escrever Verilog diretamente, o caminho é outro e está em *Fluxo completo para projetos Verilog*.

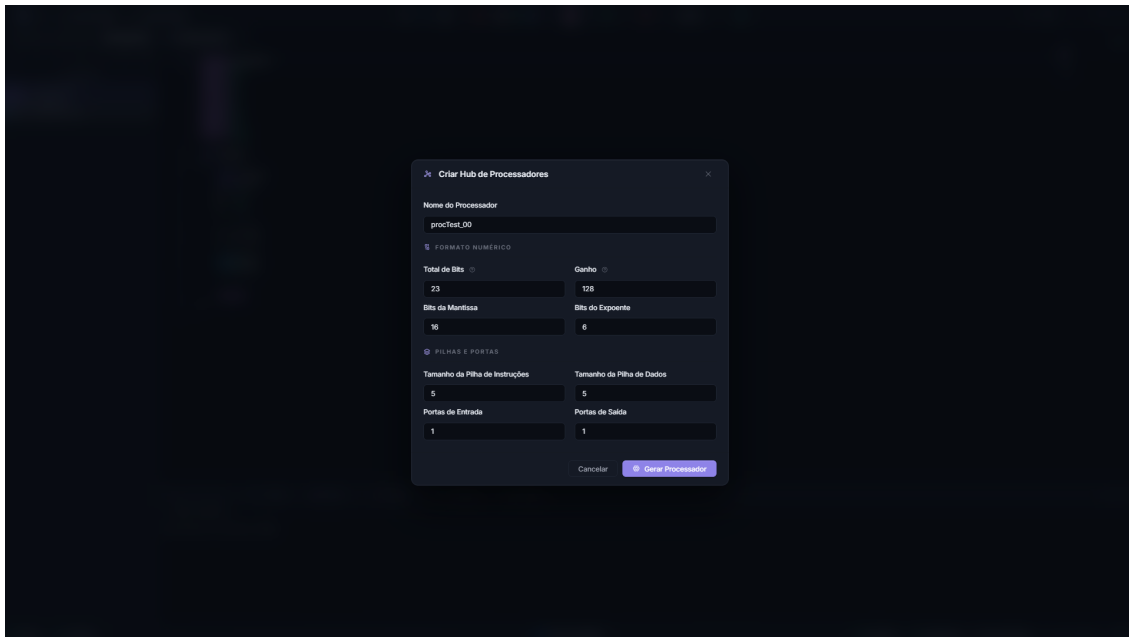
Criar e configurar processadores

Um processador reúne o algoritmo $C\pm$, a configuração de arquitetura e as pastas nas quais o *hardware* gerado será gravado. Esta página cobre as duas formas de criar um, o significado de cada campo do formulário, os ajustes de simulação e as operações de renomear e excluir.

Se você ainda não criou nenhum, faça primeiro o *Primeiro projeto: um filtro de média móvel*, que percorre o caminho completo com um exemplo.

13.1 O Hub de Processadores

Com o projeto aberto, o botão *Hub de Processadores* da barra superior abre o formulário que dá vida a um novo processador.



A imagem mostra uma janela de diálogo intitulada "Criar Hub de Processadores" com um fundo escuro. O formulário contém os seguintes campos:

- Nome do Processador:** Um campo de texto com o valor "procTest_00".
- FORMATO NUMÉRICO:** Uma seção com dois campos: "Total de Bits" (valor 23) e "Ganho" (valor 128).
- FORMATO NUMÉRICO:** Uma seção com dois campos: "Bits da Mantissa" (valor 16) e "Bits do Exponente" (valor 6).
- FILHAS E PORTAS:** Uma seção com quatro campos: "Tamanho da Pilha de Instruções" (valor 5), "Tamanho da Pilha de Dados" (valor 5), "Portas de Entrada" (valor 1) e "Portas de Saída" (valor 1).

Na base do formulário, há dois botões: "Cancelar" e "Gerar Processador".

Figura13.1: O formulário pede o nome e, em duas seções, os parâmetros numéricos. A validação acontece enquanto você digita.

Tabela13.1: Campos do formulário, valores de fábrica e validações

Campo	Diretiva	Padrão	Validação
Nome do Processador	#PRNAME	proc-Test_00	Letras, números, sublinhado e hífen
Total de Bits	#NUBITS	23	Inteiro positivo, igual a mantissa mais expoente mais um
Ganho	#NUGAIN	128	Inteiro positivo, potência de dois
Bits da Mantissa	#NBMAINT	16	Inteiro positivo
Bits do Expoente	#NBEXPO	6	Inteiro positivo
Pilha de Instruções	#SDEPTH	5	Inteiro positivo
Pilha de Dados	#NDSTAC	5	Inteiro positivo
Portas de Entrada	#NUIOIN	1	Inteiro positivo
Portas de Saída	#NUIOOU	1	Inteiro positivo

Um campo inválido ganha borda vermelha e uma dica explicando a regra, e o botão *Gerar Processador* só habilita quando tudo está consistente. Note que o padrão de fábrica já respeita a igualdade estrutural, pois $23 = 16 + 6 + 1$.

13.1.1 Como escolher os valores

O total de bits dimensiona os inteiros e o consumo de *hardware*. Dezesesseis bits bastam para muitos sinais de instrumentação, enquanto o padrão de 23 dá folga com um *float* de boa precisão.

A mantissa e o expoente definem a precisão e a faixa do ponto flutuante. Se o programa só usa inteiros, esses valores têm pouco efeito prático, mas a igualdade estrutural continua obrigatória. Veja [O ponto flutuante do SAPHO](#).

A pilha de instruções limita a profundidade de chamadas de função aninhadas, e a de dados, a complexidade das expressões. Os padrões atendem programas típicos.

Reserve uma porta para cada fluxo de dados que o processador troca com o mundo, e lembre que o ganho só importa se o programa usa a função `norm()`.

Perigo

A validação que mais reprova O total de bits precisa ser exatamente a soma da mantissa, do expoente e do bit de sinal. Se o botão não habilitar, confira essa conta antes de mexer em qualquer outro campo.

13.2 O atalho \$cmm

Existe um caminho mais rápido quando você já sabe o que quer:

1. Use `Ctrl+N` para criar um arquivo `Untitled-N`.
2. Digite somente `$cmm`.
3. Aguarde a AURORA expandir o modelo inicial do algoritmo `C±`.
4. Use `Ctrl+S` e informe o nome do processador.

Ao salvar dentro de um projeto aberto, a AURORA atualiza a diretiva `#PRNAME`, registra o processador no `.spf` e cria a estrutura de pastas. O arquivo é gravado em `Software` e passa a aparecer como processador do projeto.

O atalho usa os valores de fábrica das diretivas; ajuste-as no próprio arquivo se precisar de outra configuração.

13.3 O que é gerado

Ao confirmar, a AURORA cria dentro do projeto a pasta do processador com as três subpastas de trabalho e registra o novo processador no `.spf`, bloqueando nomes duplicados.

```
<processador>/
├── Software/      o algoritmo C± editável, e depois o assembly gerado
├── Hardware/      o Verilog e as imagens de memória, gerados
└── Simulation/    o testbench gerado, os estímulos e as saídas
```

O arquivo `Software/<nome>.cmm` nasce com o cabeçalho de diretivas preenchido com os valores do formulário e um `main()` vazio à sua espera:

Listing 13.1: O arquivo recém-criado

```
#PRNAME media_move1
#NUBITS 16
#NDSTAC 5
#SDEPTH 5
#NUIOIN 1
#NUIIOU 1
#NBMAINT 10
#NBEXPO 5
#NUGAIN 128

void main()
{
    // Ok. Voce criou um processador em C+-, mas e agora?
}
```

O “e agora” é *A linguagem C±: fundamentos*.

13.4 O processador ativo

A AURORA deduz o processador ativo do arquivo `.cmm` em foco no editor, e o nome aparece na barra de status. É esse processador que os botões de compilação e de teste vão usar.

Se o botão *Compilar C±* estiver desabilitado, confirme nesta ordem:

1. o projeto está aberto;
2. o arquivo `.cmm` do processador está aberto e em foco;
3. nenhuma compilação está em andamento.

13.5 Configurações de simulação

Ao lado do botão de compilar há uma engrenagem que abre um painel com três ajustes por processador.

Frequência de *clock*

Em MHz, padrão 100. Define a temporização do *testbench* gerado.

Número de ciclos

Padrão 2000. É quantos ciclos a simulação executa antes de encerrar.

Exibir cada elemento de vetor

Emite cada posição de vetor como um sinal visível nas formas de onda, equivalente à opção `--array` do compilador. Útil para depurar filtros; caro em vetores grandes.

O painel também exibe o tempo estimado de simulação, os ciclos divididos pelo *clock*, em microssegundos.

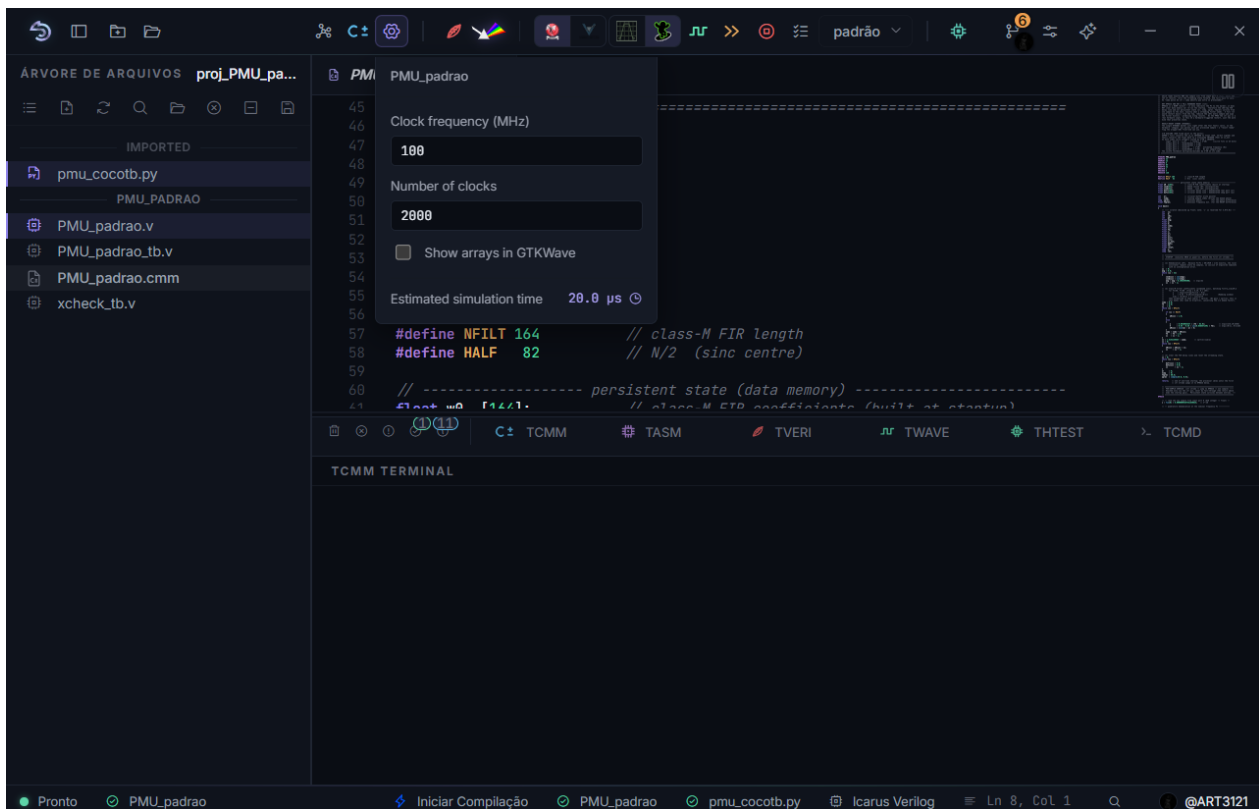


Figura13.2: O painel ajusta a frequência, o limite de ciclos e a inclusão de vetores nas formas de onda. O tempo estimado é recalculado a cada mudança.

Dica

O sintoma mais comum de todos Se a simulação termina antes de o programa produzir os resultados, aumente aqui o número de ciclos. É a primeira coisa a verificar quando a onda abre vazia ou incompleta.

13.6 Gerar o hardware

1. Abra o arquivo .cmm e salve as alterações.
2. Clique em *Compilar C±*.
3. Acompanhe os terminais **TCMM** e **TASM**.
4. Confirme que os arquivos apareceram ou foram atualizados em Hardware.

Leia primeiro o TCMM e depois o TASM. O processo deve terminar sem erro antes que os arquivos em Hardware sejam considerados atualizados.

⚠ Aviso

Um arquivo antigo em `Hardware` não comprova que a compilação atual passou. Se a geração falhar, preserve a primeira mensagem de erro para o diagnóstico; as seguintes costumam ser consequências dela.

O detalhamento de cada etapa da cadeia está em [Compilação: de C± a Verilog](#).

13.7 Testar o processador isoladamente

O botão *Teste do processador sintetizado* valida o processador como caixa-preta: um *harness* em C++ construído com o Verilator alimenta as portas de entrada e confere as saídas, sem despejo de ondas, com barra de progresso no terminal **THTEST**.

Use quando quiser confirmar rapidamente que as portas respondem e que o programa termina, sem montar uma inspeção completa no visualizador. Para detectar o fim do programa, a AURORA usa o mecanismo do `#TOAQUI` e do pino `cheguei`, inserindo o marcador ao final do `main()` quando necessário.

13.8 Renomear e excluir

Ambas as operações ficam no menu de contexto do processador na árvore.

Renomear é uma operação completa: a pasta é movida, os artefatos gerados são renomeados e a diretiva `#PRNAME` dentro do fonte é corrigida, tudo em uma só ação, sob as mesmas regras de nome do formulário.

Excluir remove a pasta do processador e o registro no `.spf`. Faça *backup* antes e leia a confirmação.

ⓘ Importante

O nome do processador e a diretiva `#PRNAME` precisam concordar: é assim que a cadeia de ferramentas conecta o fonte aos artefatos. Renomeie sempre pela AURORA, que mantém os dois lados em sincronia. Renomear pastas pelo Explorador de Arquivos deixa referências inválidas no `.spf`.

13.9 Vários processadores no mesmo projeto

Um projeto pode conter quantos processadores forem necessários, cada um com a sua pasta, o seu fonte e os seus parâmetros. O padrão de projeto do SAPHO é justamente esse: em vez de uma máquina grande que faz tudo, várias máquinas pequenas, uma por tarefa, integradas por um *top-level* em Verilog.

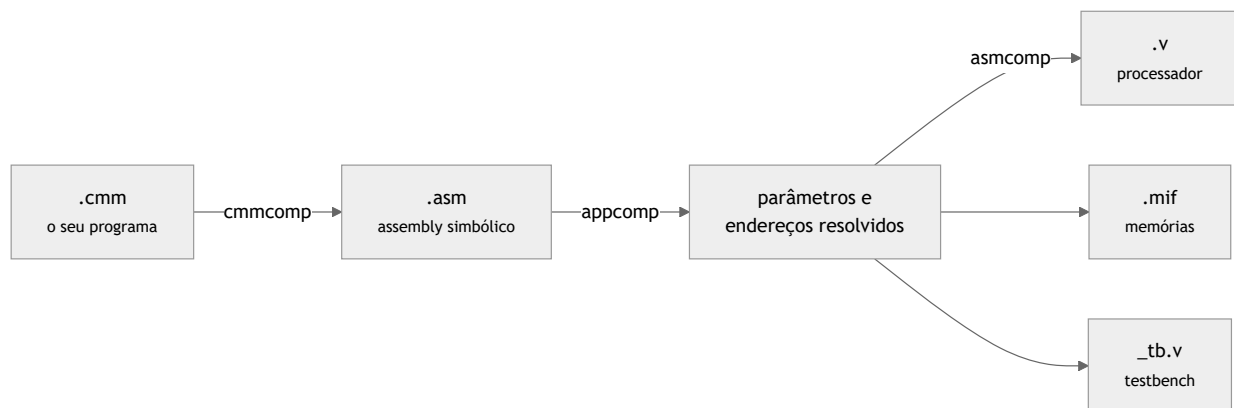
O raciocínio de arquitetura por trás dessa escolha está em *O processador SAPHO por dentro*, e a integração dos módulos, em *Arquivos Verilog e Testbenches*.

Compilação: de C± a Verilog

Compilar, no SAPHO, não é gerar um executável: é gerar um circuito. Esta página explica o que acontece quando cada botão de compilação é acionado, quais arquivos aparecem no projeto e como ler o que os terminais informam.

14.1 O pipeline do YANC em três estágios

Todo clique em *Compilar C±* executa, por processador, uma cadeia de três compiladores.



cmmcomp

Traduz o <proc>.cmm em *assembly* simbólico, o <proc>.asm, junto com os registros e as tabelas que ligam cada instrução à linha do fonte. É dele que vêm as trilhas sincronizadas das formas de onda.

appcomp

Faz a primeira passada sobre o *assembly*, coletando os parâmetros do processador e resolvendo os endereços de variáveis e rótulos.

asmcomp

Na segunda passada, gera o processador em Verilog, as duas imagens de memória e o *testbench*.

O terminal **TCMM** mostra a saída do primeiro estágio. O **TASM** mostra a dos outros dois, incluindo os avisos de recurso instanciado que revelam o custo em *hardware* do seu código.

i Nota

O processador gerado instancia os moldes de HDL do SAPHO que acompanham a instalação: o núcleo, a unidade lógica e aritmética, as memórias.

O programa não é um arquivo carregado depois. As imagens `.mif` nascem embutidas no projeto de *hardware*, e reprogramar significa recompilar e sintetizar de novo. Essa é uma diferença central em relação a um microcontrolador comum.

14.2 Os botões de compilação

Botão	O que faz
<i>Compilar C±</i>	Roda o <i>pipeline</i> completo, do fonte ao Verilog, memórias e <i>testbench</i>
<i>Sintetizar Verilog</i>	Verifica os arquivos Verilog com o Icarus, apontando erros de sintaxe e elaboração no TVERI, com linhas clicáveis
<i>Analisar Verilog (forma de onda)</i>	Percorre o fluxo completo até as ondas: compila o que for preciso, simula e abre o visualizador
<i>Execução rápida</i>	Simula sem gravar ondas, quando só interessam as saídas em arquivo
<i>Abrir PRISM</i>	Sintetiza com o Yosys e abre o diagrama de RTL. Exige <i>top-level</i> definido
<i>Teste do processador sintetizado</i>	Constrói um <i>harness</i> em C++ com o Verilator e exercita apenas as portas, como caixa-preta
<i>Cancelar</i>	Interrompe todos os processos em andamento, sem fechar a IDE

O botão *Iniciar Compilação*, o raio da barra de status, é um atalho para o fluxo principal. Todos esses comandos também existem na paleta (Ctrl+Shift+P), no grupo de compilação.

14.3 Antes de clicar

A AURORA salva os arquivos automaticamente antes de compilar, mas vale conferir três coisas na barra de status:

1. o processador ativo é o que você quer compilar, o que depende do arquivo em foco no editor;
2. o *top-level* está definido, se a ação exige síntese;
3. o *testbench* está definido, se a ação exige simulação.

A barra de status avisa o que falta com os rótulos “Sem top-level” e “Sem testbench”.

14.4 Onde cada artefato aparece

Depois de compilar o `media_movel` do exemplo condutor, o projeto ganha esta forma:

```
media_movel/
├── Software/
│   ├── media_movel.cmm      o seu fonte
│   ├── media_movel.asm      assembly gerado
│   └── pc_media_movel_mem.txt tabela PC -> linha, usada nas ondas
├── Hardware/
│   ├── media_movel.v        o processador em Verilog
│   ├── media_movel_inst.mif imagem da memória de programa
│   └── media_movel_data.mif  imagem da memória de dados
└── Simulation/
    └── media_movel_tb.v      testbench gerado
```

O `.v` e os `.mif` da pasta `Hardware` são exatamente o que se leva ao Quartus ou ao Vivado na hora de sintetizar para o FPGA real.

O `testbench` gerado produz `clock` e `reset`, alimenta o processador com os estímulos de `input_<i>.txt`, um valor por linha e um arquivo por porta de entrada, e grava as saídas em `output_<i>.txt`, além do despejo de ondas.

Aviso

A existência de um arquivo em `Hardware` não prova que a compilação atual passou: ele pode ser de uma tentativa anterior. Confirme sempre nos terminais que as etapas terminaram sem erro.

14.5 Validar um projeto Verilog

No fluxo Verilog puro, não há C $\pm$ nem processador. A etapa equivalente é a validação:

1. adicione todas as fontes sintetizáveis necessárias ao projeto;
2. defina o *top-level* pelo menu de contexto da árvore;
3. clique em *Sintetizar Verilog*;
4. leia o terminal **TVERI**.

A validação está correta quando termina sem erros de sintaxe, módulos ausentes ou módulos duplicados. Ela não executa o comportamento do circuito: confirma que o *top-level* e suas dependências formam um conjunto coerente para as etapas posteriores.

Se falhar, corrija a primeira mensagem de erro e execute de novo. Um único módulo ausente costuma produzir várias mensagens secundárias.

14.6 Ações que preparam as suas dependências

As ações abaixo compilam automaticamente o que precisam, o que evita a sequência manual:

Ação	O que ela prepara sozinha
<i>Compilar C$\pm$</i>	Atualiza o <i>hardware</i> do processador ativo
<i>Analisar Verilog (forma de onda)</i>	Compila, simula e abre o visualizador
<i>Execução rápida</i>	Compila e simula, sem visualizador
<i>Abrir PRISM</i>	Valida o RTL e gera o diagrama

Executar uma ação mais completa não elimina a necessidade de ler os terminais. *Analisar Verilog (forma de onda)* pode recompilar dependências e ainda assim falhar antes da simulação, se o Verilog estiver inválido.

14.7 Interromper uma execução

Clique em *Cancelar*. A interrupção pode levar alguns segundos enquanto processos auxiliares são encerrados. Não feche nem exclua arquivos temporários nesse período.

Depois que o cancelamento terminar, revise a última mensagem do terminal. Se uma nova execução permanecer bloqueada, feche a ferramenta externa que ainda estiver aberta, como uma janela do GTKWave, e tente novamente.

14.8 Uma decisão de segurança

A AURORA só executa binários da própria cadeia de ferramentas, validados contra uma lista fechada, e os botões não passam por um *shell*. É por isso que a IDE não oferece a execução de comandos arbitrários fora do terminal TCMD.

14.9 Erros comuns

Sintoma	Causa e solução
<i>Compilar C±</i> desabilitado	Abra o .cmm do processador desejado e confirme o nome na barra de status
<i>Top-level</i> não encontrado	Confirme o módulo principal e inclua todas as dependências Verilog como fontes sintetizáveis
Módulo duplicado	Remova da seleção uma das cópias que declaram o mesmo módulo
O arquivo gerado não mudou	Salve o .cmm, confirme o processador ativo e compile de novo
Erro de sintaxe no TCMM	Clique na linha para ir ao ponto. Confira as ausências da linguagem em Tipos, operadores e controle
Porta de entrada ou saída inexistente	O índice em in() ou out() excede as diretivas de porta. Aumente #NUI-0IN ou #NUI00U
Binário do simulador não encontrado	A pasta components da instalação está incompleta ou foi movida. Reinstale pelo instalador oficial
A execução não termina	Use <i>Cancelar</i> e aguarde a limpeza. Se persistir, veja Solução de problemas

14.10 Leitura relacionada

- [Os terminais](#) explica quem escreve em cada aba e como ler os avisos de recurso instanciado.
- [Recursos avançados](#) mostra o custo em área de cada construção.
- [Simular com Icarus, Verilator ou cocotb](#) assume daqui, com os motores e os tipos de *testbench*.

A linguagem $C^{\pm}$

A linguagem C±: fundamentos

A C± (lê-se “C mais-menos”; nos arquivos, extensão `.cmm`) é a linguagem principal do SAPHO. É um dialeto de C criado pelo NIPS-CERN, enxuto o bastante para virar *hardware* eficiente e expressivo o bastante para algoritmos de processamento de sinais, com números complexos como tipo nativo e álgebra linear em notação de Dirac.

Se você já programa em C, vai se sentir em casa em cinco minutos. O que exige atenção não é o que a linguagem tem, e sim o que ela deliberadamente não tem: cada ausência corresponde a um bloco de *hardware* que não precisa ser gerado.

Nota

Tudo nesta seção foi extraído da gramática oficial do compilador, nos arquivos `CMMComp.y` e `CMMComp.l` do repositório `nipsclab/yanc`. Quando o manual diz que algo não existe na linguagem, é porque não existe na gramática, e o compilador rejeitará o código.

15.1 A estrutura de um programa

Um programa C± é uma sequência de três tipos de elemento, em qualquer ordem:

Diretivas

Linhas iniciadas por `#`, que configuram o *hardware* do processador. Detalhadas em [Diretivas: configurando o processador no código](#).

Declarações de variáveis globais

Nomes que ocupam endereços fixos na memória de dados.

Funções

Incluindo a obrigatória `void main()`, que é o ponto de entrada. A ausência dela encerra a compilação com erro.

A forma canônica coloca as diretivas no topo e, em seguida, o `main()` com um laço infinito. O programa modela um circuito, e circuitos não terminam: eles processam amostras para sempre.

Listing 15.1: A forma canônica de um programa SAPHO

```
1 #PRNAME media_movel
2 #NUBITS 16
3 #NDSTAC 4
4 #SDEPTH 2
5 #NUIOIN 1
6 #NUIOOU 1
7 #NBMAINT 10
8 #NBEXPO 5
9
10 void main()
11 {
12     int x[4];    // historico das ultimas 4 amostras
13     int soma;
```

(continua na próxima página)

(continuação da página anterior)

```

14
15 while (1)
16 {
17     x[3] = x[2];           // desloca o historico
18     x[2] = x[1];
19     x[1] = x[0];
20     x[0] = in(0);         // le nova amostra da porta 0
21
22     soma = x[0] + x[1] + x[2] + x[3];
23     out(0, soma >> 2);    // media = soma/4, sem usar divisor
24 }
25 }

```

Esse é o `media_movel`, o processador construído no *Primeiro projeto: um filtro de média móvel* e usado como exemplo ao longo do manual.

15.2 O que ler primeiro

Diretivas As linhas com # que definem a arquitetura: largura da palavra, formato do ponto flutuante, pilhas e portas.

Diretivas: configurando o processador no código Tipos, operadores e controle Os quatro tipos, os operadores disponíveis, as estruturas de controle, funções e vetores.

Tipos, operadores e controle Entrada, saída e biblioteca As portas, as funções intrínsecas, as não lineares, os complexos e a notação de Dirac.

Entrada, saída e biblioteca padrão Recursos avançados Interrupção, sincronização com o *hardware*, FFT, custo de cada construção e o caminho C++.

Recursos avançados

15.3 Um resumo de uma página

Se você conhece C e quer apenas o essencial antes de escrever o primeiro programa, esta tabela basta.

Assunto	Em uma linha
Tipos	Apenas <code>void</code> , <code>int</code> , <code>float</code> e <code>comp</code> (complexo). Não há <code>char</code> , <code>double</code> , <code>struct</code> nem <i>strings</i>
Ponto flutuante	Formato próprio, não IEEE 754, com mantissa e expoente que você escolhe
Operadores ausentes	Sem <code>--</code> , sem <code>+=</code> e afins, sem ternário <code>?:</code> , sem <i>cast</i> explícito, sem ponteiros
Controle	<code>if</code> , <code>else</code> , <code>while</code> , <code>do while</code> , <code>for</code> , <code>switch</code> com <i>fall-through</i> , <code>break</code> , <code>continue</code> , <code>return</code>
Funções	Sintaxe do C, sem recursão e sem passar vetores como parâmetro
Vetores	Uma ou duas dimensões, tamanho fixo, inicializáveis por arquivo de texto
Entrada e saída	Só pelas portas: <code>in()</code> , <code>fin()</code> , <code>out()</code> , <code>fout()</code>
Identificador reservado	<code>i</code> é a unidade imaginária; use <code>k</code> , <code>n</code> ou <code>idx</code> para contadores
Pré-processador	Apenas <code>#define</code> de objeto, sem argumentos, sem <code>#include</code> e sem <code>#ifdef</code>
Comentários	// até o fim da linha e /* ... */ em bloco, como em C

15.4 Onde o arquivo vive

Dentro de um projeto AURORA, o código de cada processador fica em `<projeto>/<processador>/Software/<processador>.cmm`. A compilação gera o *assembly* na mesma pasta `Software`, o *Verilog* e as imagens de memória em `Hardware` e o *testbench* em `Simulation`, conforme *Compilação: de C± a Verilog*.

O nome do arquivo e a diretiva `#PRNAME` precisam concordar: é assim que a cadeia de ferramentas conecta o

fonte aos artefatos. Renomeie sempre pela AURORA, que mantém os dois lados em sincronia.

15.5 Constantes com #define

A C± tem um pré-processador mínimo embutido. A construção `#define NOME corpo` define uma constante simbólica, expandida onde o nome aparecer.

Listing 15.2: Constantes simbólicas

```
#define SCALE 3
#define OFFSET 7

void main()
{
    int x;
    while (1)
    {
        x = in(0);
        x = x + SCALE;
        out(0, x);
        x = x + OFFSET;
        out(0, x);
    }
}
```

Três limitações distinguem esse pré-processador do C tradicional:

- só existem *defines* de objeto, de modo que macros com argumentos, como `#define DOBR0(x) ((x)*2)`, não são aceitas;
- não há `#include` nem `#ifdef` no lado C±, embora o fluxo C++ tenha pré-processador completo, conforme [Recursos avançados](#);
- o limite é de 256 *defines* por programa.

O editor da AURORA reconhece os seus `#define` e os colore como constantes conforme você digita.

15.6 E se eu precisar do que a C± não tem

Existe uma saída. O YANC também compila C++ para o mesmo processador, por uma cadeia paralela que oferece ponteiros, recursão e `struct`. O preço é a visibilidade nas formas de onda. A decisão está detalhada em [o caminho C++](#).

A recomendação prática do laboratório é começar todo projeto em C± e migrar apenas quando o algoritmo exigir especificamente um desses recursos.

Diretivas: configurando o processador no código

As diretivas são as linhas que começam com #, uma por linha, sem ponto e vírgula ao final. Elas não são código executável: são a especificação do *hardware* que será gerado. É por elas que o mesmo programa pode virar um processador de 16 bits com duas portas ou um de 32 bits com oito.

Quando você cria um processador pelo *Hub de Processadores*, o formulário escreve essas diretivas no `.cmm` inicial. A partir daí o arquivo é a fonte da verdade: editar uma diretiva muda o processador na compilação seguinte, sem passar pelo formulário de novo.

16.1 As diretivas de configuração

Aparecem no topo do arquivo. A tabela abaixo é a referência completa; a coluna do padrão indica o valor assumido quando a diretiva é omitida, embora o Hub as escreva todas explicitamente.

Tabela 16.1: Diretivas de configuração da arquitetura

Diretiva	Argumento	Padrão	Efeito
#PRNAME	nome		Nome do processador. Deve casar com o nome do arquivo e da pasta
#NUBITS	inteiro	23	Largura da palavra e dos inteiros, em complemento de dois
#NBMANT	inteiro	16	Bits de mantissa do ponto flutuante próprio
#NBEXP0	inteiro	6	Bits de expoente, em complemento de dois
#NDSTAC	inteiro	10	Profundidade da pilha de dados
#SDEPTH	inteiro	10	Profundidade da pilha de sub-rotinas
#NUI0IN	inteiro	1	Número de portas de entrada; <code>in(p)</code> exige <code>p</code> menor que esse valor
#NUI0OU	inteiro	1	Número de portas de saída; a mesma regra vale para <code>out</code>
#NUGAIN	potência de 2	64	Divisor fixo usado pela função <code>norm()</code>
#FFTSIZ	inteiro	8	Bits invertidos no endereçamento bit-reverso <code>x[k]</code> , para FFT de 2^n pontos

Perigo

A restrição estrutural `#NUBITS` deve ser exatamente igual a `#NBMANT` mais `#NBEXP0` mais um, o bit de sinal.

No exemplo condutor, $16 = 10 + 5 + 1$. A AURORA valida essa igualdade no formulário de criação e recusa combinações inválidas, mas quem edita as diretivas à mão responde por mantê-la.

16.2 Como escolher cada valor

A tabela diz o que cada diretiva faz. Esta seção diz como decidir.

#NUBITS, a largura da palavra

Dimensiona os inteiros e boa parte do consumo de *hardware*. Dezesesseis bits bastam para muitos sinais de instrumentação; o padrão de 23 dá folga com um `float` de boa precisão. Lembre que ela está amarrada às duas seguintes pela restrição estrutural.

#NBMANT e #NBEXP0, o formato do ponto flutuante

Definem precisão e faixa. Com dez bits de mantissa, espere cerca de três dígitos decimais significativos. Se o programa só usa inteiros, esses valores têm pouco efeito prático, mas a igualdade continua obrigatória. Veja [O ponto flutuante do SAPHO](#).

#SDEPTH, a pilha de sub-rotinas

Limita a profundidade de chamadas de função aninhadas. Se o seu `main()` chama `a()`, que chama `b()`, você precisa de pelo menos três níveis. Os padrões atendem programas típicos.

#NDSTAC, a pilha de dados

Limita a complexidade das expressões. Uma expressão como $a*b + c*d - e*f$ empilha resultados parciais; expressões muito aninhadas exigem mais profundidade.

#NUIOIN e #NUI00U, as portas

Reserve uma porta para cada fluxo de dados que o processador troca com o mundo. Com uma única porta a ligação é direta; com mais de uma, o *hardware* instancia automaticamente um decodificador de endereços.

#NUGAIN, o ganho

Só importa se o programa usa `norm()`. Deve ser potência de dois, porque é isso que permite implementar a divisão como deslocamento.

#FFTSIZ, o tamanho da FFT

Só importa se o programa usa a indexação bit-reversa `x[k]`. Veja [Recursos avançados](#).

Dica

Ao ajustar parâmetros, mude um de cada vez e recompile. O terminal TASM informa, a cada compilação, o percentual do conjunto de instruções e da unidade lógica e aritmética efetivamente usados, o que dá uma medida direta do efeito de cada mudança.

16.3 As diretivas comportamentais

Duas diretivas fogem ao padrão: aparecem no meio do programa e marcam pontos do código, em vez de configurar a arquitetura.

#PRACA

Marca o ponto para onde o processador desvia quando o pino de interrupção pulsa. O uso típico é reiniciar o laço de processamento quando o *hardware* externo sinaliza um novo evento, como um gatilho de aquisição, sem esperar o programa completar a volta.

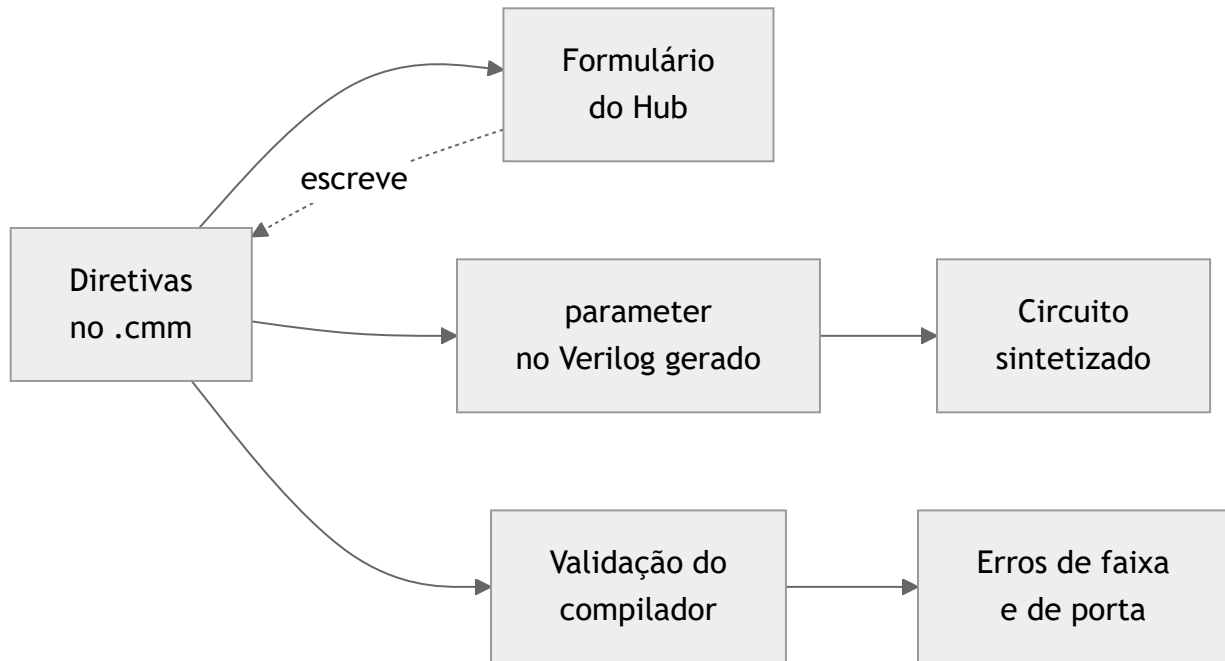
#TOAQUI

Marca um endereço cujo alcance faz pulsar o pino `chequei` do processador. Serve como farol para sincronizar blocos externos com fases do algoritmo, e é também o mecanismo interno pelo qual a AURORA detecta o término do programa nos testes de *hardware* e nas simulações com o Verilator.

Ambas são detalhadas, com exemplos, em [Recursos avançados](#).

16.4 Onde as diretivas aparecem depois

Vale saber o destino de cada valor, porque ele reaparece em três lugares diferentes ao longo do fluxo.



No Verilog gerado, cada diretiva vira um `parameter` do módulo, o que torna o circuito legível e ajustável. No compilador, elas viram regras de validação: usar `in(2)` em um processador com `#NUI0IN 1` é erro de compilação, assim como um literal inteiro que não cabe em `#NUBITS` bits.

16.5 Erros comuns com diretivas

Sintoma	Causa e solução
O botão <i>Gerar Processador</i> não habilita Erro de porta inexistente	A igualdade estrutural foi violada, ou o ganho não é potência de dois. O índice em <code>in()</code> ou <code>out()</code> excede <code>#NUI0IN</code> ou <code>#NUI00U</code> . Aumente a diretiva e recompile.
Erro de estouro de inteiro	Um literal ou resultado não cabe em <code>#NUBITS</code> bits em complemento de dois.
Erro de faixa de <code>float</code>	O valor não cabe no formato definido por <code>#NBMANT</code> e <code>#NBEXP0</code> .
Os artefatos não são encontrados	<code>#PRNAME</code> não corresponde ao nome do arquivo e da pasta. Renomeie pela AURORA.

16.6 Referência rápida

A tabela completa de diretivas, com argumentos e padrões, também está no apêndice [Referência de diretivas](#), para consulta sem sair da página de referência.

Tipos, operadores e controle

Esta página cobre o núcleo da linguagem: os quatro tipos, os operadores, as estruturas de controle, as funções e os vetores. Se você programa em C, leia com atenção especial as caixas de aviso: elas listam o que a C $\pm$ não tem, e é aí que mora a maioria dos erros de compilação de quem está começando.

17.1 Os quatro tipos

A C $\pm$ tem exatamente quatro palavras-chave de tipo.

void

Marca a ausência de tipo, no retorno de funções que não retornam nada.

int

O inteiro de #NUBITS bits, em complemento de dois. É o tipo de trabalho da maior parte dos programas de instrumentação.

float

O ponto flutuante no formato próprio do SAPHO, com mantissa de #NBMANT bits e expoente de #NBEXPO bits. Não segue o padrão IEEE 754; veja [O ponto flutuante do SAPHO](#).

comp

O número complexo, um par de float com as partes real e imaginária, tratado como tipo de primeira classe da linguagem.

Aviso

O que não existe Não há tipo de ponto fixo separado, papel que cabe ao float configurável. E não existem char, double, struct, union, enum nem strings. Um literal de texto só aparece na inicialização de vetores por arquivo, mostrada adiante.

17.1.1 Literais complexos e o i reservado

Um literal complexo escreve-se como em matemática, a+bi:

```
comp z;  
z = 3.0 + 4.0i;
```

⚠ Perigo

O identificador `i` é reservado. Por causa da notação acima, `i` designa a unidade imaginária, e usá-lo como nome de variável é erro de compilação. Adote `k`, `n` ou `idx` para contadores de laço.

Este é, de longe, o erro mais comum de quem chega do C, onde `for (i = 0; ...)` é reflexo muscular.

17.1.2 Conversões entre tipos

Misturar `int`, `float` e `comp` em uma expressão ou atribuição é permitido, mas o compilador emite avisos de conversão implícita, por exemplo ao guardar um `float` em um `int` com perda. Usar `float` ou `comp` como condição de `if` ou `while`, ou como índice de vetor, também gera aviso.

Trate esses avisos como cheiro de projeto e não como ruído: conversões explícitas e índices inteiros deixam o *hardware* gerado mais barato e o comportamento mais óbvio.

17.2 Operadores

Tabela 17.1: Operadores da C±, com precedência igual à do C

Grupo	Operadores
Aritméticos	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> (módulo, só inteiros) e o <code>-</code> unário
Bit a bit	<code>&</code> <code> </code> <code>^</code> <code>~</code> (inversão)
Deslocamentos	<code><<</code> <code>>></code> <code>>>></code> (à direita, preservando o sinal)
Relacionais	<code><</code> <code>></code> <code><=</code> <code>>=</code> <code>==</code> <code>!=</code>
Lógicos	<code>&&</code> <code> </code> <code>!</code>
Incremento	<code>++</code> , como comando ou em expressão, inclusive sobre elemento de vetor

⚠ Aviso

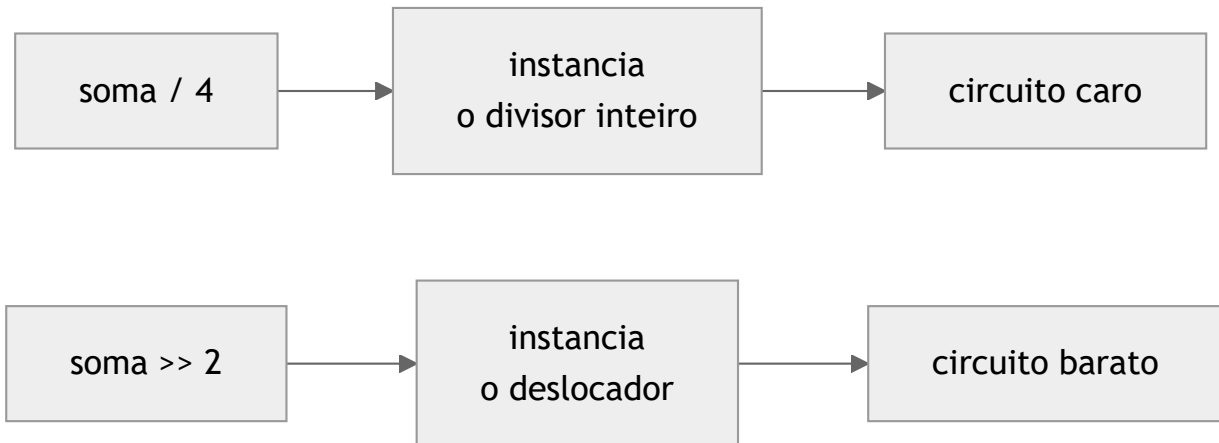
O que não existe Não existem em C±: o decremento `--`; os operadores compostos `+=`, `-=`, `*=` e afins; o operador ternário `? :`; o `cast` explícito; e os ponteiros. Os símbolos `*` e `&` são apenas multiplicação e E bit a bit.

Escreva `x = x - 1;` onde escreveria `x--;`, e `x = x + 2;` onde escreveria `x += 2;`.

17.2.1 Operadores custam hardware

Aqui está a diferença mais importante entre escrever para o SAPHO e escrever para um processador comum.

Cada operador usado pela primeira vez no programa instancia o circuito correspondente na unidade lógica e aritmética do processador gerado. O terminal TASM anuncia cada instância durante a compilação, e o percentual final indica quanto do conjunto de instruções o seu programa efetivamente usa.



Divisão e módulo são os blocos mais caros; os deslocamentos, os mais baratos. Daí o idioma do exemplo condutor, `soma >> 2` em vez de `soma / 4`. A aritmética de complexos aceita as quatro operações, cada uma expandindo para o conjunto de operações reais e imaginárias necessárias.

Dica

É a primeira ocorrência de um operador que paga o bloco. A segunda em diante reaproveita o circuito. Não há economia em evitar a segunda divisão de um programa que já divide uma vez.

17.3 Estruturas de controle

Todas as estruturas clássicas do C existem, com a mesma sintaxe:

- `if` com `else`;
- `while` e `do while`;
- `for`, que o compilador reescreve internamente como um `while` equivalente;
- `switch` com `case` e `default`, incluindo o *fall-through* real do C, em que a execução continua no próximo `case` na ausência de `break`;
- `break`, `continue` e `return`, os dois primeiros válidos apenas dentro de laços.

Listing 17.1: Um saturador, exercitando `if` e `return`

```

int satura(int x, int lim)
{
    if (x > lim) return lim;
    if (x < -lim) return -lim;
    return x;
}
  
```

17.4 Funções

As funções seguem a sintaxe do C, `tipo nome(tipo p1, tipo p2, ...)`, com chamadas por argumentos posicionais e retorno por `return`. O número de argumentos é conferido em cada chamada, e a contagem errada é erro de compilação; conversões de tipo em parâmetros e no retorno geram aviso. A profundidade de chamadas aninhadas é limitada pela pilha de sub-rotinas, a diretiva `#SDEPTH`.

Listing 17.2: Funções com parâmetros e retorno

```

1 int add(int a, int b)
2 {
3     return a + b;
4 }
5
6 int triple_sum(int x, int y, int z)
7 {
8     return x + y + z;
9 }
10
11 void main()
12 {
13     while (1)
14     {
15         int v = in(0);
16         out(0, add(v, 1));
17         out(0, triple_sum(v, 1, 2));
18     }
19 }

```

⚠ Aviso

Duas restrições que surpreendem **Vetores não podem ser passados como parâmetro**. Trabalhe com vetores globais ou dentro do próprio `main()`.

A recursão não é suportada no fluxo C±. Cada variável, incluindo parâmetros e locais, vive em um endereço fixo da memória de dados, de modo que uma função que chamasse a si mesma sobrescreveria os próprios dados.

Essa segunda restrição tem uma contrapartida valiosa: é justamente o endereçamento fixo que permite à AURORA mostrar cada variável pelo nome nas formas de onda. Você troca recursão por visibilidade total da simulação, o que em depuração de *hardware* costuma ser o melhor negócio. Se o algoritmo é essencialmente recursivo, o caminho C++ atende, conforme [Recursos avançados](#).

17.5 Vetores

A linguagem aceita vetores de uma e duas dimensões, com tamanho fixo, e uma forma de inicialização peculiar e muito útil:

Listing 17.3: As três formas de declarar um vetor

```

int x[128];           // sem inicializacao
int h[8]   "coefs.txt"; // inicializado por arquivo, em tempo de compilacao
int m[8][8] "matriz.txt"; // 2D, tambem por arquivo

```

A inicialização por arquivo lê os valores de um arquivo de texto ao lado do fonte e os grava diretamente na imagem da memória de dados do processador. Os coeficientes do seu filtro já nascem na memória, sem nenhum custo de inicialização em tempo de execução.

💡 Dica

Esse mecanismo é o que torna prático embarcar modelos treinados fora do SAPHO. Uma rede neural

convolucional portada para o processador, por exemplo, carrega seus milhares de pesos exatamente assim, um arquivo .txt por camada.

17.5.1 As duas formas de indexação

A normal, $x[k]$, e a bit-reversa, $x[k]$, que se distingue pelo fecha-parênteses. Na segunda, o índice é lido com os bits invertidos, que é o endereçamento clássico da FFT (*Fast Fourier Transform*), com a quantidade de bits definida pela diretiva `#FFTSIZ`. Veja [Recursos avançados](#).

Elementos de vetor aceitam o incremento, como em `hist[k]++;`.

17.6 Variáveis e escopo

Declarações podem aparecer no topo do programa ou dentro de funções e blocos. Na prática, porém, todo nome vive em um endereço fixo e único da memória de dados, e declarar duas variáveis com o mesmo nome é erro.

ⓘ Importante

Pense nas variáveis C± como registradores nomeados do seu circuito. É exatamente isso que elas se tornam, e é por isso que cada uma aparece pelo nome na forma de onda da simulação.

17.7 O próximo passo

Com tipos e controle assentados, falta a parte que conversa com o mundo: [Entrada, saída e biblioteca padrão](#) cobre as portas, as funções intrínsecas e a notação de Dirac.

Entrada, saída e biblioteca padrão

Um processador que não conversa com o mundo não serve para nada. Esta página cobre as portas, pelas quais o SAPHO recebe e devolve dados, e a biblioteca de funções intrínsecas que a linguagem oferece, incluindo o recurso mais idiossincrático da C±: a álgebra linear escrita em notação de Dirac.

18.1 As portas de entrada e saída

O processador conversa com o exterior por portas numeradas, criadas pelas diretivas `#NUI0IN` e `#NUI00U`. Todo o tráfego passa por quatro funções intrínsecas.

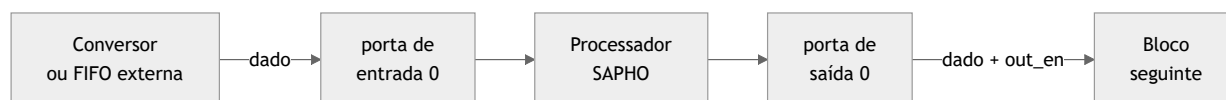
Função	Tipo	O que faz
<code>in(p)</code>	<code>int</code>	Lê um inteiro da porta de entrada <code>p</code>
<code>fin(p)</code>	<code>float</code>	Lê da porta <code>p</code> convertendo para <code>float</code>
<code>out(p, x);</code>	qualquer	Escreve o valor de <code>x</code> na porta de saída <code>p</code>
<code>fout(p, x);</code>	<code>float</code>	Escreve em formato <code>float</code>

O número da porta é um literal inteiro, validado contra as diretivas: usar `in(2)` em um processador com `#NUI0IN 1` é erro de compilação.

No exemplo condutor, uma porta de cada lado basta: `x[0] = in(0);` lê a amostra e `out(0, soma >> 2);` publica a média.

18.1.1 O que acontece no hardware

Vale saber, porque é isso que o projeto de FPGA precisa conectar. A leitura por `in()` é um *handshake*, no qual o processador sinaliza `req_in` e aguarda o dado. A escrita por `out()` pulsa `out_en` com o dado estável no barramento. Esses sinais bastam para acoplar FIFOs, conversores analógico-digitais e outros blocos.



Na simulação, o *testbench* gerado assume esse papel: alimenta cada porta de entrada com o conteúdo de `input_<i>.txt`, um valor por linha, e grava cada porta de saída em `output_<i>.txt`.

18.2 Funções especiais

Cinco funções existem por um motivo bem específico: economizar *hardware*. Cada uma evita um bloco caro da unidade lógica e aritmética ou uma sequência de instruções.

norm(x)

Divide pelo valor da diretiva `#NUGAIN`, que é uma potência de dois, entregando a normalização sem instanciar o divisor completo. Funciona só com inteiros.

pset(x)

Devolve `x` quando positivo e zero quando negativo. É o equivalente de `if (x<0) x=0;` em uma única instrução, e também a forma natural de implementar a função de ativação ReLU de redes neurais.

abs(x)

Valor absoluto e, sobre um complexo, a magnitude.

sign(x, y)

Devolve y com o sinal de x.

copy(x, y);

Copia os bits de x em y sem conversão nem checagem de tipo. É uma reinterpretação crua, a ser usada com consciência.

18.3 Funções não lineares e de arredondamento

A biblioteca cobre raiz e trigonometria (`sqrt`, `sin`, `cos`, `tan`, `atan`), as hiperbólicas (`sinh`, `cosh`, `tanh`), exponencial e logaritmo (`exp`, `log`, `pow`) e os arredondamentos `floor`, `ceil` e `round`, que retornam `float`, com o `round` arredondando o meio para longe do zero.

ⓘ Importante

Elas custam ciclos, não blocos Essas funções não são circuitos dedicados: são macros de *assembly* otimizadas, como o método de Newton na `sqrt` e séries nas trigonométricas, injetadas no programa quando usadas.

Elas custam instruções e tempo de execução, não área de *hardware*, exceto pelas operações de ponto flutuante que elas próprias empregam. É o contrário do que acontece com os operadores aritméticos.

A `pow(x, y)` merece nota, porque o compilador escolhe entre três estratégias: com expoente inteiro constante, gera a sequência mínima de multiplicações; com expoente inteiro variável, um laço em tempo de execução; nos demais casos, a identidade $x^y = e^{y \ln x}$.

18.4 Funções para complexos

Seis funções operam sobre o tipo `comp`:

Função	O que devolve
<code>real(z)</code> , <code>imag(z)</code>	As partes real e imaginária
<code>fase(z)</code>	O argumento, ou ângulo
<code>mod2(z)</code>	A magnitude ao quadrado, $a^2 + b^2$, mais barata que <code>abs</code> por dispensar a raiz
<code>complex(re, im)</code>	Monta um complexo a partir de dois <code>float</code>
<code>conj(z)</code>	O conjugado

Listing 18.1: Multiplicação de complexos e emissão das partes

```
comp x; comp y; comp r;
x = 1.0 + 2.0i;
y = 3.0 + 4.0i;
r = x * y;
fout(0, real(r));
fout(0, imag(r));
```

⚠ Aviso

Nem tudo se aplica a `comp` `sqrt`, `exp`, `log`, `pow`, `sign` e `pset` sobre complexos são erro de compilação, assim como o incremento `z++`. O módulo `%` e a `norm()` são exclusivos de inteiros.

Dica

Quando só interessa comparar magnitudes, prefira `mod2()` a `abs()`. A comparação dá o mesmo resultado e você evita instanciar a raiz quadrada.

18.5 Álgebra linear em notação de Dirac

O recurso mais distintivo da $C\pm$ é a escrita de operações vetoriais e matriciais em notação *bra-ket*, familiar a quem vem da física. Sendo a e b vetores e M uma matriz:

Tabela 18.1: Construções em notação de Dirac

Sintaxe	Operação
$\langle a b \rangle$	Produto interno, usado como expressão
$a \# 0 \rangle;$	Zera o vetor
$a \# M b \rangle;$	Produto matriz-vetor
$a \# c b \rangle;$	Vetor escalado por uma constante
$A \# a \rangle \langle b ;$	Produto externo
<code>out(p, c a>);</code>	Emite o vetor escalado por uma porta de saída

Essas construções expandem para os laços e instruções indexadas adequados, e são a forma idiomática de escrever filtros FIR, correlações e projeções em $C\pm$.

Um filtro de resposta finita ao impulso, por exemplo, cabe em uma linha:

Listing 18.2: Um FIR completo em notação de Dirac

```

1  int h[16] "coefs.txt";    // coeficientes, carregados na compilacao
2  int x[16];                // janela de amostras
3  int y;
4
5  void main()
6  {
7      int k;
8      while (1)
9      {
10         for (k = 15; k > 0; k = k - 1)
11             x[k] = x[k-1];
12         x[0] = in(0);
13
14         y = <h|x>;          // produto interno: o filtro inteiro
15         out(0, y);
16     }
17 }
```

Compare com a alternativa: um laço acumulando $y = y + h[k]*x[k]$. O resultado compilado é equivalente, mas a intenção fica muito mais legível, e é assim que os projetos do laboratório são escritos.

18.6 Referência rápida

A tabela completa da biblioteca, agrupada por família, está em [Referência da biblioteca padrão](#).

18.7 O próximo passo

[Recursos avançados](#) cobre a interrupção, a sincronização com o *hardware* externo, o endereçamento de FFT, o custo de cada construção e o caminho alternativo em C++.

Recursos avançados

Esta página reúne o que você não precisa no primeiro programa, mas vai querer no terceiro: interrupção, sincronização com o *hardware* externo, endereçamento de FFT, o custo em área de cada construção e o caminho alternativo em C++. Fecha com a leitura das mensagens do compilador.

19.1 Interrupção: #PRACA

O processador SAPHO tem um pino de interrupção, o *itr*. Quando ele pulsa, o contador de programa desvia para o ponto do código marcado pela diretiva `#PRACA`, que, diferentemente das diretivas de configuração, aparece no meio do programa e marca a linha de retorno.

O uso típico é reiniciar o laço de processamento quando o *hardware* externo sinaliza um novo evento, como um gatilho de aquisição, sem esperar o programa completar a volta.

Listing 19.1: Estrutura de um programa acionado por interrupção

```
1 void main()
2 {
3     int amostra;
4
5     #PRACA                // a interrupcao devolve a execucao para aqui
6     while (1)
7     {
8         amostra = in(0);
9         // processamento de uma janela completa
10        out(0, amostra);
11    }
12 }
```

Esse padrão é o que sustenta aplicações de aquisição em tempo real, nas quais o processador precisa abandonar o trabalho em curso assim que uma nova janela de dados chega.

19.2 Sincronização com o hardware: #TOAQUI

A diretiva `#TOAQUI`, colocada em um ponto do programa, faz o pino *cheeguei* do processador pulsar sempre que a execução passa por ali. É um farol para o mundo externo, útil para sincronizar blocos de *hardware* com fases do algoritmo.

Ela é também um mecanismo interno da própria AURORA: no teste do processador sintetizado e nas simulações com o Verilator, a IDE insere um `#TOAQUI` ao final do `main()` para detectar o término do programa e encerrar a simulação no instante certo.

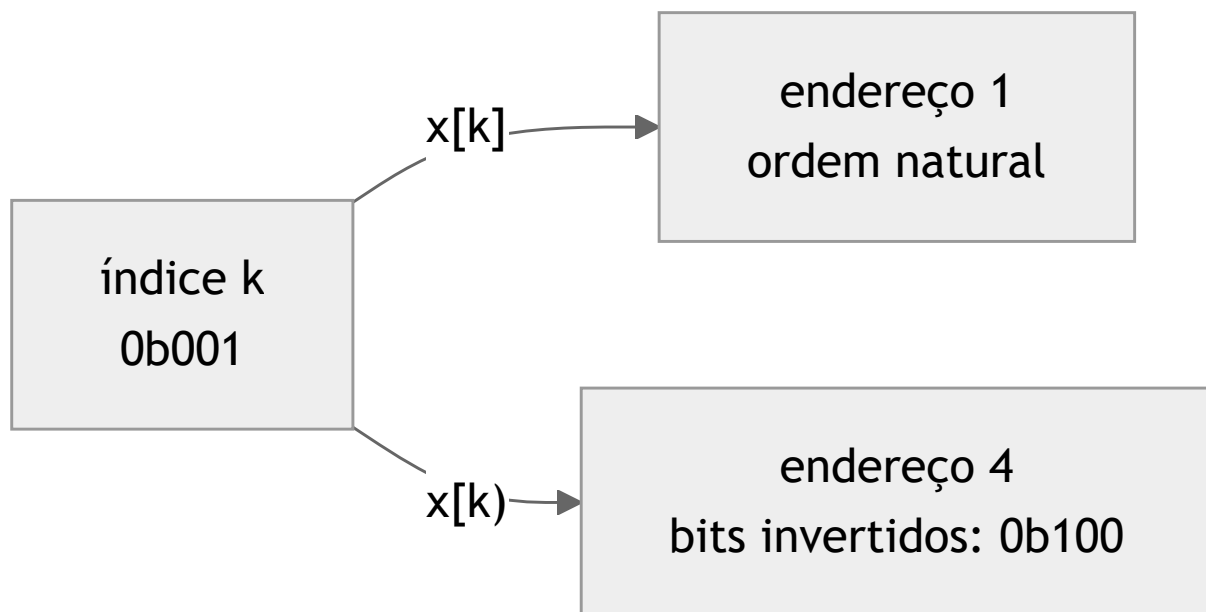
Dica

Se você escreve o seu próprio *testbench* e precisa saber quando o algoritmo terminou uma passada, `#TOAQUI` no ponto certo é mais confiável do que contar ciclos.

19.3 FFT e o endereçamento bit-reverso

A diretiva `#FFTSIZ n` configura o circuito de reversão de bits para transformadas de 2^n pontos, usado pela indexação $x[k]$ apresentada em *Tipos, operadores e controle*.

Na FFT radix-2 os resultados saem em ordem bit-reversa, e reordená-los custaria um laço inteiro. Com $x[k]$ a reordenação é gratuita: o *hardware* inverte os bits do índice na própria leitura ou escrita, sem instruções extras.



O repositório do YANC traz um projeto de exemplo completo, o `proc_fft`, com uma FFT inteira escrita em C±.

19.4 Quanto custa cada construção

O princípio de pagar apenas pelo que se usa tem uma consequência prática direta: o custo em FPGA do processador é proporcional ao subconjunto da linguagem que o programa emprega.

Durante a compilação, o terminal TASM informa cada bloco de *hardware* instanciado à medida que os *opcodes* aparecem pela primeira vez. Ao final, ele resume o percentual do conjunto de instruções e da unidade lógica e aritmética efetivamente usados.

Tabela 19.1: O que cada construção liga no circuito

Construção no programa	O que instancia no hardware
Chamar uma função	A memória da pilha de sub-rotinas
Usar um vetor	O circuito de endereçamento indexado
Usar a forma $x[k]$	O circuito de reversão de bits
Dividir com $/$	O divisor inteiro, um dos blocos mais caros
Usar $\%$	O bloco de módulo
Operar em <code>float</code>	Os blocos de ponto flutuante correspondentes a cada operação usada
Deslocar com $\ll$, $\gg$, $\ggg$	O deslocador, um dos blocos mais baratos

19.4.1 Regras de bolso para hardware enxuto

- prefira deslocamentos a divisões por potências de dois, ou use `norm()`;
- prefira `mod2()` a `abs()` quando só compara magnitudes, evitando a raiz quadrada;
- evite `float` quando a dinâmica do sinal cabe em inteiros;
- lembre que é a primeira ocorrência de um operador que paga o bloco, pois a segunda em diante reaproveita o circuito.

Dica

O experimento que ensina isso em dois minutos Compile o exemplo condutor, abra o PRISM e guarde o diagrama na memória. Troque `soma >> 2` por `soma / 4`, recompile e clique em *Recompile*. O divisor aparece no desenho, e o TASM anuncia o bloco novo. Desfaça e ele some.

19.5 O caminho C++

Além da C $\pm$, o YANC compila C++ para o mesmo processador, por uma cadeia paralela: o pré-processador `cpppp`, com `#include`, `#define` completo e compilação condicional, seguido do compilador `cppcomp`, que aceita um subconjunto de C99 com extensões de C++ e emite o mesmo *assembly* intermediário.

O que o C++ oferece a mais

- ponteiros, restritos à indexação de vetores de tamanho fixo, sem alocação dinâmica;
- recursão, com quadros de pilha de verdade;
- `struct` simples;
- recursos modernos como *lambdas* e destrutores;
- os cabeçalhos padrão adaptados `array`, `cmath`, `cstdint` e `vector`.

O que se perde Variáveis locais deixam de ter endereço fixo e, com isso, deixam de aparecer pelo nome nas formas de onda. Perde-se em parte o vínculo íntimo entre código e onda que marca o fluxo C $\pm$.

Os parâmetros do processador nesse fluxo têm padrões próprios, palavra de 32 bits com mantissa de 23 e expoente de 8, ajustáveis por `#pragma yanc`.

i Nota

Recomendação prática do laboratório Comece todo projeto em C $\pm$ e migre para C++ apenas quando precisar especificamente de recursão, ponteiros ou `struct`. Mantenha em C $\pm$ os processadores de sinal críticos, nos quais a visibilidade de simulação é total.

19.6 As mensagens do compilador

Os diagnósticos do YANC são bilíngues, acompanhando o idioma da IDE, e têm personalidade: um erro de sintaxe rende um comentário bem-humorado, e esquecer o `main()` provoca a pergunta clássica sobre onde ele está. Por trás do humor, a informação é séria e cai em poucas classes.

Erros de estrutura

- falta do `main()`;
- `break` ou `continue` fora de laço;
- contagem errada de parâmetros em uma chamada.

Erros de nome

- variável inexistente;
- variável já declarada;
- uso do `i` reservado como identificador.

Erros de tipo

- módulo `%` e `norm()` com operandos não inteiros;
- funções intrínsecas sem versão para complexos;
- incremento de um `comp`.

Erros de faixa

- estouro de inteiro para o `#NUBITS` escolhido;
- `float` fora da faixa do formato configurado;
- porta de entrada ou saída inexistente.

Avisos

- conversões implícitas entre tipos;
- uso de `float` ou `comp` em condições e índices.

Os erros aparecem no terminal TCMM com a linha clicável, e o clique leva o editor ao ponto exato.

 Dica

A Aurora Intelligence sabe explicar qualquer mensagem do compilador: ela consulta o catálogo bilíngue de mensagens gerado a partir das próprias fontes do YANC, e não depende da memória do modelo. Veja *Tarefas com a Aurora Intelligence*.

19.7 Leitura relacionada

- *O processador SAPHO por dentro* explica por que o endereçamento fixo impede a recursão.
- *O conjunto de instruções* ajuda a ler as trilhas de *assembly* nas formas de onda.
- *Compilação: de C $\pm$ a Verilog* detalha o que cada compilador da cadeia produz.

O processador SAPHO

O processador SAPHO por dentro

Antes de ajustar parâmetros com confiança, vale entender o que exatamente está sendo criado. Esta página descreve a arquitetura do *soft-processor* SAPHO do ponto de vista de quem o usa: o que os parâmetros significam, como a máquina executa o programa e quais são as suas restrições.

Nada aqui precisa ser decorado para usar a plataforma. Mas quem entende a máquina escreve programas melhores para ela, e principalmente entende por que certas construções da linguagem custam caro.

20.1 Visão geral

O SAPHO é um processador de acumulador com arquitetura Harvard.

Processador de acumulador significa que não existe um banco de registradores. Existe um único registrador central, o acumulador (ACC), que recebe o resultado de toda operação da unidade lógica e aritmética (ULA). O padrão dominante é buscar um operando na memória de dados, tomar o próprio acumulador como segundo operando e devolver o resultado ao acumulador, com uma pilha de dados fornecendo o segundo operando quando a expressão exige.

Arquitetura Harvard significa que as memórias de programa e de dados são fisicamente separadas, cada uma com a sua largura e o seu barramento. O programa não pode se sobrescrever, e instruções e dados são acessados em paralelo.

20.1.1 Os três traços que definem o estilo da máquina

Totalmente parametrizável Toda largura é um parâmetro Verilog: da palavra de dados à mantissa e ao expoente do ponto flutuante, passando pelas profundidades das memórias e das pilhas.

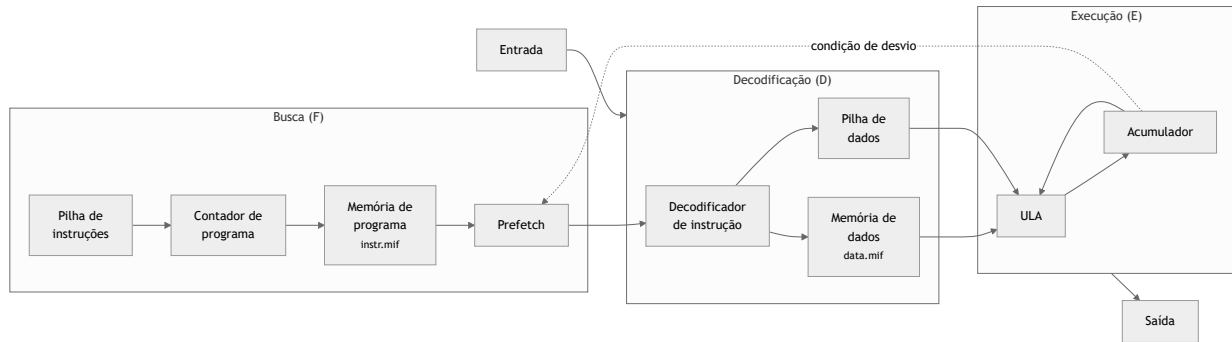
Paga-se só pelo que se usa Cada instrução do conjunto é um parâmetro booleano. O montador liga somente as instruções que o programa emprega, e o *hardware* das demais é eliminado na geração do circuito.

Ciclo curto e previsível Um *pipeline* raso de três estágios, sem *forwarding*. Como a ULA é combinacional e acumula a cada ciclo, nem os saltos condicionais quebram o fluxo.

Esse último ponto merece ênfase, porque é raro. Em instrumentação, saber que qualquer algoritmo executa sem interromper a cadeia de busca, decodificação e execução vale mais do que o desempenho de pico de uma máquina superescalar: o tempo de resposta é determinístico.

20.2 O caminho de dados

O processador executa em três estágios síncronos: busca (F), decodificação (D) e execução (E).



Repare na seta tracejada: a condição de desvio realimenta a busca a partir do acumulador. É esse laço curto que permite executar saltos condicionais e chamadas de função sem bolhas no *pipeline*.

20.3 Os módulos gerados

O processador compõe-se de poucos módulos Verilog, criados a partir de moldes parametrizáveis que acompanham o YANC. Reconhecê-los é útil ao navegar no PRISM, que desenha cada um com símbolo próprio.

O módulo `processor` é o topo e instancia o núcleo e as duas memórias: `mem_instr`, a memória de programa, somente leitura, carregada com a imagem `<proc>_inst.mif`, e `mem_data`, a memória de dados, carregada com `<proc>_data.mif` e capaz de uma leitura e uma escrita por ciclo.

O núcleo, `core`, reúne o caminho de dados com o acumulador, as pilhas, a busca de instruções e o contador de programa, apoiado pelo decodificador `instr_dec` e pela `ula`, na qual cada operação é um submódulo e somente os usados sobrevivem. Completam o conjunto o decodificador de endereços `addr_dec`, para múltiplas portas, e a FIFO genérica `myFIFO`, útil para acoplar o processador ao mundo externo sem depender de propriedade intelectual de fabricante.

20.4 Os parâmetros fundamentais

Ao criar um processador você escolhe, direta ou indiretamente, os valores da tabela abaixo. Eles aparecem como diretivas no topo do arquivo C++ e como `parameter` no Verilog gerado.

Tabela20.1: Parâmetros de arquitetura e as diretivas correspondentes

Parâmetro	Diretiva	Significado
NUBITS	#NUBITS	Largura da palavra de dados. Todo inteiro do programa tem esse tamanho, em complemento de dois
NBMANT	#NBMANT	Largura da mantissa do ponto flutuante
NBEXPO	#NBEXPO	Largura do expoente do ponto flutuante
	#NDSTAC	Profundidade da pilha de dados
	#SDEPTH	Profundidade da pilha de sub-rotinas
	#NUIOIN	Número de portas de entrada
	#NUIOOU	Número de portas de saída
	#NUGAIN	Constante de divisão da função <code>norm()</code>
	#FFTSIZ	Tamanho da FFT, 2^{FFTSIZ} pontos, no endereçamento com reversão de bits

⚠ Perigo

A palavra precisa comportar exatamente o formato de ponto flutuante: NUBITS deve ser igual a NBMANT mais NBEXP0 mais um, o bit de sinal. A AURORA valida essa igualdade no formulário de criação e recusa combinações inválidas.

20.5 Entrada, saída e o mundo externo

O processador conversa com o exterior por portas numeradas, criadas conforme #NUI0IN e #NUI00U e acessadas no programa por `in()`, `out()`, `fin()` e `fout()`. No *hardware*, cada porta vira sinais de dados e de controle no topo do módulo `processor`; é por eles que o projeto de FPGA conecta o processador a conversores, FIFOs e outros blocos.

Dois recursos opcionais completam a interface:

A interrupção

Faz o processador desviar para um ponto configurado do programa quando o pino `itr` pulsa, marcado no código pela diretiva `#PRACA`.

O marcador #TOAQUI

Gera um pino chamado `cheguei`, pulsado sempre que o programa passa pelo ponto marcado, o que permite sincronizar *hardware* externo com fases do algoritmo.

20.6 Por que não há recursão

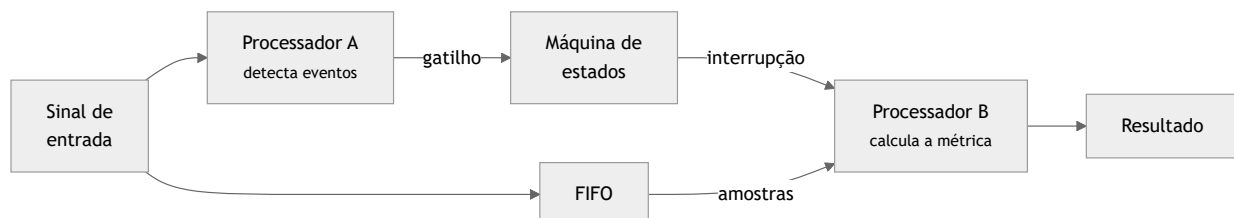
Esta é a restrição que mais confunde quem chega do *software*, e a explicação está na arquitetura.

Cada variável do programa, incluindo parâmetros e locais, vive em um endereço fixo e único da memória de dados. Não há quadros de pilha: a pilha de sub-rotinas guarda endereços de retorno, não contextos. Uma função que chamasse a si mesma sobrescreveria os próprios dados na segunda entrada.

A contrapartida é valiosa: é justamente esse endereçamento fixo que permite à AURORA mostrar cada variável pelo nome nas formas de onda, ciclo a ciclo. Você troca recursão por observabilidade total, o que em projeto de *hardware* costuma ser o melhor negócio. Quando o algoritmo é essencialmente recursivo, o caminho C++ atende, conforme [Recursos avançados](#).

20.7 Um processador por tarefa

Como cada processador é gerado sob medida, o padrão de projeto no SAPHO difere do mundo dos processadores fixos. Em vez de uma máquina grande que faz tudo, criam-se várias máquinas pequenas, uma por tarefa, conectadas por um *top-level* em Verilog.



O diagrama acima é a forma de um caso real do laboratório: um detector de novidade no qual um processador segmenta o sinal pelos cruzamentos por zero e dispara, por interrupção, um segundo processador que drena a FIFO e mede a distância até uma referência.

A AURORA suporta esse fluxo diretamente. Um projeto pode conter quantos processadores forem necessários, cada um com a sua pasta, o seu fonte e os seus parâmetros, todos integrados por um *top-level* e um

testbench comuns, conforme [Projetos](#).

20.8 Leitura relacionada

- *O ponto flutuante do SAPHO* detalha o formato numérico próprio e a precisão que se pode esperar dele.
- *O conjunto de instruções* lista as famílias de *opcodes* e ensina a ler as trilhas de *assembly* nas ondas.
- *Recursos avançados* mostra o custo em área de cada construção da linguagem.

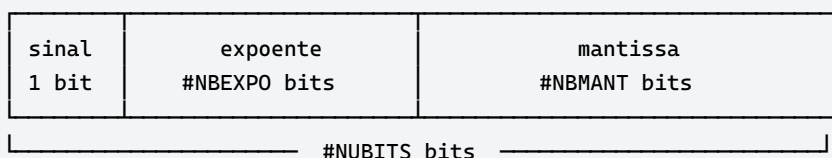
O ponto flutuante do SAPHO

O SAPHO tem *hardware* de ponto flutuante próprio, que não segue o padrão IEEE 754. Esta página explica o formato, o que se pode esperar dele em precisão e por que a escolha faz sentido em um processador gerado sob demanda.

Se o seu programa só usa inteiros, a leitura é opcional. Se ele usa `float` ou `comp`, vale conhecer o terreno.

21.1 O formato

O número reúne três campos:



Um bit de sinal, #NBEXPO bits de expoente em complemento de dois e #NBMANT bits de mantissa com o bit líder explícito. A soma dos três é exatamente #NUBITS, que é a origem da restrição estrutural cobrada pelo formulário de criação:

$$\text{NUBITS} = \text{NBMANT} + \text{NBEXPO} + 1$$

21.2 Por que não IEEE 754

A pergunta é justa, e a resposta é o princípio da plataforma. O padrão IEEE 754 fixa precisões: 32 bits com 23 de mantissa e 8 de expoente, ou 64 bits com 52 e 11. Um processador gerado sob medida não tem motivo para aceitar essa imposição.

Com o formato próprio, você escolhe precisões arbitrárias. Uma mantissa de 10 bits com expoente de 5, como no exemplo condutor, economiza *hardware* exatamente onde a aplicação permite: se o sinal que você processa tem três dígitos significativos úteis, pagar por 23 bits de mantissa é desperdiçar área de FPGA em zeros.

🔔 Importante

A escolha do formato é uma decisão de projeto, não um detalhe. Em um processador que roda em tempo real dentro de um detector de partículas, cada elemento lógico economizado é orçamento para outro canal de leitura.

21.3 O que esperar em precisão

A precisão obtida é exatamente a configurada. Com #NBMANT 10, espere cerca de três dígitos decimais significativos. A regra prática é que cada bit de mantissa vale aproximadamente 0,3 dígito decimal.

Tabela21.1: Precisão aproximada por largura de mantissa

#NBMANT	Dígitos decimais	Uso típico
10	cerca de 3	Sinais de instrumentação com dinâmica modesta
16	cerca de 5	Padrão de fábrica, folgado para a maioria dos algoritmos
23	cerca de 7	Equivalente à precisão simples do IEEE 754

O expoente, por sua vez, define a faixa: com #NBEXPO 5 em complemento de dois, o expoente varia de -16 a 15, o que cobre cerca de dez ordens de grandeza. Valores fora da faixa do formato configurado são apontados como erro de faixa na compilação.

Um dado experimental do laboratório

Na verificação de uma Unidade de Medição Fasorial implementada no SAPHO, a saída do processador com `float` de 24 bits reproduziu um modelo de referência em dupla precisão com desvio máximo da ordem de 5×10^{-6} no fasor. O erro remanescente na conformidade com a norma foi atribuído ao método de estimação, e não à aritmética do *hardware*.

A conclusão prática é que o formato reduzido raramente é o fator limitante: antes de ampliar a mantissa, confira se o gargalo não está no algoritmo.

21.4 Na prática, dentro do programa

Dentro do programa `C±`, o tipo `float` simplesmente funciona. Literais, aritmética e as funções da biblioteca operam nesse formato interno, e a conversão entre o formato IEEE do seu computador e o formato do SAPHO é feita em *software* pela cadeia de ferramentas, na carga das memórias e na leitura dos resultados.

Listing 21.1: Ponto flutuante em uso, sem cerimônia

```
#NUBITS 16
#NBMANT 10
#NBEXPO 5

void main()
{
    float x;
    float y;
    while (1)
    {
        x = fin(0);           // le convertendo para float
        y = sqrt(x) * 2.5;
        fout(0, y);          // escreve em formato float
    }
}
```

21.5 Números complexos

O tipo `comp` não tem *hardware* dedicado. Cada complexo é um par de `float`, parte real e imaginária, manipulado componente a componente pelo código gerado. As quatro operações aritméticas funcionam e expandem para o conjunto de operações reais necessárias.

Isso tem duas consequências práticas. A primeira é de custo: uma multiplicação de complexos gera quatro multiplicações e duas somas reais. A segunda é de precisão: cada componente tem a precisão do `float` configurado, e não mais do que isso.

 Dica

Nas formas de onda, as variáveis do tipo `comp` são decodificadas para a forma legível $a + bi$ pelo conversor `comp2gtkw` do YANC, de modo que você não precisa ler dois sinais separados e recompor mentalmente o número.

21.6 Quando o float não vale a pena

Uma regra de bolso do laboratório: se a dinâmica do sinal cabe em inteiros, use inteiros.

Ponto flutuante instancia blocos de *hardware* substanciais na unidade lógica e aritmética, um por operação usada, e consome mais ciclos que a aritmética inteira. Em processamento de sinais de instrumentação, é muito comum que uma escala fixa bem escolhida com `int` produza o mesmo resultado por uma fração do custo.

O caminho intermediário é a função `norm()`, que divide por uma potência de dois definida em `#NUGAIN` e permite trabalhar com inteiros escalados sem instanciar o divisor. Veja [Entrada, saída e biblioteca padrão](#).

21.7 Leitura relacionada

- [O processador SAPHO por dentro](#) descreve onde o bloco de ponto flutuante se encaixa no caminho de dados.
- [O conjunto de instruções](#) lista os *opcodes* de ponto flutuante, que começam por `F_`.
- [Diretivas: configurando o processador no código](#) explica como escolher `#NBMANT` e `#NBEXPO`.

O conjunto de instruções

Você nunca vai escrever *assembly* do SAPHO à mão: o compilador o gera. Mas você vai lê-lo, e com frequência, porque a AURORA exhibe nas formas de onda o mnemônico executado a cada ciclo de *clock*, em sincronia com a linha do seu código C±.

Esta página existe para que essa trilha faça sentido.

22.1 Como o conjunto é organizado

O conjunto usa *opcode* de 7 bits, com famílias de carga e armazenamento, pilha, entrada e saída, controle de fluxo e cálculo. Nem todas as instruções existem em um processador dado: o montador liga apenas as *opcodes* que o programa emprega, e a *hardware* das demais é eliminado na geração do circuito.

Ao final de cada compilação, o terminal TASM resume o percentual do conjunto de instruções e da unidade lógica e aritmética efetivamente usados pelo programa.

22.2 Convenções de prefixo e sufixo

Antes da tabela, decore estas cinco marcas. Elas explicam a maioria dos mnemônicos que você vai ver.

Marca	Significado
F_	Versão em ponto flutuante da operação
S_ e SF_	O operando vem da pilha de dados
P_	Há empilhamento prévio
_M	A operação atua sobre memória
_V	Variante indexada, para acesso a vetores

Assim, F_ADD é uma soma em ponto flutuante, S_ADD é uma soma cujo segundo operando vem da pilha, e LOD_V é uma carga indexada.

22.3 As famílias

Tabela22.1: Famílias de instruções e mnemônicos principais

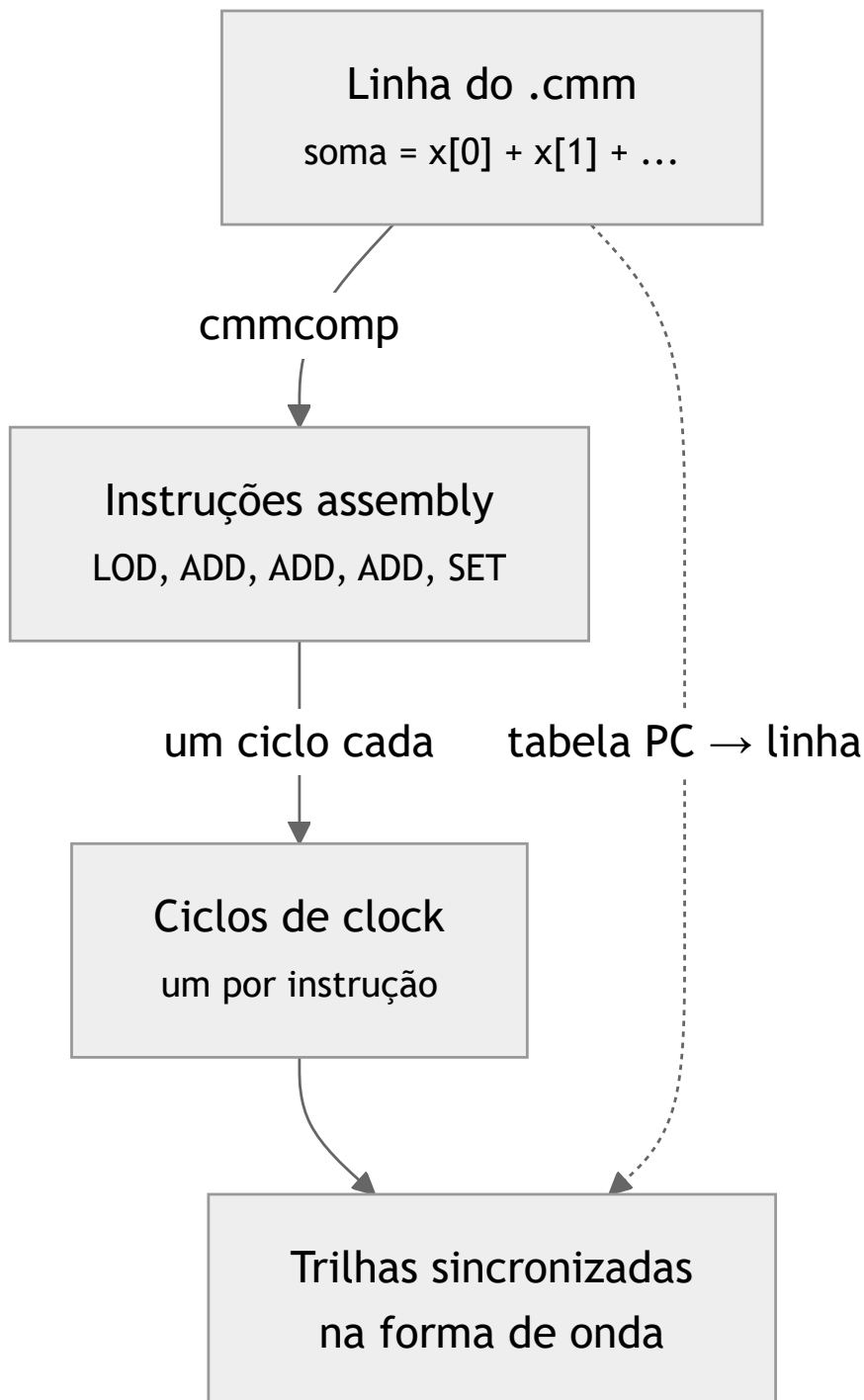
Família	Mnemônicos principais
Carga e armazenamento	LOD, SET, LDI, STI, ILI, ISI
Pilha	PSH, POP
Entrada e saída	INN, F_INN, OUT
Controle de fluxo	JMP, JIZ, CAL, RET, NOP
Aritmética inteira	ADD, MLT, DIV, MOD, NEG, ABS, SGN, PST, NRM
Aritmética em ponto flutuante	F_ADD, F_SU1, F_SU2, F_MLT, F_DIV, F_NEG, F_ABS, F_ROT, F_SCL, XPO
Conversões	I2F, F2I
Lógica e bits	AND, ORR, XOR, INV, LAN, LOR, LIN
Comparações	LES, GRE, EQU, e as versões com prefixo F_
Deslocamentos	SHL, SHR, SRS, este último correspondendo ao operador >>>

Cada mnemônico usado pela primeira vez liga o bloco de *hardware* correspondente no processador gerado.

É por isso que o custo em área do circuito é proporcional ao subconjunto da linguagem que o programa emprega, como detalhado em [Recursos avançados](#).

22.4 Lendo a trilha na forma de onda

A AURORA gera, a cada compilação, duas tabelas de tradução: uma que liga cada endereço do contador de programa ao mnemônico *assembly*, e outra que liga esse endereço à linha do fonte C±. Elas viram trilhas de texto no visualizador de ondas.



Na prática, ao percorrer a onda com o cursor você lê três coisas ao mesmo tempo: o valor de cada variável, a instrução que está executando e a linha do seu programa que a gerou. É o vínculo mais direto entre código e

circuito que a plataforma oferece, e a razão pela qual o fluxo $C\pm$ preserve o endereçamento fixo de variáveis.

Dica

Se uma variável tem um valor inesperado, posicione o cursor no ciclo anterior à mudança e leia o mne-mônico. Na maioria dos casos a instrução responsável está ali, e a linha $C\pm$ correspondente aparece na trilha logo abaixo.

22.5 Uma leitura comentada

Considere a linha do exemplo condutor:

```
soma = x[0] + x[1] + x[2] + x[3];
```

O compilador a traduz para uma sequência que carrega o primeiro elemento no acumulador e soma os demais um a um, terminando com o armazenamento em `soma`. Nas ondas, você verá algo próximo de `LOD, ADD, ADD, ADD, SET`, um por ciclo, com o acumulador crescendo a cada passo e a variável `soma` mudando apenas no último.

Esse é o comportamento típico de uma máquina de acumulador, e explica por que expressões longas consomem mais ciclos que o mesmo cálculo repartido: cada operando extra é um acesso a mais à memória de dados.

22.6 Leitura relacionada

- *O processador SAPHO por dentro* descreve o caminho de dados no qual essas instruções executam.
- *O ponto flutuante do SAPHO* explica o formato manipulado pelos *opcodes* com prefixo `F_`.
- *Formas de onda: GTKWave e Surfer* mostra como as trilhas são preparadas e exibidas em cada visualizador.

Simular e analisar

Simular com Icarus, Verilator ou cocotb

Uma simulação executa o circuito em um ambiente de teste e compara seu comportamento com o resultado esperado. Nesta página, você aprenderá a preparar o projeto, escolher o simulador e reconhecer quando o teste realmente foi concluído.

23.1 Preparar a simulação

Antes de clicar em **Analisar Verilog (forma de onda)** ou **Execução rápida**:

1. Selecione as fontes Verilog do projeto.
2. Defina o **Top Level**.
3. Defina o **Testbench Top**.
4. Salve os arquivos.
5. Selecione Icarus ou Verilator na barra superior quando o fluxo depender dessa opção.

O Top Level identifica o circuito; o Testbench Top identifica o teste. Antes de continuar, confira os dois nomes na barra de status e verifique se o testbench referencia sinais que existem na versão atual do RTL.

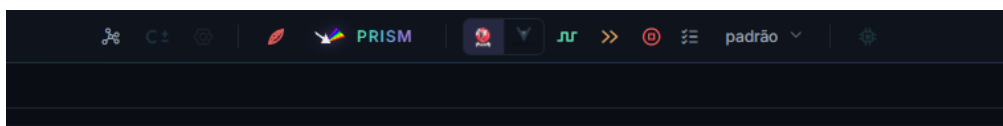


Figura23.1: Os logotipos selecionam o simulador usado pela análise de forma de onda e por testbenches cocotb. O ícone de onda executa **Analisar Verilog (forma de onda)**, os dois chevrons executam **Execução rápida** e a lista abre a configuração dos sinais.

23.2 Escolher o simulador

Opção	Indicação	Característica principal	Atenção
Icarus Verilog	Primeira execução, testes curtos e investigação de sinais	Simula diretamente o HDL e oferece boa compatibilidade com testbenches Verilog tradicionais	Pode ser mais lento em testes extensos
Verilator	Simulações longas ou projetos maiores	Compila o modelo para obter maior desempenho	Nem toda construção de testbench aceita pelo Icarus é compatível
cocotb	Testes automatizados escritos em Python	Oferece corrotinas, asserções e acesso às portas do circuito	Não é um terceiro simulador: usa Icarus ou Verilator como backend

Comece com Icarus para validar a estrutura básica e observar sinais. Troque para Verilator quando a duração ou o tamanho do projeto exigir mais desempenho. Escolha cocotb quando o teste se beneficiar de Python, automação e asserções legíveis, selecionando antes o backend desejado na barra superior.

A **Execução rápida** com testbench Verilog exige Verilator. Com testbench `.py`, a AURORA executa cocotb em modo headless usando Icarus ou Verilator, conforme a seleção. Como os backends não aceitam necessariamente as mesmas construções, avalie uma falha pelo terminal da opção selecionada.

23.3 Análise de forma de onda ou execução rápida

Analisar Verilog (forma de onda)

Executa o testbench, gera formas de onda e abre o viewer selecionado.

Execução rápida

Executa sem abrir viewer de ondas. Com testbench Verilog, usa Verilator em modo headless. Com testbench .py, executa cocotb em modo headless com o backend selecionado.

Use **Analisar Verilog (forma de onda)** quando precisar observar a evolução dos sinais. Use **Execução rápida** para testes automatizados, mensagens no terminal ou verificações que já possuem `assert` e não dependem de inspeção visual.

23.4 Testbench Verilog

Selecione um arquivo .v como Testbench Top. O testbench deve:

- Instanciar o módulo testado.
- Gerar clock e reset quando necessários.
- Aplicar estímulos.
- Encerrar a execução.
- Produzir dump de sinais ou permitir que a AURORA o configure.

O teste deve possuir uma condição clara de encerramento. Um testbench que gera clock indefinidamente e nunca chama sua condição de término continuará executando até ser cancelado.

O exemplo abaixo testa uma porta AND. O módulo do circuito deve ser adicionado como fonte sintetizável e definido como **Top Level**.

```

1 module porta_and (
2     input wire a,
3     input wire b,
4     output wire y
5 );
6     assign y = a & b;
7 endmodule

```

O arquivo seguinte deve ser adicionado como testbench e definido como **Testbench Top**.

```

1 `timescale 1ns/1ps
2
3 module tb_porta_and;
4     reg a;
5     reg b;
6     wire y;
7
8     porta_and dut (
9         .a(a),
10        .b(b),
11        .y(y)
12    );
13
14    task verificar;
15        input valor_a;
16        input valor_b;
17        input esperado;
18        begin

```

(continua na próxima página)

(continuação da página anterior)

```

19     a = valor_a;
20     b = valor_b;
21     #1;
22
23     if (y !== esperado) begin
24         $display(
25             "ERRO: a=%0b b=%0b esperado=%0b obtido=%0b",
26             a, b, esperado, y
27         );
28         $fatal(1);
29     end
30 end
31 endtask
32
33 initial begin
34     $dumpfile("tb_porta_and.vcd");
35     $dumpvars(0, tb_porta_and);
36
37     verificar(1'b0, 1'b0, 1'b0);
38     verificar(1'b0, 1'b1, 1'b0);
39     verificar(1'b1, 1'b0, 1'b0);
40     verificar(1'b1, 1'b1, 1'b1);
41
42     $display("SUCESSO: tabela verdade validada.");
43     $finish;
44 end
45 endmodule

```

O testbench instancia o circuito com o nome `dut`, aplica os quatro casos da tabela-verdade e compara cada saída. Quando um valor está incorreto, `$fatal(1)` interrompe a simulação como falha. Quando todos os casos passam, `$finish` encerra normalmente. As chamadas `$dumpfile` e `$dumpvars` criam o registro usado na visualização das formas de onda.

23.5 Testbench cocotb

Selecione um arquivo `.py` como Testbench Top. A AURORA usa o ambiente Python incluído na toolchain; você não precisa configurar o Python global do Windows.



Figura 23.2: No projeto `porta_AND`, o testbench `test_porta_and.py` aparece na mesma árvore da fonte `porta_and.v` e é identificado pelo ícone Python.

O mesmo circuito pode ser verificado com o testbench `cocotb` abaixo:

```

1 # aurora-toplevel: porta_and
2
3 import cocotb
4 from cocotb.triggers import Timer
5
6
7 async def verificar(dut, valor_a, valor_b, esperado):
8     dut.a.value = valor_a
9     dut.b.value = valor_b
10    await Timer(1, unit="ns")
11
12    obtido = int(dut.y.value)
13    assert obtido == esperado, (
14        f"a={valor_a} b={valor_b}: esperado={esperado}, obtido={obtido}"
15    )
16
17
18 @cocotb.test()
19 async def testa_tabela_verdade(dut):
20     casos = [
21         (0, 0, 0),
22         (0, 1, 0),
23         (1, 0, 0),
24         (1, 1, 1),
25     ]
26
27     for valor_a, valor_b, esperado in casos:
28         await verificar(dut, valor_a, valor_b, esperado)

```

O exemplo apresenta as partes essenciais de um teste cocotb:

- A diretiva `# aurora-toplevel: porta_and` informa explicitamente qual módulo Verilog será elaborado como DUT.
- O decorador `@cocotb.test()` registra a corrotina como um teste executável.
- O parâmetro `dut` representa o módulo Verilog e expõe suas portas pelos nomes `a`, `b` e `y`.
- `Timer(1, unit="ns")` permite que a lógica combinacional atualize a saída antes da leitura.
- A asserção compara resultado esperado e resultado obtido e inclui o caso completo na mensagem de erro.

Mantenha o nome do arquivo Python compatível com um módulo Python, por exemplo, `test_porta_and.py`. Evite espaços, hífen e caracteres especiais. Se um sinal não existir com o nome usado no código, o teste falhará antes de concluir as verificações.

Para circuitos com clock, use `Clk`, aguarde `RisingEdge` e leia os sinais após `ReadOnly`. A galeria apresenta esse padrão em um contador e também mostra como testar o Verilog gerado por algoritmos C±.

Consulte [Galeria de projetos e testbenches](#) para baixar todos os arquivos e comparar testbenches `.v` e `.py` equivalentes.

23.6 Confirmar o resultado

A simulação está correta quando:

- O terminal termina sem erro.
- O testbench alcança o final esperado.
- As verificações do teste passam.

- **Analisar Verilog (forma de onda)** abre o arquivo de onda correto.

Uma execução sem mensagens de erro não é suficiente quando o testbench deveria verificar valores. Confirme que as asserções foram alcançadas, que o tempo simulado avançou e que a mensagem final corresponde ao teste executado.

23.7 Problemas comuns

Módulo principal não encontrado

Revise Top Level e arquivos selecionados.

Testbench não inicia

Confirme Testbench Top, clock, reset e nome do módulo.

O cocotb não encontra o teste

Confirme o arquivo .py, o nome da função decorada e os sinais usados.

Verilator falha, mas Icarus funciona

O projeto pode usar uma construção não aceita pelo Verilator. Leia a primeira mensagem de erro no terminal.

Nenhuma forma de onda foi criada

Siga *Formas de onda*: [GTKWave](#) e [Surfer](#).

Galeria de projetos e testbenches

Esta galeria reúne exemplos pequenos para praticar o fluxo completo de simulação na AURORA. Cada circuito ou algoritmo possui duas alternativas de teste: um testbench Verilog (.v) e um testbench cocotb (.py). Use apenas uma alternativa como **Testbench Top** em cada execução.

24.1 Como usar os exemplos

Para um exemplo Verilog:

1. Baixe ou copie os três arquivos do conjunto para a pasta do projeto.
2. Adicione o arquivo do circuito como fonte sintetizável.
3. Defina o arquivo do circuito como **Top Level**.
4. Adicione tb_*.v ou test_*.py como testbench.
5. Defina o testbench escolhido como **Testbench Top**.
6. Execute **Analisar Verilog (forma de onda)** para abrir as formas de onda ou **Execução rápida** para conferir apenas as verificações.

Para um exemplo C±:

1. Crie no **Hub de Processadores** um processador com o mesmo nome indicado por #PRNAME.
2. Substitua o conteúdo do arquivo em Software pelo exemplo .cmm.
3. Execute **C±** para gerar o módulo em Hardware/<nome>.v e os arquivos de memória necessários.
4. Adicione o testbench .v ou .py correspondente ao projeto.
5. Defina Hardware/<nome>.v como **Top Level** e o teste escolhido como **Testbench Top**.
6. Execute **Analisar Verilog (forma de onda)** ou **Execução rápida**.

ⓘ Importante

Os testbenches dos exemplos C± exercitam o Verilog gerado pelo compilador. Compile o .cmm antes da simulação e mantenha os arquivos gerados pela AURORA no projeto. O arquivo Python não executa C± diretamente.

24.2 Porta AND em Verilog

Este primeiro conjunto cobre um circuito combinacional. Os quatro pares possíveis de entrada são aplicados, e cada saída é comparada com a tabela-verdade da operação AND.

Arquivos: porta_and.v, tb_porta_and.v e test_porta_and.py.

24.2.1 Circuito porta_and.v

```
1 module porta_and (
2     input wire a,
3     input wire b,
```

(continua na próxima página)

(continuação da página anterior)

```

4     output wire y
5 );
6     assign y = a & b;
7 endmodule

```

24.2.2 Testbench Verilog tb_porta_and.v

```

1 `timescale 1ns/1ps
2
3 module tb_porta_and;
4     reg a;
5     reg b;
6     wire y;
7
8     porta_and dut (
9         .a(a),
10        .b(b),
11        .y(y)
12    );
13
14    task verificar;
15        input valor_a;
16        input valor_b;
17        input esperado;
18        begin
19            a = valor_a;
20            b = valor_b;
21            #1;
22
23            if (y !== esperado) begin
24                $display(
25                    "ERRO: a=%0b b=%0b esperado=%0b obtido=%0b",
26                    a, b, esperado, y
27                );
28                $fatal(1);
29            end
30        end
31    endtask
32
33    initial begin
34        $dumpfile("tb_porta_and.vcd");
35        $dumpvars(0, tb_porta_and);
36
37        verificar(1'b0, 1'b0, 1'b0);
38        verificar(1'b0, 1'b1, 1'b0);
39        verificar(1'b1, 1'b0, 1'b0);
40        verificar(1'b1, 1'b1, 1'b1);
41
42        $display("SUCESSO: tabela verdade validada.");
43        $finish;
44    end
45 endmodule

```

O bloco task verificar evita repetir a sequência de atribuição, espera e comparação. \$fatal(1) encerra

a execução com falha assim que um caso produz uma saída incorreta. \$dumpfile e \$dumpvars registram os sinais para o viewer de ondas.

24.2.3 Testbench cocotb test_porta_and.py

```

1 # aurora-toplevel: porta_and
2
3 import cocotb
4 from cocotb.triggers import Timer
5
6
7 async def verificar(dut, valor_a, valor_b, esperado):
8     dut.a.value = valor_a
9     dut.b.value = valor_b
10    await Timer(1, unit="ns")
11
12    obtido = int(dut.y.value)
13    assert obtido == esperado, (
14        f"a={valor_a} b={valor_b}: esperado={esperado}, obtido={obtido}"
15    )
16
17
18 @cocotb.test()
19 async def testa_tabela_verdade(dut):
20     casos = [
21         (0, 0, 0),
22         (0, 1, 0),
23         (1, 0, 0),
24         (1, 1, 1),
25     ]
26
27     for valor_a, valor_b, esperado in casos:
28         await verificar(dut, valor_a, valor_b, esperado)

```

A diretiva # aurora-toplevel: porta_and seleciona explicitamente o módulo testado. A função auxiliar aplica um caso, aguarda a propagação combinacional e inclui entradas, valor esperado e valor obtido na mensagem da asserção.

24.3 Contador de 4 bits em Verilog

Este conjunto introduz clock, reset e controle de habilitação. O teste confirma que o reset zera o estado, que o contador avança a cada borda quando enable vale 1 e que o valor permanece estável quando enable vale 0.

Arquivos: contador4.v, tb_contador4.v e test_contador4.py.

24.3.1 Circuito contador4.v

```

1 module contador4 (
2     input wire    clk,
3     input wire    rst,
4     input wire    enable,
5     output reg    [3:0] valor
6 );
7     always @(posedge clk) begin

```

(continua na próxima página)

(continuação da página anterior)

```

8         if (rst)
9             valor <= 4'd0;
10        else if (enable)
11            valor <= valor + 4'd1;
12    end
13 endmodule

```

24.3.2 Testbench Verilog tb_contador4.v

```

1  `timescale 1ns/1ps
2
3  module tb_contador4;
4      reg clk = 1'b0;
5      reg rst = 1'b1;
6      reg enable = 1'b0;
7      wire [3:0] valor;
8
9      contador4 dut (
10         .clk(clk),
11         .rst(rst),
12         .enable(enable),
13         .valor(valor)
14     );
15
16     always #5 clk = ~clk;
17
18     initial begin
19         $dumpfile("tb_contador4.vcd");
20         $dumpvars(0, tb_contador4);
21
22         repeat (2) @(posedge clk);
23         @(negedge clk);
24         rst = 1'b0;
25         enable = 1'b1;
26
27         repeat (4) @(posedge clk);
28         #1;
29         if (valor !== 4'd4) begin
30             $display("ERRO: esperado=4 obtido=%0d", valor);
31             $fatal(1);
32         end
33
34         @(negedge clk);
35         enable = 1'b0;
36         repeat (2) @(posedge clk);
37         #1;
38         if (valor !== 4'd4) begin
39             $display("ERRO: contador mudou com enable=0. obtido=%0d", valor);
40             $fatal(1);
41         end
42
43         $display("SUCESSO: reset, contagem e enable validados.");
44         $finish;

```

(continua na próxima página)

(continuação da página anterior)

```

45     end
46 endmodule

```

O clock alterna a cada 5 ns, portanto o período é de 10 ns. O reset é retirado na borda de descida para estar estável antes da próxima borda de subida. O atraso #1 após a borda permite observar o valor atualizado pela atribuição não bloqueante do contador.

24.3.3 Testbench cocotb test_contador4.py

```

1  # aurora-toplevel: contador4
2
3  import cocotb
4  from cocotb.clock import Clock
5  from cocotb.triggers import FallingEdge, ReadOnly, RisingEdge
6
7
8  async def ler_apos_borda(dut):
9      await RisingEdge(dut.clk)
10     await ReadOnly()
11     return int(dut.valor.value)
12
13
14  @cocotb.test()
15  async def testa_reset_contagem_e_enable(dut):
16      cocotb.start_soon(Clock(dut.clk, 10, unit="ns").start())
17
18      dut.rst.value = 1
19      dut.enable.value = 0
20      await ler_apos_borda(dut)
21      assert await ler_apos_borda(dut) == 0, "0 reset nao zerou o contador."
22
23      await FallingEdge(dut.clk)
24      dut.rst.value = 0
25      dut.enable.value = 1
26      for esperado in range(1, 5):
27          obtido = await ler_apos_borda(dut)
28          assert obtido == esperado, (
29              f"Contagem incorreta: esperado={esperado}, obtido={obtido}"
30          )
31
32      await FallingEdge(dut.clk)
33      dut.enable.value = 0
34      for _ in range(2):
35          obtido = await ler_apos_borda(dut)
36          assert obtido == 4, (
37              f"0 contador mudou com enable=0: esperado=4, obtido={obtido}"
38          )

```

Clock gera o clock em uma corrotina separada. A função `ler_apos_borda` aguarda `RisingEdge` e depois `ReadOnly`, fase em que as atualizações daquele instante já podem ser lidas sem disputar com a lógica do simulador. As mudanças de `rst` e `enable` são feitas após `FallingEdge`, fora da fase somente de leitura e antes da próxima borda ativa.

24.4 Soma de constantes em C±

Este exemplo gera um processador chamado `soma_constantes`. O algoritmo soma $3 + 4$, escreve 7 na porta de saída 0 e alcança `#TOAQUI`, que permite ao hardware indicar o término pelo sinal `cheguei`.

Arquivos: `soma_constantes.cmm`, `tb_soma_constantes.v` e `test_soma_constantes.py`.

24.4.1 Algoritmo `soma_constantes.cmm`

```

1  #PRNAME soma_constantes
2  #NUBITS 32
3  #NBMANT 23
4  #NBEXPO 8
5  #NDSTAC 8
6  #SDEPTH 8
7  #NUIOIN 1
8  #NUIOOU 1
9  #NUGAIN 128
10
11 void main()
12 {
13     int a = 3;
14     int b = 4;
15     int resultado;
16
17     resultado = a + b;
18     out(0, resultado);
19
20     #TOAQUI
21 }
```

24.4.2 Testbench Verilog `tb_soma_constantes.v`

```

1  `timescale 1ns/1ps
2
3  module tb_soma_constantes;
4      reg clk = 1'b0;
5      reg rst = 1'b1;
6      wire signed [31:0] out;
7      wire [0:0] out_en;
8      wire chegou;
9
10     integer ciclos = 0;
11     integer recebeu_saida = 0;
12     integer terminou = 0;
13
14     soma_constantes dut (
15         .clk(clk),
16         .rst(rst),
17         .out(out),
18         .out_en(out_en),
19         .cheguei(chegou)
20     );
21
22     always #5 clk = ~clk;
```

(continua na próxima página)

(continuação da página anterior)

```

23
24 always @(posedge clk) begin
25     if (!rst && (out_en == 1)) begin
26         recebeu_saida = 1;
27         if ($signed(out) !== 32'sd7) begin
28             $display("ERRO: esperado=7 obtido=%0d", $signed(out));
29             $fatal(1);
30         end
31     end
32
33     if (cheguei === 1'b1)
34         terminou = 1;
35 end
36
37 initial begin
38     $dumpfile("tb_soma_constantes.vcd");
39     $dumpvars(0, tb_soma_constantes);
40
41     repeat (2) @(posedge clk);
42     @(negedge clk);
43     rst = 1'b0;
44
45     while (!terminou && (ciclos < 200)) begin
46         @(posedge clk);
47         ciclos = ciclos + 1;
48     end
49
50     repeat (2) @(posedge clk);
51     #1;
52
53     if (!recebeu_saida) begin
54         $display("ERRO: nenhuma escrita foi observada na porta 0.");
55         $fatal(1);
56     end
57
58     if (!terminou) begin
59         $display("ERRO: tempo limite antes de #TOAQUI.");
60         $fatal(1);
61     end
62
63     $display("SUCESSO: saida 7 e termino do algoritmo validados.");
64     $finish;
65 end
66 endmodule

```

O bloco acionado na borda de subida observa `out_en == 1`, condição que identifica uma escrita na porta 0. A captura ocorre na própria borda porque esse sinal funciona como um pulso de habilitação da escrita. O limite de 200 ciclos evita uma simulação infinita se o algoritmo não produzir a saída ou não alcançar #TOAQUI.

24.4.3 Testbench cocotb test_soma_constantes.py

```

1 # aurora-toplevel: soma_constantes
2
3 import cocotb

```

(continua na próxima página)

(continuação da página anterior)

```

4 from cocotb.clock import Clock
5 from cocotb.triggers import FallingEdge, ReadOnly, RisingEdge
6
7
8 @cocotb.test()
9 async def testa_soma_e_termino(dut):
10     cocotb.start_soon(Clock(dut.clk, 10, unit="ns").start())
11
12     dut.rst.value = 1
13     for _ in range(2):
14         await RisingEdge(dut.clk)
15         await FallingEdge(dut.clk)
16     dut.rst.value = 0
17
18     saidas = []
19     ciclos_apos_termino = 0
20     for _ in range(200):
21         await FallingEdge(dut.clk)
22         await ReadOnly()
23
24         if int(dut.out_en.value) == 1:
25             saidas.append(int(dut.out.value))
26
27         if int(dut.cheguei.value) == 1 and ciclos_apos_termino == 0:
28             ciclos_apos_termino = 1
29         elif ciclos_apos_termino:
30             ciclos_apos_termino += 1
31
32         if ciclos_apos_termino >= 3:
33             break
34     else:
35         raise AssertionError("Tempo limite antes de o algoritmo atingir #TOAQUI.")
36
37     assert saidas == [7], f"Esperado [7] na porta 0, obtido {saidas}."

```

O teste amostra `out_en`, `out` e `cheguei` na borda de descida do clock, quando os sinais produzidos na borda ativa já estão estáveis. Ele exige a lista exata [7]; dessa forma, detecta ausência de saída, valor incorreto e escritas adicionais inesperadas. O limite de ciclos impede uma espera infinita.

24.5 Sequência de quadrados em C±

Este exemplo usa um laço `while` para escrever 0, 1, 4 e 9 na porta 0. Além do valor de cada amostra, os testes verificam a ordem e a quantidade de escritas.

Arquivos: `sequencia_quadrados.cmm`, `tb_sequencia_quadrados.v` e `test_sequencia_quadrados.py`.

24.5.1 Algoritmo `sequencia_quadrados.cmm`

```

1 #PRNAME sequencia_quadrados
2 #NIBITS 32
3 #NBMAINT 23
4 #NBEXPO 8
5 #NDSTAC 8
6 #SDEPTH 8

```

(continua na próxima página)

(continuação da página anterior)

```

7  #NUIOIN 1
8  #NUIOOU 1
9  #NUGAIN 128
10
11 void main()
12 {
13     int indice = 0;
14
15     while (indice < 4) {
16         out(0, indice * indice);
17         indice = indice + 1;
18     }
19
20     #TOAQUI
21 }

```

24.5.2 Testbench Verilog tb_sequencia_quadrados.v

```

1  `timescale 1ns/1ps
2
3  module tb_sequencia_quadrados;
4      reg clk = 1'b0;
5      reg rst = 1'b1;
6      wire signed [31:0] out;
7      wire [0:0] out_en;
8      wire cheguei;
9
10     integer ciclos = 0;
11     integer indice = 0;
12     integer terminou = 0;
13     integer esperado [0:3];
14
15     sequencia_quadrados dut (
16         .clk(clk),
17         .rst(rst),
18         .out(out),
19         .out_en(out_en),
20         .cheguei(cheguei)
21     );
22
23     always #5 clk = ~clk;
24
25     always @(posedge clk) begin
26         if (!rst && (out_en == 1)) begin
27             if (indice > 3) begin
28                 $display("ERRO: o processador produziu saidas extras.");
29                 $fatal(1);
30             end
31
32             if ($signed(out) != esperado[indice]) begin
33                 $display(
34                     "ERRO: indice=%0d esperado=%0d obtido=%0d",
35                     indice, esperado[indice], $signed(out)
36                 );
37             end
38         end
39     end

```

(continua na próxima página)

(continuação da página anterior)

```

36         );
37         $fatal(1);
38     end
39
40     indice = indice + 1;
41 end
42
43 if (cheguei === 1'b1)
44     terminou = 1;
45 end
46
47 initial begin
48     esperado[0] = 0;
49     esperado[1] = 1;
50     esperado[2] = 4;
51     esperado[3] = 9;
52
53     $dumpfile("tb_sequencia_quadrados.vcd");
54     $dumpvars(0, tb_sequencia_quadrados);
55
56     repeat (2) @(posedge clk);
57     @(negedge clk);
58     rst = 1'b0;
59
60     while (!terminou && (ciclos < 400)) begin
61         @(posedge clk);
62         ciclos = ciclos + 1;
63     end
64
65     repeat (2) @(posedge clk);
66     #1;
67
68     if (indice !== 4) begin
69         $display("ERRO: esperadas=4 saidas_recebidas=%0d", indice);
70         $fatal(1);
71     end
72
73     if (!terminou) begin
74         $display("ERRO: tempo limite antes de #TOAQUI.");
75         $fatal(1);
76     end
77
78     $display("SUCESSO: sequencia 0, 1, 4, 9 validada.");
79     $finish;
80 end
81 endmodule

```

O vetor `esperado` funciona como um modelo de referência simples. Cada pulso de `out_en` consome uma posição. O teste falha se houver valor incorreto, quantidade diferente de quatro saídas ou ausência do sinal de término.

24.5.3 Testbench cocotb test_sequencia_quadrados.py

```

1 # aurora-toplevel: sequencia_quadrados
2
3 import cocotb
4 from cocotb.clock import Clock
5 from cocotb.triggers import FallingEdge, ReadOnly, RisingEdge
6
7
8 @cocotb.test()
9 async def testa_sequencia_e_termino(dut):
10     cocotb.start_soon(Clock(dut.clk, 10, unit="ns").start())
11
12     dut.rst.value = 1
13     for _ in range(2):
14         await RisingEdge(dut.clk)
15     await FallingEdge(dut.clk)
16     dut.rst.value = 0
17
18     saidas = []
19     ciclos_apos_termino = 0
20     for _ in range(400):
21         await FallingEdge(dut.clk)
22         await ReadOnly()
23
24         if int(dut.out_en.value) == 1:
25             saidas.append(int(dut.out.value))
26
27         if int(dut.cheguei.value) == 1 and ciclos_apos_termino == 0:
28             ciclos_apos_termino = 1
29         elif ciclos_apos_termino:
30             ciclos_apos_termino += 1
31
32         if ciclos_apos_termino >= 3:
33             break
34     else:
35         raise AssertionError("Tempo limite antes de o algoritmo atingir #TOAQUI.")
36
37     assert saidas == [0, 1, 4, 9], (
38         f"Esperado [0, 1, 4, 9] na porta 0, obtido {saidas}."
39 )

```

A lista Python torna a comparação da sequência inteira direta. Esse padrão pode ser ampliado para filtros, máquinas de estados e processadores que produzam séries de amostras.

24.6 Escolher entre os dois tipos de testbench

Situação	Testbench recomendado
Aprender eventos e tarefas do Verilog	.v
Controlar diretamente o dump de sinais	.v
Percorrer muitos casos de entrada	.py com cocotb
Criar listas, modelos matemáticos e mensagens detalhadas	.py com cocotb
Depurar a integração inicial de um módulo	Comece com .v e reproduza em .py quando necessário

Os dois formatos verificam o mesmo hardware. A escolha altera a linguagem e a organização do teste, não

o circuito sintetizável. Para o procedimento de execução e o diagnóstico de falhas, consulte *Simular com Icarus, Verilator ou cocotb*.

Formas de onda: GTKWave e Surfer

Depois que a simulação gera o arquivo de ondas, a AURORA abre um visualizador para inspecionar cada sinal do circuito ciclo a ciclo. Esta página explica o que a IDE prepara para você, como escolher os sinais gravados e como usar cada um dos dois visualizadores.

25.1 O que a AURORA prepara

Abrir um despejo bruto em um visualizador qualquer significa começar do zero: nenhum sinal selecionado, tudo em binário, nomes hierárquicos crus. A AURORA elimina esse trabalho gerando, a cada simulação, um *layout* curado.

Organização automática Cada processador do projeto vira uma seção própria, com divisores nomeados e, no Surfer, um grupo colapsável.

A ordem é pensada para leitura: *clock*, *reset* e interrupção primeiro; depois as portas de entrada e saída em amarelo, as instruções em violeta, as variáveis em laranja e as *flags* ao final.

Decodificação de tipos Variáveis do tipo *comp* são decodificadas para a forma legível $a + bi$ pelo conversor *comp2gtkw* do YANC, em vez de aparecerem como dois sinais binários separados.

O destaque, porém, são as trilhas de texto que mostram, a cada ciclo de *clock*, o mnemônico *assembly* executado e a linha $C \pm$ correspondente, avançando em sincronia com o circuito. É o seu programa rodando dentro do *hardware*, visível linha a linha.

Essas trilhas vêm dos arquivos de tradução gerados na compilação. No GTKWave, a curadoria chega como um arquivo *.gtkw*; no Surfer, como um arquivo de estado *.surf.ron*. Nos dois casos, o visualizador já abre pronto para leitura.

📌 Importante

Essa sincronia entre onda, *assembly* e código-fonte só é possível porque no fluxo $C \pm$ cada variável ocupa um endereço fixo da memória de dados. É a contrapartida direta da ausência de recursão, explicada em [O processador SAPHO por dentro](#).

25.2 Gerar uma forma de onda

1. Defina o *testbench* pelo menu de contexto da árvore.
2. Escolha o simulador na barra superior: Icarus para depurar, Verilator para simulações longas.
3. Escolha o visualizador, GTKWave ou Surfer.
4. Abra *Configuração de ondas* e selecione os sinais desejados.
5. Clique em *Analisar Verilog (forma de onda)*.

A AURORA executa a simulação e abre o resultado. Acompanhe o progresso no terminal **TWAVE**; se a compilação ou o *testbench* falhar, o visualizador pode não abrir, e é esse terminal que explica por quê.

Aviso

Com o Verilator selecionado, apenas os sinais do topo do *testbench* são expostos. Os sinais internos do processador, incluindo as variáveis pelo nome, exigem o Icarus. Se os sinais internos sumiram, é quase sempre esse o motivo.

25.3 Escolher os sinais gravados

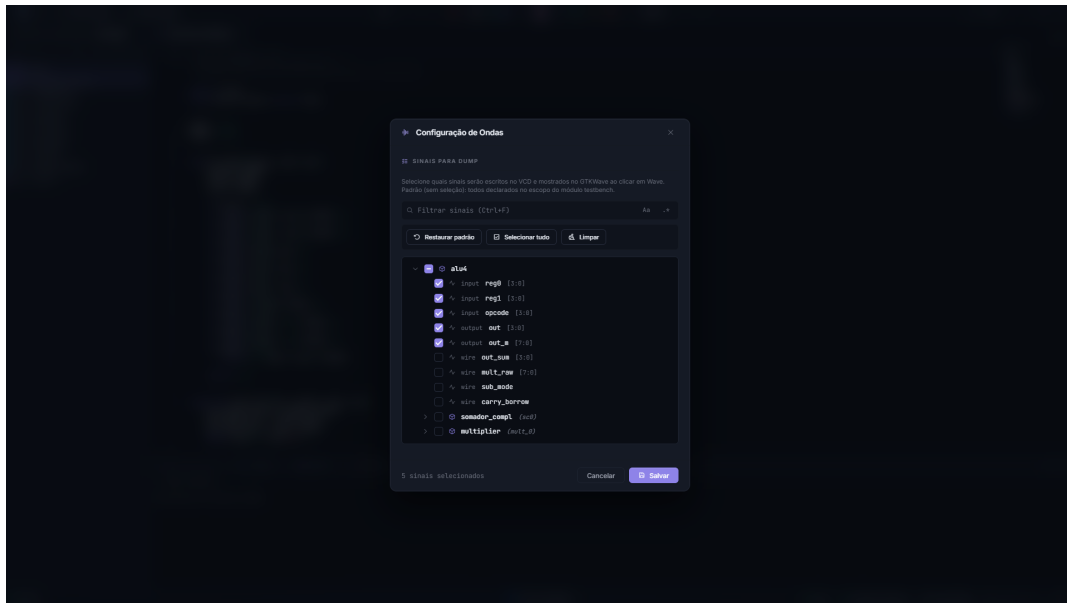


Figura25.1: O modal lista todos os sinais descobertos no projeto, com busca por texto ou expressão regular, seleção em massa e um contador no rodapé.

Por padrão, o despejo grava os sinais do escopo do *testbench*. O botão *Configuração de ondas* abre o modal que permite pesquisar por nome, navegar pela hierarquia, selecionar tudo ou nada, restaurar o padrão e filtrar apenas os sinais do processador.

Atalho	Ação
Ctrl+F	Focar a pesquisa
Esc	Limpar a pesquisa; pressione de novo para fechar

Selecione primeiro *clock*, *reset*, entradas, saídas e os poucos sinais internos necessários para responder à pergunta do teste. Menos sinais significam arquivos menores e simulação mais leve, o que pesa muito em execuções longas.

25.3.1 A ordem de precedência do despejo

Na hora de gravar, a AURORA decide a lista de sinais por uma ordem definida. Conhecê-la evita surpresas:

1. um *.gtkw* ativo no seletor da barra superior, cujos sinais referenciados mandam;
2. a sua seleção no modal de configuração;
3. um *\$dumpvars* escrito à mão no *testbench*, que é respeitado sem qualquer injeção;
4. o padrão, que é todo o escopo do *testbench*.

i Nota

O seu *testbench* nunca é modificado. A instrumentação acontece em uma cópia temporária, de modo que um `$dumpfile` ou `$dumpvars` manual permanece intacto no arquivo original.

25.4 O seletor de layout

Ao lado do botão de ondas, o seletor escolhe o *layout* ativo. A opção padrão usa o *layout* curado gerado pela AURORA, e + *Adicionar arquivo .gtkw...* registra um *layout* seu, salvo de dentro do GTKWave.

O seletor acompanha o visualizador ativo: em GTKWave trabalha com `.gtkw`; em Surfer, com `.surf.ron` e `.suc1`.

O arquivo de ondas em si continua sendo o `.vcd` ou o `.fst`. Os *layouts* apenas organizam quais sinais mostrar, em que ordem e com que cores.

⚠ Aviso

Use apenas *layouts* criados para o mesmo *testbench* e para uma hierarquia compatível. Um *layout* não contém a simulação, e um *layout* de outro projeto pode apontar para sinais inexistentes mesmo com o arquivo de ondas gerado corretamente.

25.5 GTKWave

O GTKWave empacotado não é o original puro: é o *fork* do laboratório, com tema escuro por padrão, ajuste automático de *zoom* ao abrir, rótulos justificados à esquerda, ícones modernizados e *zoom* animado.

No uso cotidiano:

- os sinais são adicionados a partir da árvore de escopos à esquerda;
- o cursor primário se posiciona com o clique e o secundário com o botão do meio, com a barra exibindo a diferença de tempo entre eles;
- o formato de exibição de cada sinal muda pelo menu de contexto, em *Data Format*;
- o *zoom* responde aos botões de lupa e a `Ctrl` com a roda do mouse;
- a seleção atual de sinais, cores e ordem pode ser salva em um `.gtkw` próprio por *File* □ *Write Save File*;
- após uma nova simulação, *File* □ *Reload Waveform* recarrega o arquivo.

💡 Dica

Se você salvar o seu próprio `.gtkw` no projeto e o marcar como ativo no seletor, ele passa a ter precedência sobre o *layout* automático, e a simulação seguinte grava exatamente os sinais que ele referencia.

25.6 Surfer

O Surfer é um visualizador moderno escrito em Rust, com interface acelerada por GPU e carregamento preguiçoso dos arquivos, confortável em simulações grandes. A AURORA usa o *fork* `surfer-aurora`, com decodificadores específicos do SAPHO, e baixa o binário automaticamente com verificação de integridade.

Recursos que valem conhecer:

- formatos de exibição ricos, de binário e hexadecimal a ponto fixo e ponto flutuante IEEE em vários tamanhos;

- grupos colapsáveis, um por processador no *layout* da AURORA, essenciais em projetos multiprocessados;
- as trilhas de *assembly* e C± instaladas automaticamente como tradutores de mapeamento;
- cursores e navegação rápidos, com uma linha de comando interna de autocompletar difuso;
- desenho analógico de sinais, útil para ver amostras processadas como curvas.

Os complexos do SAPHO também funcionam: a AURORA pré-decodifica os sinais *comp* e instala o resultado como tradutor, de modo que a faixa já abre na forma $a + bi$.

i Nota

Uma limitação conhecida no Windows O recarregamento automático do arquivo de ondas não dispara na versão empacotada. Por isso a AURORA fecha e reabre a janela do Surfer a cada nova simulação. No modal de configuração há uma opção de manter as janelas abertas, para comparar execuções lado a lado.

Se o Surfer estiver selecionado mas o executável não estiver presente, a AURORA recua para o GTKWave sem erro, de propósito.

25.7 Se o visualizador abrir sem sinais

Verifique nesta ordem:

1. o *testbench* executou até o fim;
2. o terminal TWAVE não informou erro;
3. existe despejo de sinais;
4. os sinais selecionados ainda existem no RTL atual;
5. o *layout* ativo pertence ao *testbench* atual.

Volte ao *layout* padrão e gere de novo para descartar um *layout* incompatível.

Se os sinais aparecerem mas permanecerem sem mudança, verifique o *clock*, o *reset*, os estímulos e o intervalo de tempo mostrado. O problema pode estar no *testbench* ou apenas no *zoom* da janela, e não na geração do arquivo.

25.8 Se o arquivo de ondas não for encontrado

- confira o nome usado em *\$dumpfile*, se o *testbench* for seu;
- verifique se o *testbench* encerrou antes de produzir o arquivo;
- remova arquivos de onda antigos que possam causar ambiguidade;
- execute novamente e leia o terminal TWAVE.

25.9 Leitura relacionada

- *Simular com Icarus, Verilator ou cocotb* cobre os motores e os tipos de *testbench*.
- *O conjunto de instruções* ajuda a ler a trilha de *assembly*.
- *Criar e configurar processadores* mostra onde aumentar o número de ciclos quando a onda termina cedo.

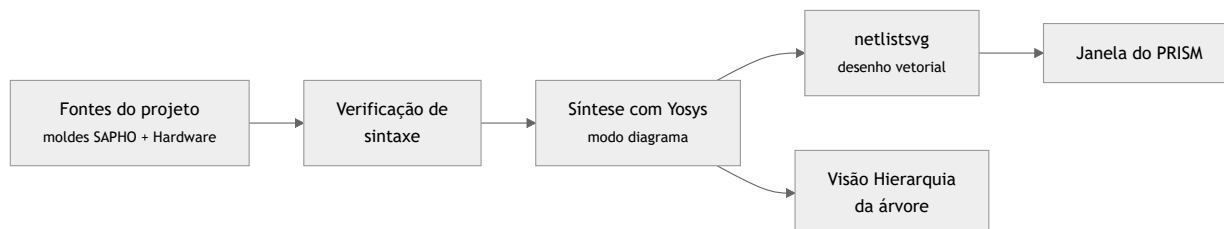
Visualizar e simular o RTL no PRISM

O PRISM (*Processor Rendering Interface for Schematic Models*) desenha o projeto como um diagrama de circuito navegável, com módulos, portas e conexões, do *top-level* até o interior do processador. É a resposta visual à pergunta sobre que *hardware* o seu código virou.

Ele tem dois modos complementares. O modo *Esquemático* apresenta a estrutura RTL sintetizada; o modo *Simular* abre um circuito lógico interativo no qual entradas podem ser alteradas entre 0 e 1, com as saídas atualizadas na tela.

26.1 O que acontece quando você clica

Vale conhecer a cadeia, porque ela explica tanto o tempo de espera quanto as exigências.



A síntese roda em modo diagrama, sem mapeamento tecnológico, justamente para que o desenho preserve a estrutura RTL do código em vez de mostrar portas lógicas genéricas de uma biblioteca de células. Cada módulo é materializado como desenho vetorial antes de a janela abrir, o que torna a navegação instantânea depois.

A saída do Yosys corre no terminal **TVERI**, e a mesma síntese alimenta a visão *Hierarquia* da árvore de arquivos.

Dica

Símbolos próprios Os blocos do processador SAPHO não são retângulos genéricos: a unidade lógica e aritmética, as memórias, as pilhas e o contador de programa têm símbolos desenhados sob medida, os mesmos reproduzidos em [O processador SAPHO por dentro](#). Reconhecê-los acelera muito a leitura de um projeto grande.

26.2 Abrir o PRISM

1. Adicione todas as fontes sintetizáveis ao projeto.
2. Defina o *top-level* pelo menu de contexto da árvore.
3. Salve os arquivos.
4. Execute *Sintetizar Verilog* e corrija os erros mostrados no terminal.
5. Clique em *Abrir PRISM*.

O diagrama corresponde ao RTL salvo e processado naquele momento. Se o *top-level* ou o conjunto de fontes mudar, confirme as seleções na janela principal e use *Recompile*.

26.3 Projeto Verilog puro

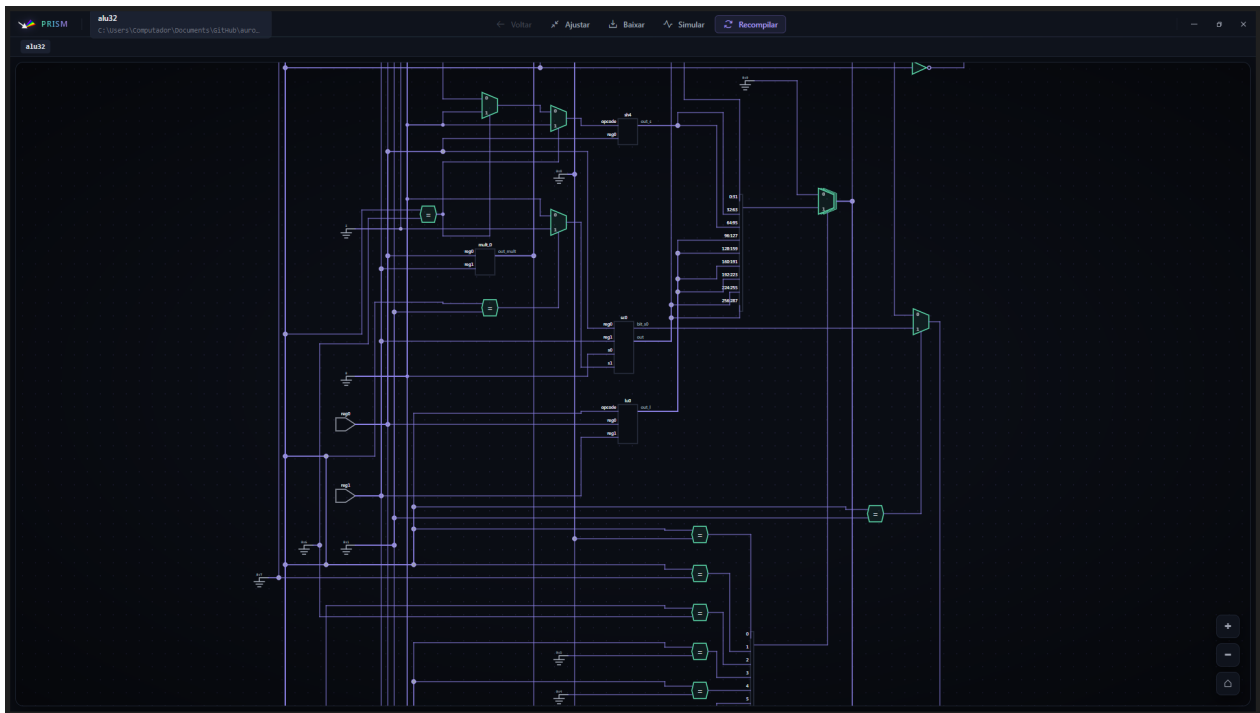


Figura26.1: O projeto ALU32 mostra operações, multiplexadores, portas e conexões derivados diretamente do módulo `alu32.v`.

No modo **Esquemático**, use o zoom para aproximar ou afastar, arraste a área livre para navegar e abra instâncias navegáveis para inspecionar módulos internos. **Voltar** retorna ao nível anterior e **Ajustar** enquadra o diagrama na janela.

26.4 Processador SAPHO

A aparência própria do bloco `processor` facilita reconhecer o núcleo SAPHO dentro de um projeto maior. O ícone representa uma instância do módulo; abra a instância quando quiser navegar pela estrutura interna disponível.

26.5 Navegar no diagrama

A janela do PRISM é independente da principal e abre maximizada, no mesmo tema escuro. O mouse faz o deslocamento e o *zoom*.

Um clique em um módulo entra nele, e como os diagramas dos submódulos já foram materializados, a navegação é instantânea; a trilha no topo mostra o caminho na hierarquia e permite voltar.

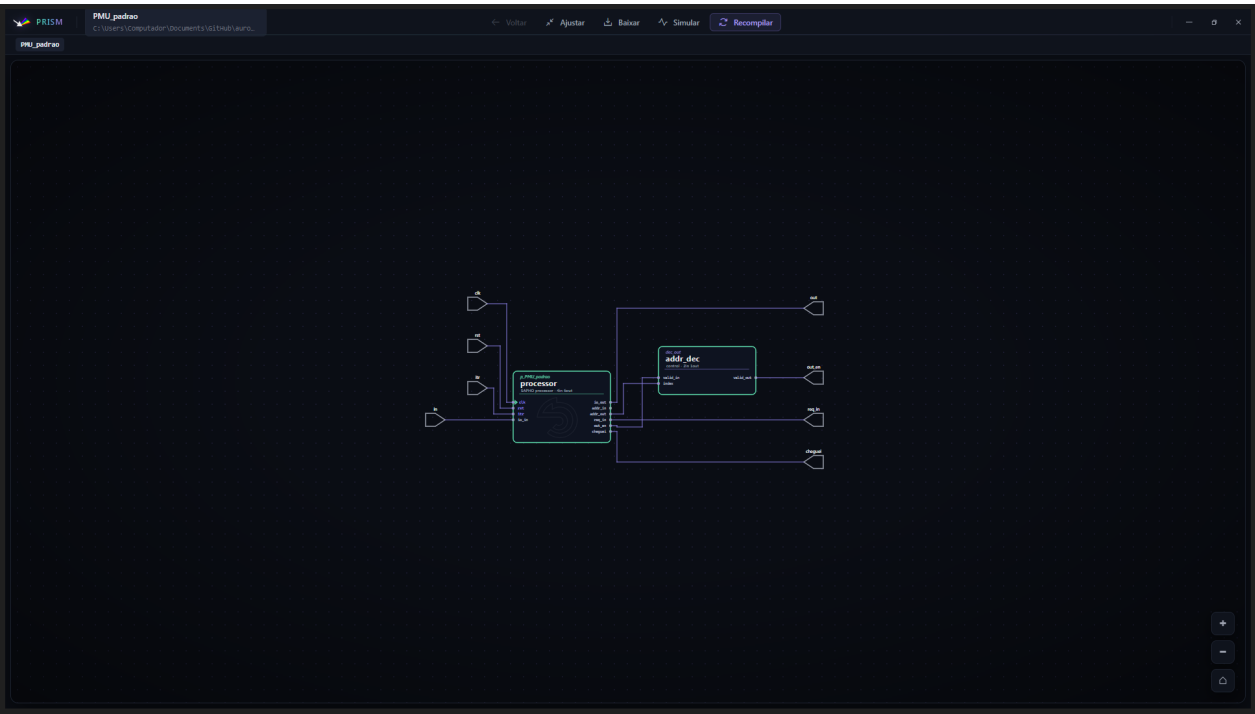


Figura26.2: No projeto proj_PMU_padrao, o processador SAPHO aparece com um ícone próprio no RTL, ao lado dos demais blocos e conexões do Top Level.

 **Dica**

Um duplo clique em uma célula abre o código-fonte correspondente no editor principal, na linha exata da declaração. É o caminho mais curto do desenho ao código.

Tabela26.1: Controles da janela

Controle	O que faz
Back	Retorna ao módulo visualizado anteriormente
Fit View	Centraliza e enquadra todo o circuito
Baixar	Exporta a visualização atual
Simular	Troca do esquemático para a simulação lógica interativa
Esquemático	Retorna da simulação interativa ao diagrama RTL
Recompile	Processa novamente os arquivos salvos e atualiza a visualização

26.6 Simulação interativa

Para testar um circuito combinacional:

1. Abra o esquemático do Top Level no PRISM.
2. Clique em **Simular** e aguarde a construção do circuito interativo.
3. Clique nos controles de entrada para alternar cada valor entre 0 e 1.
4. Observe as conexões e os valores das saídas atualizados na própria tela.
5. Clique em **Esquemático** para retornar à estrutura RTL.

Em circuitos sequenciais, a simulação pode disponibilizar controles adicionais de *clock*. Valores desconhecidos podem aparecer como x até que entradas, *clock* ou *reset* definam um estado válido.

O modo interativo é apoiado no DigitalJS e limitado a três mil células. Acima disso o PRISM recusa e sugere

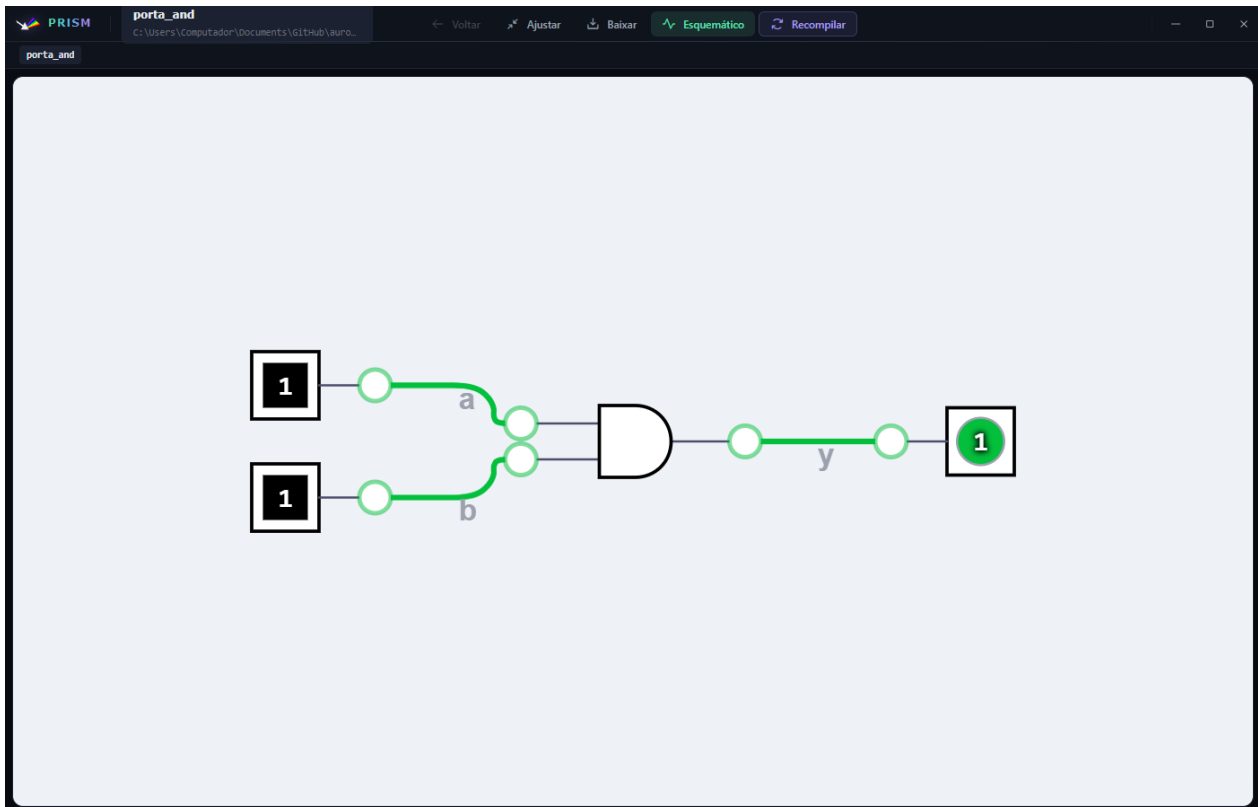


Figura26.3: Na simulação do projeto `porta_AND`, as entradas `a` e `b` estão em 1; a saída `y` responde com 1 conforme a tabela-verdade da porta AND.

a visão esquemática comum: é uma lupa didática para circuitos pequenos, não um simulador de propósito geral.

26.7 O que o PRISM confirma

Use o modo *Esquemático* para conferir hierarquia, instâncias e conexões. Use *Simular* para experimentar estados lógicos diretamente, sobretudo em circuitos pequenos e combinacionais.

A simulação interativa não substitui um *testbench* Verilog ou cocotb, nem a análise temporal em formas de onda. Para verificar sequências extensas, temporização, *clock* e asserções automatizadas, use *Simular com Icarus*, *Verilator ou cocotb* e *Formas de onda: GTKWave e Surfer*.

Dica

Use o PRISM cedo e com frequência. Compilar um processador e abrir o diagrama depois de cada mudança torna palpável a relação entre as construções da linguagem e o custo em *hardware*. Acrescente uma divisão ao programa e veja o divisor aparecer no desenho; remova-a e ele some.

É a forma mais rápida de internalizar o princípio de pagar apenas pelo que se usa, detalhado em *Recursos avançados*.

26.8 Quando o PRISM falhar

Top Level incorreto

Escolha o módulo raiz do projeto.

Módulo ausente

Inclua o arquivo que declara a dependência.

Módulo duplicado

Remova uma das fontes que declaram o mesmo nome.

Diagrama não é gerado

Execute **Compile Verilog** primeiro e corrija o erro mais inicial do terminal.

Uma instância não abre

Ela pode ser uma primitiva, um módulo filtrado ou não possuir informação suficiente para navegação.

A simulação mostra x

Defina as entradas e aplique reset ou clock quando o circuito exigir estado inicial.

Aurora Intelligence

Aurora Intelligence

A Aurora Intelligence é a assistente de inteligência artificial integrada à AURORA. Ela conversa sobre o projeto, a linguagem C $\pm$, o Verilog, a arquitetura SAPHO e as mensagens do compilador e, com a sua permissão, age sobre a IDE: edita arquivos, compila, dispara simulações, seleciona sinais de onda e opera o Git.

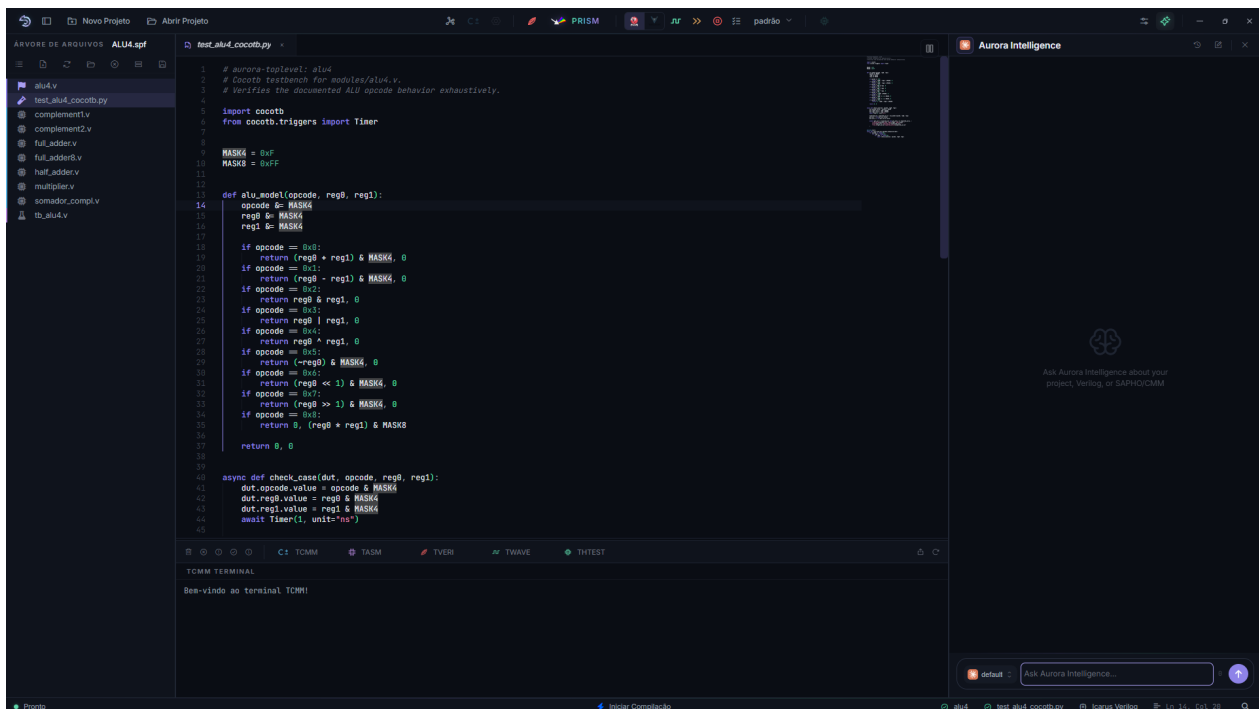


Figura27.1: O painel permanece ao lado do editor, de modo que você formula o pedido enquanto o arquivo e o estado do projeto continuam visíveis.

27.1 O que a distingue de uma assistente genérica

A diferença não é o modelo: é o conhecimento de domínio. A assistente é especializada na linguagem C $\pm$ e no fluxo do SAPHO por dois mecanismos complementares.

Conhecimento derivado do compilador A identidade e as regras da linguagem vêm de um *system prompt* extenso montado a partir da própria gramática do YANC: os tipos, as diretivas obrigatórias, as restrições de projeto, a notação de Dirac e o conjunto de *opcodes* do *assembly*.

Como esse conhecimento é derivado do compilador, e não apenas da memória do modelo, o risco de alucinação sobre a sintaxe e as regras do SAPHO cai substancialmente. Quando o conjunto de instruções cresce, o *prompt* acompanha.

Ferramentas de domínio Além do texto, a assistente dispõe de ferramentas específicas: a listagem de diretivas, a análise de arquivos *assembly* e a consulta ao catálogo bilingue de mensagens do compilador, cujo conteúdo é gerado automaticamente a partir das fontes do YANC.

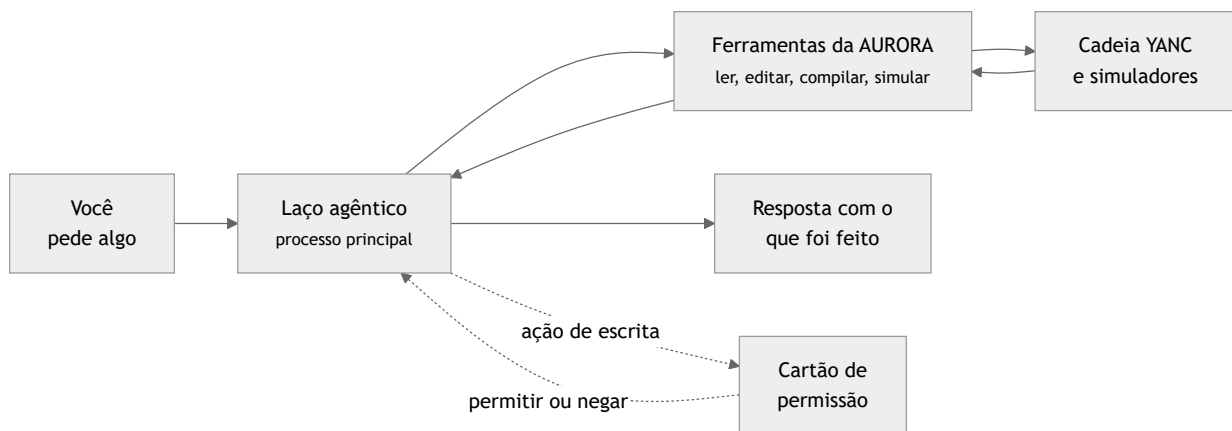
Perguntar por que um `.cmm` não compila é, portanto, uma pergunta que ela responde consultando a fonte, e não adivinhando.

i Nota

A assistente funciona no modelo de trazer a própria chave (BYOK, *Bring Your Own Key*): você conecta a sua conta de um provedor. Modelos próprios do NIPS-CERN estão no roteiro do laboratório, mas ainda não foram treinados; até lá, a assistente opera sobre provedores externos. Veja [Configurar o provedor de IA](#).

27.2 Como ela funciona por dentro

O comportamento é agêntico: em vez de devolver uma única resposta em texto, ela atua em um laço no qual invoca ferramentas, observa os resultados e decide o passo seguinte, até cumprir a tarefa. Esse laço executa no processo principal da IDE e é limitado a um número máximo de passos por turno, o que evita recursões descontroladas.



Falhas nunca interrompem o laço: são devolvidas como resultado de erro, permitindo que o modelo se recupere e tente outro caminho. Toda chamada de ferramenta e o seu desfecho ficam registrados em um *log* de auditoria.

27.3 Abrir e usar o painel

O botão *Assistente IA*, a estrela da barra superior, abre o painel à direita, empurrando o editor. A largura é ajustável e lembrada entre sessões. O cabeçalho traz o ícone do provedor ativo e os botões de histórico de conversas, nova conversa e fechamento.

Na caixa de mensagem ficam os seletores de provedor e de modelo, o controle de esforço de raciocínio para os modelos que o suportam, o indicador de uso da conversa, o seletor de permissões, o botão de anexar arquivos ou imagens e o de interromper a geração. Enter envia.

💡 Dica

O caminho mais curto Selecione um trecho de código no editor e clique na estrela que aparece junto à seleção. O trecho vai para a assistente já contextualizado, com caminho e linguagem do arquivo.

27.4 Para que ela é boa no fluxo SAPHO

Alguns usos que rendem particularmente bem:

- perguntar por que um C++ não compila, já que ela lê o terminal, conhece as mensagens do YANC e propõe a correção;

- estimar quanto *hardware* custa uma função, consultando a referência de *opcodes* e o *assembly* gerado;
- pedir um *testbench* em cocotb que compare o processador com um modelo de referência em NumPy;
- pedir que selecione os sinais das portas e abra as formas de onda;
- explicar um trecho de Verilog gerado que você não reconhece.

i Um exemplo real do que ela consegue fazer sozinha

Em um estudo de caso publicado pelo laboratório, o pedido foi deliberadamente aberto: olhar o *assembly* gerado de um processador de FFT e reduzir o número de instruções sem alterar o comportamento, recompilando e simulando para confirmar.

A assistente identificou uma redundância semântica no bloco de saída, aplicou a transformação, recompilou e verificou a equivalência sobre as formas de onda, reduzindo o código de 325 para 269 instruções, 17,2% a menos, sem alterar a saída da simulação. Todo o percurso foi conduzido por ela, decidindo a cada passo qual ferramenta invocar.

27.5 Como pedir bem

A assistente trabalha melhor quando o pedido informa o objetivo, o arquivo e a etapa atual. Em vez de pedir apenas “corrija o projeto”, indique o erro observado e peça que a primeira causa seja explicada antes de qualquer modificação.

Explique o arquivo ativo e diga qual é o módulo principal.

Leia o terminal Verilog e identifique o primeiro erro que preciso corrigir.

Liste os sinais do testbench e sugira apenas clock, reset, entradas e saídas.

Antes de editar, mostre exatamente quais linhas pretende alterar.

27.6 Um fluxo de trabalho seguro

1. peça uma análise;
2. solicite um plano curto;
3. aprove uma alteração pequena;
4. revise o resultado;
5. compile ou simule;
6. só então continue.

Esse ciclo evita acumular várias mudanças antes de descobrir qual delas introduziu o problema. Para tarefas maiores, peça que a assistente divida o trabalho em etapas independentes.

Uma confirmação autoriza a ação mostrada naquele momento; ela não substitui a revisão do resultado. Depois de qualquer escrita, abra o arquivo alterado, confira o conteúdo e execute a validação adequada.

🔔 Importante

Versione o projeto em Git antes de trabalhar com a assistente. Cada passo dela vira um *diff* revisável no painel Dagr, o que torna qualquer alteração reversível. Veja [Source Control e Git](#).

27.7 Conversas e disponibilidade

Cada conversa é salva localmente, e o histórico reabre conversas antigas com o contexto completo. Sem conexão com o provedor, o painel indica que a assistente está *offline*.

27.8 Privacidade

O conteúdo enviado depende do provedor configurado:

- provedores em nuvem recebem as mensagens e o contexto necessário;
- o Ollama processa o modelo localmente, na sua máquina;
- Claude Code e Codex usam as suas próprias contas e ferramentas de linha de comando.

As chaves de API são cifradas em repouso pelo cofre do sistema operacional e nunca atravessam a fronteira de processos, não havendo canal para a sua leitura. A interface exibe apenas se existe uma chave configurada, jamais o valor.

⚠️ Aviso

Não envie projetos confidenciais sem verificar as regras do provedor. Remova segredos dos arquivos antes de usá-los como contexto, e ao relatar um erro de autenticação descreva apenas a mensagem recebida e o nome do provedor.

27.9 Leitura relacionada

- [Configurar o provedor de IA](#) cobre a configuração de cada provedor e das assinaturas.
- [Tarefas com a Aurora Intelligence](#) lista o que ela pode ler e o que exige confirmação.
- [Usar Claude Code e ChatGPT via Codex CLI](#) explica o servidor local que expõe as ferramentas da AURORA a agentes externos.

Configurar o provedor de IA

O provedor determina qual serviço processará as mensagens da Aurora Intelligence. Esta página explica os requisitos de cada opção e como confirmar a conexão antes de utilizar ferramentas no projeto.

Abra **Configurações da AURORA** → **Assistente IA**.

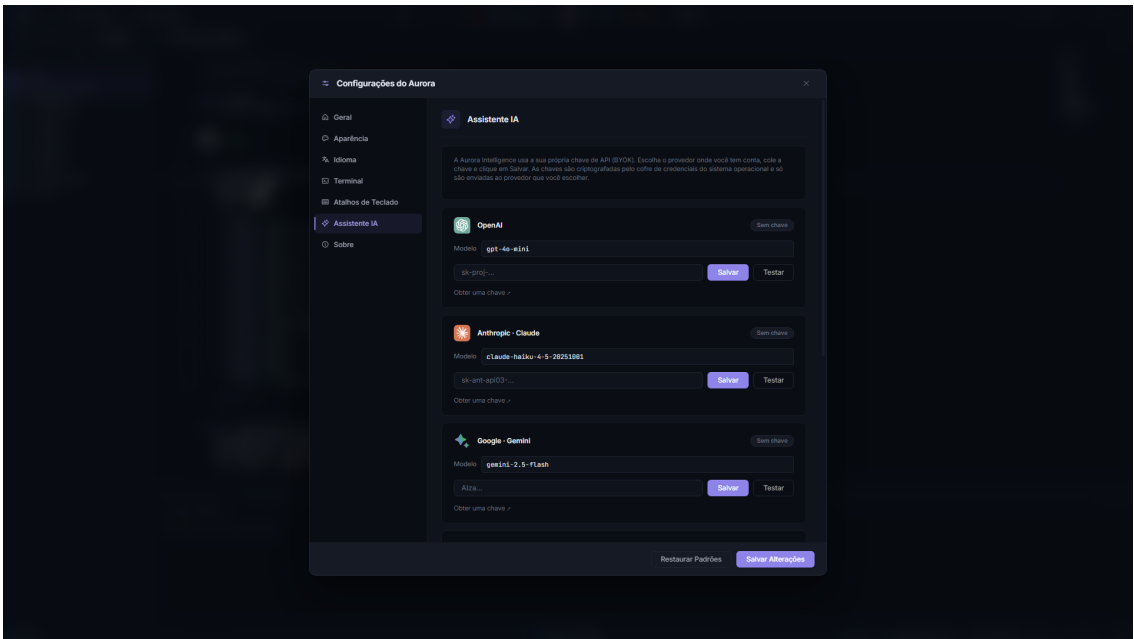


Figura28.1: Cada cartão reúne o modelo, a credencial e os botões para salvar e testar o provedor. Nunca publique uma captura com uma chave preenchida.

28.1 Escolher uma opção

Tipo	Opções	Requisito
API em nuvem	OpenAI, Anthropic, Google, DeepSeek e Groq	Chave de API e modelo válido.
Local	Ollama	Ollama iniciado e modelo instalado.
Assinatura por CLI	Claude Code e ChatGPT via Codex CLI	Login concluído no CLI correspondente.

28.2 Provedor por API

1. Abra o cartão do provedor.
2. Cole a chave de API.
3. Escolha ou informe o modelo.
4. Clique em **Save**.
5. Clique em **Test connection**.

Depois de salvar, o campo da chave pode ficar vazio. Isso não significa que a chave foi perdida.

O teste deve confirmar que a credencial, o endereço do serviço e o modelo são aceitos. Se a conexão funcionar, inicie uma conversa curta e peça apenas uma explicação antes de autorizar ações no projeto.

Aviso

As chamadas podem gerar cobrança na conta do provedor. Consulte limites e preços no serviço escolhido.

28.3 Ollama

Instale e inicie o Ollama fora da AURORA. Exemplo:

```
ollama pull llama3.1:8b
ollama serve
```

na AURORA:

1. Informe `http://localhost:11434/v1` como URL.
2. Clique em **Detect models**.
3. Selecione o modelo.
4. Teste a conexão.

Se o modelo responde, mas não consegue executar ações, escolha um modelo com melhor suporte a ferramentas. Modelos locais variam em memória necessária, velocidade e capacidade de seguir estruturas de chamadas; selecione uma opção compatível com o computador utilizado.

28.4 Claude Code

A AURORA usa o Claude Code CLI para este provedor. O executável pode ser encontrado no cache sob demanda em `userData\cli-cache`, na dependência local da aplicação ou no `PATH` do Windows. A versão pinada no manifesto atual é 2.1.196.

Para autenticar:

```
claude login
claude --version
```

Depois, selecione Claude Code na AURORA e inicie uma conversa. Se o CLI ainda não estiver cacheado, a AURORA pode baixar o pacote previsto pelo manifesto. O login continua sendo responsabilidade do CLI e usa as credenciais locais em `~\.claude\credentials.json`.

28.5 ChatGPT via Codex CLI

A opção **ChatGPT** usa o Codex CLI local. O executável pode ser encontrado no cache sob demanda em `userData\cli-cache`, na dependência local da aplicação ou no `PATH` do Windows. A versão pinada no manifesto atual é 0.131.0.

Para autenticar:

```
codex login
codex --version
```

Depois, selecione ChatGPT na AURORA. Se um modelo específico falhar, use a opção **Default**.

O Codex CLI usa `CODEX_HOME\auth.json` quando `CODEX_HOME` está definido, ou `~\.codex\auth.json` no caso padrão. A AURORA não garante que essa conta seja a mesma conta aberta no navegador.

28.6 Se o teste falhar

- Confirme a chave, conta e modelo.
- Verifique conexão, proxy e firewall.
- Para Ollama, confirme que `ollama serve` está ativo.
- Para Claude Code ou Codex, refaça o login no CLI correspondente.
- Se o CLI foi instalado ou autenticado fora da AURORA, reinicie a aplicação para atualizar o ambiente.
- Se o cache sob demanda falhar, verifique conexão com o registro npm e permissões de escrita no perfil do usuário.

Leia a mensagem apresentada pelo teste antes de trocar várias configurações. Erros de autenticação, modelo inexistente, serviço indisponível e bloqueio de rede exigem correções diferentes.

Tarefas com a Aurora Intelligence

Em vez de memorizar nomes de ferramentas internas, descreva o resultado esperado. Inclua o arquivo, a etapa ou o erro relevante e informe se o assistente pode apenas analisar ou também propor alterações.

Os exemplos desta página são modelos de pedido. Substitua os nomes genéricos pelo contexto real e mantenha uma tarefa principal por mensagem.

29.1 Entender o projeto

Comece por consultas de leitura. Elas ajudam a confirmar se o assistente reconheceu corretamente o projeto antes de executar qualquer ação.

Resuma a estrutura do projeto e identifique Top Level, Testbench Top e processador ativo.

Explique o arquivo ativo para alguém que está aprendendo Verilog.

29.2 Corrigir compilação

Peça que o terminal seja analisado na ordem em que as mensagens ocorreram. Corrigir a primeira falha evita alterações desnecessárias causadas por erros secundários.

Leia os terminais e explique somente o primeiro erro que impede a compilação.

Confira se há módulos duplicados ou dependências ausentes antes de alterar arquivos.

29.3 Editar com segurança

Defina limites claros: arquivo, trecho, comportamento esperado e validação que deverá ser executada depois da mudança.

Mostre a alteração proposta e aguarde minha confirmação antes de editar.

Faça apenas a menor mudança necessária e salve o arquivo.

29.4 Trabalhar com processadores

Liste os processadores e explique a configuração do processador ativo.

Crie um processador chamado filtro com 16 bits, uma entrada e uma saída.

29.5 Simular e analisar ondas

Informe se deseja apenas preparar a seleção, executar a simulação ou interpretar o resultado. Essas ações possuem efeitos e tempos de execução diferentes.

Confira Top Level e Testbench Top e diga o que falta para simular.

Selecione clock, reset, entradas e saídas na ****Configuração de Ondas****.

Execute a simulação e explique o resultado do terminal sem modificar o RTL.

29.6 Organizar arquivos

Liste referências ausentes e não remova nenhuma sem minha aprovação.

Crie um backup antes de renomear o processador.

29.7 Boas práticas

- Informe uma tarefa por vez.
- Peça análise antes de alterações amplas.
- Leia a confirmação de ações de escrita.
- Valide arquivos editados com compilação ou simulação.
- Mantenha backup ou controle de versão.

Depois de uma ação, peça um resumo objetivo do que foi alterado e do resultado da validação. Não considere uma resposta textual como prova de que a compilação ou a simulação foi concluída.

Usar Claude Code e ChatGPT via Codex CLI

Claude Code e ChatGPT via Codex CLI podem ser usados como provedores de assinatura dentro da Aurora Intelligence. A autenticação é feita no CLI de cada serviço. A AURORA inicia o provedor selecionado e disponibiliza ferramentas controladas para que ele consulte ou opere o projeto.

30.1 Como a AURORA encontra os CLIs

A AURORA procura o executável nesta ordem:

1. Cache sob demanda em `userData\cli-cache`.
2. Dependência local instalada com a aplicação.
3. Executável disponível no PATH do Windows.

O manifesto atual fixa Claude Code em 2.1.196 e Codex em 0.131.0. Quando o CLI não está no cache, a AURORA pode baixar o pacote previsto pelo manifesto e reutilizá-lo nas próximas execuções.

30.2 Preparar o Claude Code

1. Execute `claude login` no terminal.
2. Confirme com `claude --version`.
3. Reinicie a AURORA se o login ou instalação tiver ocorrido enquanto ela estava aberta.
4. Selecione Claude Code em **Configurações da AURORA → Assistente IA**.

O login fica em `~\.claude\credentials.json`. Se `claude --version` não for reconhecido no terminal, a AURORA ainda pode usar o cache sob demanda ou a dependência local; use o terminal principalmente para concluir login e diagnosticar credenciais.

30.3 Preparar o ChatGPT via Codex CLI

1. Execute `codex login` no terminal.
2. Confirme com `codex --version`.
3. Reinicie a AURORA se o login ou instalação tiver ocorrido enquanto ela estava aberta.
4. Selecione ChatGPT em **Configurações da AURORA → Assistente IA**.

O Codex usa `CODEX_HOME\auth.json` quando `CODEX_HOME` está definido, ou `~\.codex\auth.json` no padrão. A AURORA não comprova que essa conta seja a mesma sessão do ChatGPT no navegador.

30.4 Durante a conversa

A AURORA fornece ao CLI ferramentas controladas para consultar e operar o projeto. Alterações continuam sujeitas às confirmações exibidas pela interface.

Ao criar uma nova conversa, o contexto começa limpo. Informe novamente o objetivo e os arquivos relevantes. Ao continuar uma conversa existente, a AURORA tenta retomar a sessão anterior e preservar o encadeamento do trabalho.

Mesmo quando o CLI possui recursos próprios de execução, use as confirmações exibidas pela AURORA como limite operacional. Revise o nome da ação, os argumentos e os arquivos antes de autorizar.

30.5 Problemas comuns

CLI não encontrado

Verifique se o download sob demanda conseguiu gravar em `userData\cli-cache`, se a dependência local está presente ou se o executável está no `PATH`.

Login expirado

Execute novamente `claude login` ou `codex login`.

Conversa antiga não retoma

Inicie uma nova conversa e repita o contexto necessário.

Ferramenta não responde

Confirme que a janela principal continua aberta e reinicie a conversa.

Modelo explícito não é aceito

Selecione **Default** para usar a opção compatível com a conta.

Configuração e ajuda

Configurações do aplicativo

Abra **Configurações da AURORA** pela barra superior.

As configurações controlam a apresentação da interface, o nível de detalhes dos terminais e as integrações de IA. Faça uma alteração por vez e confirme o efeito antes de continuar, principalmente ao ajustar opções usadas em diagnóstico.

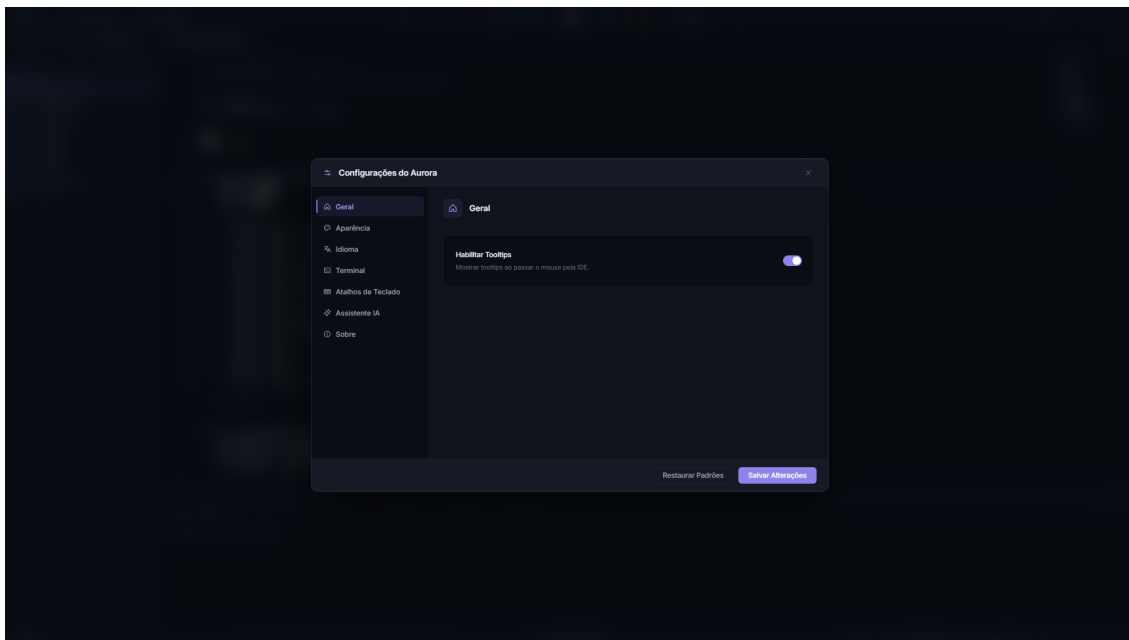


Figura31.1: As categorias ficam no menu lateral. A seção **Geral** controla as dicas exibidas ao passar o cursor sobre a interface.

31.1 Geral

Idioma

Altera a interface e as mensagens geradas pela toolchain.

Tooltips

Exibe explicações curtas ao posicionar o cursor sobre controles.

Ative tooltips enquanto estiver aprendendo a interface.

Depois de se familiarizar com os controles, você pode desativá-los para reduzir elementos sobrepostos. A alteração afeta apenas as dicas visuais; ela não modifica o projeto.

31.2 Aparência

Altere ou restaure o ícone da aplicação, quando a opção estiver disponível. A versão atual não expõe um seletor geral de tema nessa seção.

31.3 Terminal

Ative **verbose** quando precisar de detalhes para diagnóstico. Esse modo pode exibir comandos, caminhos e etapas auxiliares que ajudam a localizar a origem de uma falha. No uso normal, mantenha-o desativado para destacar as mensagens principais.

31.4 Atalhos

Consulte [Atalhos de teclado](#) para os atalhos confirmados nesta versão.

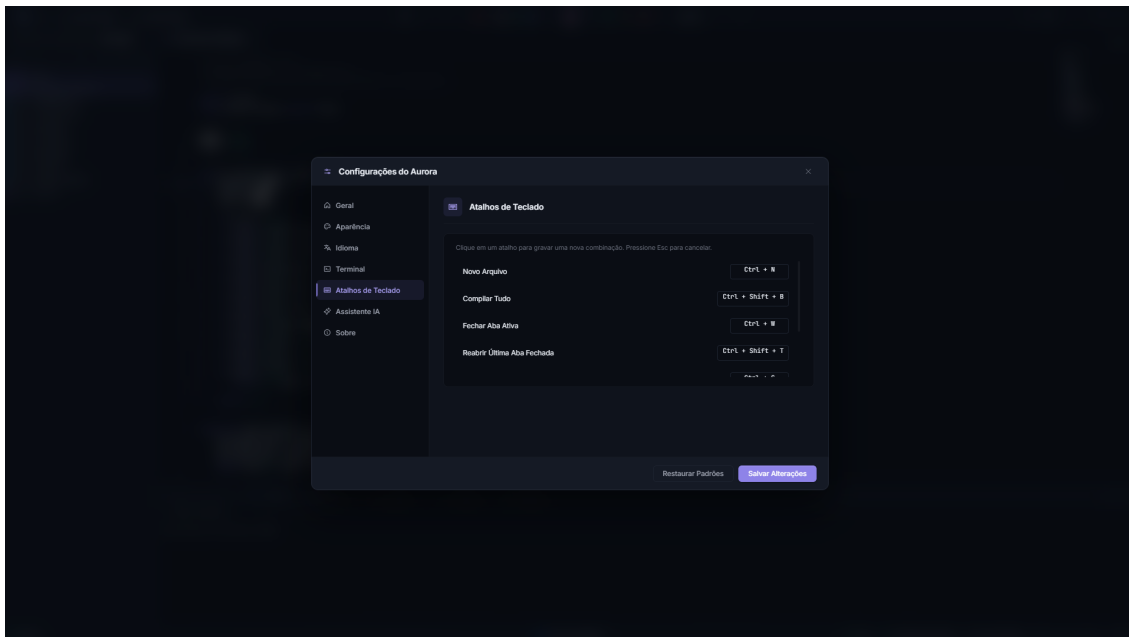


Figura31.2: Clique em uma combinação para redefini-la. Pressione Esc para cancelar a gravação de um novo atalho.

31.5 AI Assistant

Configure chaves, modelos, Ollama, Claude Code e ChatGPT via Codex CLI. Veja [Configurar o provedor de IA](#).

Depois de configurar um provedor, use o teste de conexão antes de iniciar uma tarefa no projeto. Não inclua chaves ou tokens nas mensagens da conversa.

31.6 About e atualização

Use **About** para conferir a versão instalada. Essa informação deve acompanhar relatórios de problema. Quando houver atualização disponível, leia as notas, salve os arquivos e faça backup do projeto antes de instalar.

31.7 Zoom e janela

Os controles da barra permitem minimizar, maximizar, fechar e ajustar o zoom. O PRISM possui controles equivalentes em sua própria janela.

O zoom altera a escala da interface, não o conteúdo dos arquivos. Ajuste-o quando painéis ou textos não couberem adequadamente na janela.

Referência de diretivas

Consulta rápida das diretivas da linguagem C±. Para o raciocínio por trás de cada escolha, veja [Diretivas: configurando o processador no código](#).

32.1 Diretivas de configuração

Uma por linha, no topo do `.cmm`, sem ponto e vírgula. A coluna do padrão indica o valor assumido pela cadeia de ferramentas quando a diretiva é omitida; o Hub de Processadores escreve todas explicitamente.

Diretiva	Argumento	Padrão	Efeito
#PRNAME	nome		Nome do processador. Deve casar com o nome no projeto e nos artefatos
#NUBITS	inteiro	23	Largura da palavra da ULA e dos inteiros, em complemento de dois. Deve valer a soma de mantissa, expoente e um
#NBMANT	inteiro	16	Bits de mantissa do ponto flutuante próprio
#NBEXPO	inteiro	6	Bits de expoente, em complemento de dois
#NDSTAC	inteiro	10	Profundidade da pilha de dados
#SDEPTH	inteiro	10	Profundidade da pilha de sub-rotinas
#NUIOIN	inteiro	1	Número de portas de entrada. <code>in(p)</code> exige <code>p</code> menor que esse valor
#NUIOOU	inteiro	1	Número de portas de saída. A mesma regra vale para <code>out</code>
#NUGAIN	potência de 2	64	Divisor fixo da função <code>norm()</code>
#FFTSIZ	inteiro	8	Bits invertidos no endereçamento bit-reverso <code>x[k]</code> , para FFT de 2^n pontos

Com uma única porta a ligação é direta. Com mais de uma, o *hardware* instancia automaticamente um decodificador de endereços.

32.2 Diretivas comportamentais

Aparecem no corpo do programa, marcando pontos do código.

#PRACA

Marca o ponto para onde o processador desvia quando o pino de interrupção `itr` pulsa.

#TOAQUI

Marca um endereço do programa cujo alcance faz pulsar o pino `chequei`. É também o mecanismo que a AURORA usa para detectar o fim do programa em testes e simulações.

32.3 Pré-processador

A construção `#define NOME corpo` define uma constante simbólica, apenas na forma de objeto, sem argumentos, com o limite de 256 por programa.

Não há `#include` nem `#ifdef` no fluxo C±. O fluxo C++ tem pré-processador completo, conforme [Recursos avançados](#).

32.4 A restrição estrutural

$$\text{NUBITS} = \text{NBMANT} + \text{NBEXPO} + 1$$

Combinações válidas de uso frequente:

#NUBITS	#NBMANT	#NBEXPO	Perfil
16	10	5	Compacto, cerca de 3 dígitos decimais
23	16	6	Padrão de fábrica
32	23	8	Equivalente à precisão simples do IEEE 754

Referência da biblioteca padrão

Todas as funções intrínsecas da C $\pm$, agrupadas por família. Erros de tipo são apontados na compilação; as restrições de cada grupo estão explicadas em *Entrada, saída e biblioteca padrão*.

33.1 Entrada e saída

Função	Tipos	Descrição
<code>in(p)</code>	<code>int</code>	Lê a porta de entrada <code>p</code>
<code>fin(p)</code>	<code>float</code>	Lê a porta <code>p</code> convertendo para <code>float</code>
<code>out(p, x);</code>	qualquer	Escreve <code>x</code> na porta de saída <code>p</code>
<code>fout(p, x);</code>	<code>float</code>	Escreve em formato <code>float</code>

33.2 Funções especiais

Existem para economizar *hardware*, cada uma evitando um bloco caro da ULA ou uma sequência de instruções.

Função	Tipos	Descrição
<code>norm(x)</code>	<code>int</code>	Divide pelo valor de <code>#NUGAIN</code> sem instanciar o divisor
<code>pset(x)</code>	<code>int</code> , <code>float</code>	Devolve <code>x</code> se não negativo; caso contrário, zero
<code>abs(x)</code>	todos	Valor absoluto; magnitude para <code>comp</code>
<code>sign(x, y)</code>	<code>int</code> , <code>float</code>	Devolve <code>y</code> com o sinal de <code>x</code>
<code>copy(x, y);</code>	todos	Cópia bit a bit, sem checagem de tipo

33.3 Não lineares e arredondamento

Custam instruções e ciclos, não blocos de ULA. São macros de *assembly* otimizadas injetadas no programa quando usadas.

Função	Descrição
<code>sqrt(x)</code>	Raiz quadrada, pelo método de Newton. Não aceita <code>comp</code>
<code>sin(x)</code> , <code>cos(x)</code> , <code>tan(x)</code>	Trigonométricas
<code>atan(x)</code>	Arco-tangente
<code>sinh(x)</code> , <code>cosh(x)</code> , <code>tanh(x)</code>	Hiperbólicas
<code>exp(x)</code> , <code>log(x)</code>	Exponencial e logaritmo natural. Não aceitam <code>comp</code>
<code>pow(x, y)</code>	Potência. A estratégia depende do expoente
<code>floor(x)</code> , <code>ceil(x)</code> , <code>round(x)</code>	Arredondamentos. Retornam <code>float</code> , e o <code>round</code> afasta o meio do zero

33.4 Complexos

Função	Descrição
<code>real(z)</code> , <code>imag(z)</code>	Partes real e imaginária
<code>fase(z)</code>	Argumento, ou ângulo
<code>mod2(z)</code>	Magnitude ao quadrado
<code>complex(re, im)</code>	Constrói um comp
<code>conj(z)</code>	Conjugado

⚠ Aviso

`sqrt`, `exp`, `log`, `pow`, `sign` e `pset` sobre complexos são erro de compilação, assim como o incremento `z++`. O módulo `%` e a `norm()` são exclusivos de inteiros.

33.5 Notação de Dirac

Sintaxe	Operação
<code><a b></code>	Produto interno, usado como expressão
<code>a # 0>;</code>	Zera o vetor
<code>a # M b>;</code>	Matriz vezes vetor
<code>a # c b>;</code>	Vetor escalado
<code>A # a><b ;</code>	Produto externo
<code>out(p, c a>;</code>	Emite o vetor escalado pela porta p

33.6 Operadores

Grupo	Operadores
Aritméticos	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> (só inteiros) e o <code>-</code> unário
Bit a bit	<code>&</code> <code> </code> <code>^</code> <code>~</code>
Deslocamentos	<code><<</code> <code>>></code> <code>>>></code>
Relacionais	<code><</code> <code>></code> <code><=</code> <code>>=</code> <code>==</code> <code>!=</code>
Lógicos	<code>&&</code> <code> </code> <code>!</code>
Incremento	<code>++</code>

⚠ Aviso

Ausentes na linguagem `--`, os operadores compostos (`+=`, `-=`, `*=` e afins), o ternário `?:`, o `cast` explícito e os ponteiros.

33.7 Palavras-chave

`void`, `int`, `float`, `comp`, `if`, `else`, `while`, `do`, `for`, `switch`, `case`, `default`, `break`, `continue`, `return`.

O identificador `i` é reservado para a unidade imaginária e não pode nomear variáveis.

Arquivos do projeto e backup

Esta página identifica quais arquivos fazem parte do trabalho e como mover ou arquivar um projeto com segurança. O princípio principal é preservar o `.spf` junto com os fontes e recursos aos quais ele se refere.

34.1 O que deve ser preservado

Para copiar ou arquivar um projeto, preserve a pasta completa. Os itens principais são:

Item	Conteúdo
Arquivo <code>.spf</code>	Configuração e estrutura do projeto
<code>Software/</code>	Algoritmos C±
<code>Hardware/</code>	Verilog gerado e fontes relacionadas
<code>Simulation/</code>	Arquivos de entrada, saída e testes do processador
<code>Testbenches</code>	Arquivos Verilog ou Python usados na simulação
<code>testbench/</code>	Seleção de sinais e layouts de ondas
<code>.inv</code>	Filtro visual opcional da visualização Pastas

Os arquivos em `Hardware` podem ser regenerados em muitos fluxos, mas preservá-los junto com o projeto facilita comparação, reprodução e diagnóstico. Os fontes C±, módulos adicionados manualmente e testbenches devem sempre ser tratados como dados essenciais.

34.2 Mover um projeto

1. Feche o projeto na AURORA.
2. Copie a pasta completa.
3. Abra o `.spf` na nova localização.
4. Verifique arquivos importados.
5. Confirme Top Level e Testbench Top.

Arquivos importados de fora da pasta continuam apontando para o caminho original.

Depois da cópia, execute uma validação Verilog ou abra os arquivos principais. Isso confirma que o projeto não depende de um caminho que ficou na máquina anterior.

34.3 O que não editar manualmente

Durante o uso normal, evite alterar diretamente:

- O arquivo `.spf`.
- Arquivos de estado dentro de `testbench/`.
- Dados internos da IA no perfil do Windows.
- Arquivos temporários da toolchain.

Use a interface para mudar configurações e referências. O `.inv` é a exceção: ele pode ser criado ou ajustado manualmente quando você quiser filtrar a visualização **Pastas**, pois não altera o projeto nem o Git.

Uma edição manual incorreta pode deixar o arquivo sintaticamente válido, mas inconsistente com as pastas reais. Se precisar investigar o formato, trabalhe sobre uma cópia do projeto.

34.4 Fazer backup

Use o comando de backup da AURORA antes de alterações estruturais. Para projetos importantes, mantenha também uma cópia externa ou repositório de controle de versão.

34.5 Arquivo .inv

O `.inv` fica na raiz do projeto e controla apenas a visualização **Pastas**. Ele aceita comentários, linhas vazias, negação com `!`, padrões de diretório, globs e `**`. A comparação é feita sem diferenciar maiúsculas de minúsculas.

Não use `.inv` para segurança, empacotamento ou Git. Para controle de versão, use `.gitignore` e revise o painel de Source Control.

34.6 Layouts de ondas

O diretório `testbench/` guarda um JSON por Testbench Top. Esse estado registra sinais selecionados, inicialização da **Configuração de Ondas**, layouts `.gtkw` e layouts Surfer (`.surf.ron` ou `.suc1`) associados ao testbench.

Layouts `.gtkw` organizam sinais para GTKWave. Layouts `.surf.ron` e `.suc1` configuram o Surfer. Eles não substituem o arquivo de onda (`.vcd` ou `.fst`), que continua sendo a fonte dos valores simulados.

Nunca dependa apenas da pasta de arquivos temporários ou da lista de projetos recentes.

34.7 Verificar um backup

1. Copie o backup para uma pasta de teste.
2. Abra o `.spf` dessa cópia.
3. Confira processadores, Top Level e Testbench Top.
4. Abra os fontes principais.
5. Execute a validação apropriada.

Um backup só deve ser considerado utilizável depois dessa verificação. Evite testar diretamente sobre a única cópia preservada.

Atalhos de teclado

Os atalhos seguem o contexto ativo: um comando do editor exige que o foco esteja no arquivo, enquanto um comando da *Configuração de ondas* exige que essa janela esteja ativa. Todos são reconfiguráveis em *Configurações do Aurora* ▢ *Atalhos de Teclado*.

35.1 Arquivos e abas

Atalho	Ação
Ctrl+N	Criar um arquivo novo
Ctrl+S	Salvar o arquivo ativo
Ctrl+Shift+S	Salvar todos os arquivos modificados
Ctrl+W	Fechar a aba ativa
Ctrl+Shift+T	Reabrir a última aba fechada

35.2 Navegação e comandos

Atalho	Ação
Ctrl+Shift+F	Buscar nos arquivos do projeto
Ctrl+Shift+P	Abrir a paleta de comandos da AURORA
F1	Abrir a paleta de comandos do Monaco
Ctrl+G	Ir para uma linha
F12	Ir para a definição, quando o servidor de linguagem a fornece
Shift+F12	Localizar referências, quando disponível

35.3 Edição

Atalho	Ação
Ctrl+F	Localizar no arquivo
Ctrl+H	Localizar e substituir
Ctrl+Z e Ctrl+Y	Desfazer e refazer
Alt+Click	Adicionar um cursor
Ctrl+/	Comentar ou descomentar a linha
Shift+Alt+F	Formatar o documento com o clang-format

35.4 Ferramentas de linguagem

Atalho	Ação
Ctrl+Alt+S	Ligar ou desligar a análise semântica do slang, mais pesada

35.5 Configuração de ondas

Atalho	Ação
Ctrl+F	Focar a pesquisa
Esc	Limpar a pesquisa; pressione de novo para fechar

35.6 Redefinir um atalho

Em *Configurações do Aurora* ▢ *Atalhos de Teclado*, clique na combinação que quer trocar e tecle a nova. Combinações em conflito são recusadas com aviso, e Esc cancela a gravação.

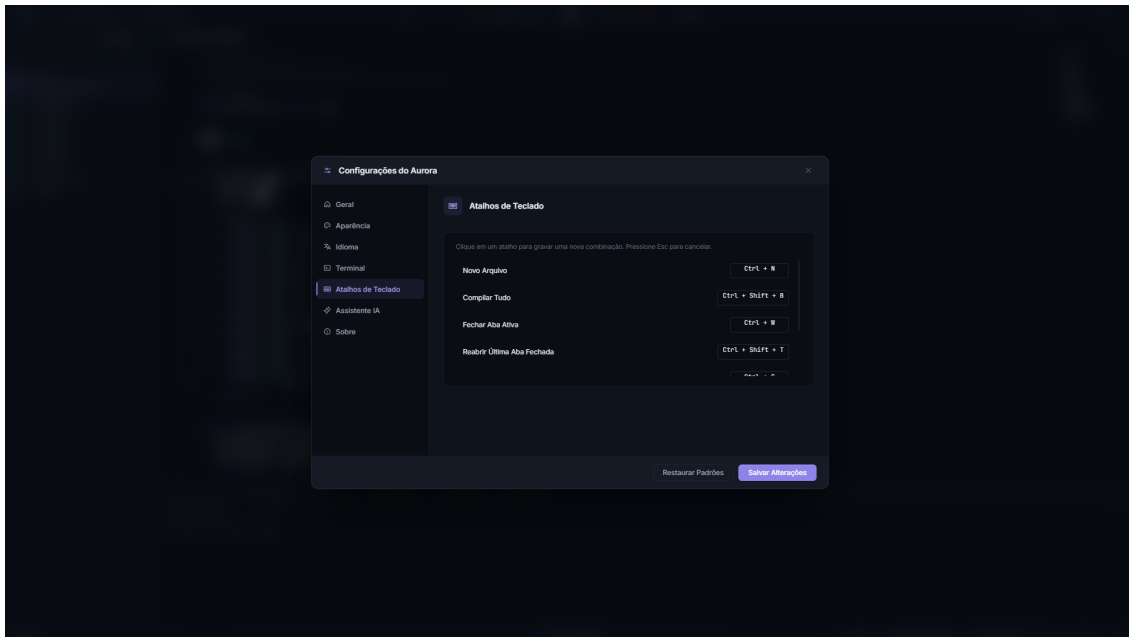


Figura35.1: Cada linha da lista pode ser regravada. As combinações já usadas por outro comando são recusadas.

i Nota

Dentro do editor valem ainda os atalhos usuais do Monaco, herdados do Visual Studio Code, mesmo os não listados aqui.

Se um atalho não responder, clique primeiro na área em que ele deve atuar. Menus, diálogos abertos ou ferramentas externas em primeiro plano podem capturar a combinação.

Solução de problemas

Use esta página para localizar a etapa que falhou antes de alterar configurações ou reinstalar ferramentas. Um diagnóstico eficiente registra o contexto, reproduz uma única ação e começa pela primeira mensagem de erro.

36.1 Comece por aqui

1. Salve todos os arquivos.
2. Confira projeto, processador, Top Level, Testbench Top e simulador na barra de status.
3. Repita apenas a ação que falhou.
4. Leia a primeira mensagem de erro no terminal correspondente.
5. Ative **verbose** somente se precisar de mais detalhes.

Anote o resultado esperado e o que realmente aconteceu. Essa comparação evita descrições genéricas como “não funciona” e ajuda a decidir se a falha está na interface, na configuração do projeto, na compilação ou no testbench.

36.2 Interface

Um botão está desabilitado

Abra o arquivo esperado e confirme Top Level ou Testbench Top. Veja [Tour pela interface](#).

O editor não carrega

Feche e abra a AURORA. Se continuar, reinicie o Windows e repita a abertura.

Um arquivo mudou fora da AURORA

Escolha conscientemente entre a versão do disco e a versão do editor.

36.3 Compilação

Top Level não encontrado

Confirme o módulo principal e inclua suas dependências.

Módulo duplicado

Remova uma das fontes que declaram o mesmo módulo.

A geração C± não atualiza o hardware

Salve o .cmm, confirme o processador ativo e execute C± novamente.

O cancelamento demora

Aguarde alguns segundos para que processos auxiliares sejam encerrados.

Depois de uma falha de compilação, não use arquivos antigos em Hardware como evidência de sucesso. Verifique a data de atualização e confirme que o terminal da execução atual terminou corretamente.

36.4 Simulação e ondas

Testbench não inicia

Confirme Testbench Top, clock, reset e nomes de módulos.

Nenhum arquivo de onda

Confirme que o testbench terminou e possui sinais configurados para dump.

Viewer de ondas abre sem sinais

Volte o layout para **padrão**, revise a **Configuração de Ondas** e gere novamente.

O cocotb não encontra um sinal

Compare o nome usado no Python com as portas reais do módulo.

Se a simulação não termina, confira se o testbench possui uma condição de encerramento. Use **Cancelar** antes de iniciar outra execução.

36.5 PRISM

Execute **Verilog** antes do PRISM. Corrija primeiro erros de módulo ausente, módulo duplicado e Top Level incorreto.

Se o Verilog está válido, mas o diagrama continua desatualizado, salve os arquivos e use **Recompile** na janela do PRISM.

36.6 Aurora Intelligence

Chave salva, mas a conexão falha

Confirme conta, modelo, rede e permissões do provedor.

Ollama não detecta modelos

Confirme que ollama serve está ativo e use `http://localhost:11434/v1`.

Claude Code ou ChatGPT via Codex CLI não conecta

Execute novamente o login, confirme `--version` e reinicie a AURORA.

36.7 Pedir suporte

Inclua:

- Versão da AURORA e do Windows.
- Ação exata que falhou.
- Resultado esperado e observado.
- Saída exportada do terminal.
- Projeto mínimo sem dados confidenciais.

Inclua apenas o necessário para reproduzir o problema. Remova chaves, dados pessoais e arquivos que não participam do fluxo. Quando possível, descreva uma sequência curta de passos que leve ao mesmo resultado.

Apêndices

Glossário

Os termos usados de forma consistente neste manual. Quando a interface mantém um nome em inglês, o termo é preservado para facilitar a localização na AURORA.

.asm

O *assembly* intermediário do SAPHO, entre o compilador e o montador.

.gtkw

Arquivo de *layout* do GTKWave: quais sinais mostrar, em que ordem e com que cores. Não contém os dados da simulação.

.mif

Memory Initialization File. Imagem de memória, de programa ou de dados, embutida no circuito gerado.

.spf

SAPHO Project File. O arquivo JSON que define um projeto: metadados, processadores, pastas, *top-level* e *testbench*.

.surf.ron

O equivalente do `.gtkw` para o Surfer.

ACC

O acumulador, registrador central do processador e destino de toda operação da ULA.

AURORA

Advanced Utility Running Optimized Resource Architectures. A IDE de desktop da plataforma, e a única interface gráfica do ecossistema.

Aurora Intelligence

A assistente de inteligência artificial integrada à AURORA, capaz de conversar sobre o projeto e de agir sobre a IDE por ferramentas com permissão controlada.

bit-reverso

Endereçamento com os bits do índice invertidos, escrito $x[k]$, usado na FFT.

BYOK

Bring Your Own Key. O modelo de chaves da Aurora Intelligence, no qual você conecta a sua própria conta de um provedor de modelos.

cocotb

Coroutine cosimulation testbench. Arcabouço Python para a escrita de *testbenches*, executado em co-simulação com o Icarus ou o Verilator.

CommandSpec

Contrato interno da AURORA que representa executável, argumentos, diretório de trabalho e ambiente sem montar um comando de *shell*.

C±

A linguagem de programação da plataforma, um dialeto de C com complexos nativos e notação de Dirac. Arquivos com extensão `.cmm`. Lê-se "C mais-menos".

Dagr

O painel de controle de versão da AURORA. Do nórdico antigo, "dia"; o painel usa a runa *dagaz* como marca.

DLP

Dispositivos Lógicos Programáveis. A disciplina da UFJF que usa a plataforma no ensino de graduação.

FFT

Fast Fourier Transform. Algoritmo de transformada discreta de Fourier, cuja variante radix-2 produz resultados em ordem bit-reversa.

FPGA

Field-Programmable Gate Array. Circuito integrado cujas conexões internas podem ser reprogramadas depois da fabricação.

FST

Fast Signal Trace. Formato binário compacto de formas de onda, cerca de dez vezes menor que o VCD.

GTKWave

Visualizador de formas de onda padrão, em *fork* próprio do laboratório com tema escuro e ajustes de usabilidade.

Harvard

Arquitetura com memórias de programa e de dados fisicamente separadas, cada uma com o seu barramento.

HDL

Hardware Description Language. A classe de linguagens usadas para descrever circuitos, à qual pertencem o Verilog e o VHDL.

Icarus Verilog

O simulador de Verilog padrão da plataforma. Compila com o `iverilog` e executa com o `vvp`.

ISA

Instruction Set Architecture. O conjunto de instruções do processador.

LSP

Language Server Protocol. O mecanismo por trás dos diagnósticos e da navegação de código, servido pelo Verible e pelo slang.

MCP

Model Context Protocol. O protocolo aberto pelo qual ferramentas de linha de comando de IA acessam as ferramentas da AURORA.

Monaco

O motor de edição de código da AURORA, o mesmo do Visual Studio Code.

NIPS-CERN

O Núcleo de Instrumentação e Processamento de Sinais da UFJF, em parceria com o CERN. O laboratório que desenvolve a plataforma.

pipeline

O encadeamento de estágios pelos quais uma instrução passa. No SAPHO são três: busca, decodificação e execução.

PRISM

Processor Rendering Interface for Schematic Models. O visualizador de RTL da AURORA, apoiado no Yosys e no netlistsvg.

RTL

Register-Transfer Level. O nível de abstração em que o circuito é descrito como transferências entre registradores.

SAPHO

Scalable-Architecture Processor for Hardware Optimization. O processador *soft-core* e, por extensão, a plataforma completa e o canal de distribuição.

soft-processor

Processador descrito em HDL e implementado na malha reconfigurável de um FPGA. Também dito *soft-core*.

Surfer

Visualizador de formas de onda opcional, escrito em Rust, no *fork* `surfer-aurora`.

testbench

O banco de testes da simulação, em Verilog (`_tb.v`) ou em Python com cocotb, que aplica estímulos ao circuito e confere as respostas.

top-level

O módulo Verilog que integra os processadores e blocos do projeto. É o topo da síntese e o alvo do PRISM.

ULA

Unidade lógica e aritmética. No SAPHO, apenas os operadores usados pelo programa são instanciados nela.

VCD

Value Change Dump. Formato textual de arquivo de formas de onda.

Verilator

Simulador de alto desempenho que converte o circuito em executável nativo em C++.

Verilog

A linguagem de descrição de *hardware* usada pela plataforma.

YANC

Yet Another Compiler. A suíte de compiladores que traduz C $\pm$ e C++ em processador Verilog, imagens de memória e *testbench*.

Yosys

A ferramenta de síntese usada pelo PRISM e pela visão Hierarquia da árvore de arquivos.

O ecossistema SAPHO

Esta página situa a AURORA no conjunto de que ela faz parte: quem desenvolve a plataforma, quais componentes a formam, onde ela é usada e como as peças se relacionam. É leitura de contexto, não de procedimento.

B.1 O laboratório

O SAPHO é desenvolvido e mantido pelo NIPS-CERN, o Núcleo de Instrumentação e Processamento de Sinais da Universidade Federal de Juiz de Fora, em Minas Gerais, em parceria com o CERN (*Conseil Européen pour la Recherche Nucléaire*).

O grupo atua no experimento ATLAS do LHC (*Large Hadron Collider*), no qual feixes de prótons se cruzam a uma taxa de 40 MHz. Processar esses sinais em tempo real e com baixa latência exige FPGAs, e quando o algoritmo a embarcar é extenso, recorre-se a processadores embarcados. Para não desperdiçar área lógica com arquiteturas fixas, o laboratório desenvolveu o SAPHO: um núcleo cuja arquitetura é gerada sob demanda, sintetizando em *hardware* apenas os recursos exigidos por cada aplicação.

Todo o ecossistema é de código aberto, hospedado na organização `nipscernlab` no GitHub. O canal oficial de distribuição para o usuário final é nipscern.com/sapho².

B.2 Os componentes

B.2.1 Desenvolvidos pelo laboratório

SAPHO

O processador *soft-core* de arquitetura gerada sob demanda, descrito em *O processador SAPHO por dentro*.

C±

A linguagem de médio nível derivada de C, estendida com tipo complexo nativo e notação de Dirac para álgebra linear, documentada em *A linguagem C±: fundamentos*.

YANC

A cadeia de compiladores, escrita em C com o auxílio das ferramentas Flex e Bison, que traduz C± e C++ até Verilog sintetizável.

AURORA

A IDE construída sobre o Electron, que orquestra a cadeia inteira e é a única interface gráfica do ecossistema.

PRISM

O visualizador de RTL embutido, apoiado no Yosys e no netlistsvg, descrito em *Visualizar e simular o RTL no PRISM*.

Aurora Intelligence

A assistente de IA especializada na linguagem e no fluxo, descrita em *Aurora Intelligence*.

Dagr

O painel de controle de versão integrado, descrito em *Source Control e Git*.

² <https://nipscern.com/sapho>

B.2.2 Ferramentas de terceiros empacotadas

Todas acompanham a instalação, em versões validadas em conjunto, e nenhuma exige licença: Icarus Verilog e Verilator para simulação, Yosys para síntese, GTKWave e Surfer para formas de onda, o editor Monaco, os servidores de linguagem Verible e slang, o clang-format e um Python com cocotb, NumPy e SciPy.

O catálogo completo com as respectivas licenças está em *Configurações do Aurora* □ *Sobre*.

B.3 Como as peças se encaixam

A separação em dois processos vem do Electron, o arcabouço que une, em um único executável, o motor de renderização Chromium, que desenha a interface, e o ambiente Node.js, que tem acesso ao sistema de

arquivos. É essa separação que permite à IDE invocar compiladores e simuladores como processos filhos, algo que um navegador comum, isolado em ambiente restrito, não poderia fazer.

B.4 Onde a plataforma é usada

Em pesquisa

Da geração de fractais em FPGA à simulação de pulsos do calorímetro de telhas do ATLAS em tempo real, passando por unidades de medição fasorial conformes à norma IEC/IEEE 60255-118-1 e por redes neurais convolucionais embarcadas para estimativa de amplitude de sinais.

No ensino

Na disciplina de Dispositivos Lógicos Programáveis da graduação em Engenharia Elétrica da UFJF, na qual os alunos projetam, simulam e inspecionam processadores de ponta a ponta em um único ambiente, aprendendo os fundamentos do *hardware* digital reconfigurável sem depender de licenças proprietárias.

B.5 O que fica de fora

Vale ser explícito sobre a fronteira. A cobertura aberta da AURORA alcança a modelagem, a simulação e a inspeção estrutural, mas não a implementação física: a síntese dirigida ao dispositivo, o posicionamento e a gravação dependem de informação proprietária sobre a arquitetura interna de cada FPGA e seguem a cargo do ambiente do fabricante, como o Quartus ou o Vivado.

Há também um limite inerente a qualquer simulação: ela valida o modelo do circuito, não o dispositivo físico. Efeitos da implementação real, como violações de temporização após o roteamento, não se manifestam no ambiente ideal, e a validação em ferramenta não dispensa a verificação em *hardware*.

Por fim, a AURORA é especializada no ecossistema SAPHO e na cadeia YANC. A generalização para outros fluxos de projeto não é imediata.

B.6 Para citar a plataforma

Os trabalhos que descrevem a AURORA, o SAPHO e os componentes do ecossistema estão publicados em anais de congressos e periódicos da área. As referências completas e atualizadas ficam na página do projeto em [nipscern.com/projects/aurora](https://www.nipscern.com/projects/aurora)³.

³ <https://www.nipscern.com/projects/aurora>

Escopo, versão e método

C.1 O que foi documentado

Esta documentação descreve a aplicação desktop AURORA no estado observado no repositório local. O conteúdo é orientado ao uso da aplicação por:

- **Estudantes e projetistas**, que precisam criar projetos, processadores, testbenches e interpretar resultados.
- **Docentes e equipes de laboratório**, que precisam explicar o fluxo SAPHO e reproduzir ambientes.
- **Usuários avançados**, que precisam configurar simulação, formas de onda, controle de versão e Aurora Intelligence.

Item	Valor congelado
Produto	AURORA IDE / SAPHO
Versão do pacote	6.3.2
Commit	cee4922189e746b83e0198fd998ed50646f18371
Branch	main
Plataforma primária	Windows 10/11
Runtime	Electron 39.8.10 e Node.js 18+
Editor	Monaco Editor 0.52.2, fixado exatamente
Data da análise	9 de julho de 2026

C.2 Fontes e precedência

As informações foram consolidadas nesta ordem de confiança:

1. Comportamento implementado no código-fonte.
2. Testes unitários e E2E.
3. Arquivos de configuração, scripts de bootstrap, empacotamento e release.
4. Documentos ARCHITECTURE.md, README.md, RELEASE.md, SECURITY.md e documentação interna.
5. Wiki local de apoio em C:\Users\Computador\Documents\Arthur\NIPSCERN\Backup_Last_Project\Project_Wiki, especialmente a rebaseline e o inventário funcional de 9 de julho de 2026.
6. Páginas públicas oficiais do NIPSCERN sobre AURORA, SAPHO e YANC.
7. Documentação oficial das tecnologias integradas.

i Nota

A wiki local é usada como apoio de rastreabilidade, não como substituta do código. Quando a wiki e o código divergem, o comportamento implementado no commit documentado prevalece.

C.3 Convenções

- **C±** e **CMM** referem-se à linguagem de entrada usada pelo SAPHO e à extensão `.cmm`.
- **Processador** é o núcleo de hardware gerado a partir de um algoritmo C±.
- **Top Level** é o módulo Verilog raiz da síntese.
- **Testbench Top** é o arquivo de simulação selecionado; pode ser Verilog (`.v`) ou cocotb/Python (`.py`).
- Caminhos entre `<...>` são exemplos e devem ser substituídos pelo valor local.
- Opções marcadas como *legadas* existem no DOM ou em configurações antigas, mas não constituem um fluxo ativo confiável nesta versão.

C.4 Limites

Este manual não reproduz a especificação completa da linguagem C± nem os manuais de cada ferramenta externa. Ele explica como executar tarefas na AURORA, compreender os arquivos produzidos e verificar os resultados observáveis de cada fluxo.

Não foram alterados arquivos do código-fonte e nenhum commit foi criado durante a produção deste material.

Índice

Símbolos

.asm, [146](#)
.gtkw, [146](#)
.mif, [146](#)
.spf, [146](#)
.surf.ron, [146](#)

A

ACC, [146](#)
AURORA, [146](#)
Aurora Intelligence, [146](#)

B

bit-reverso, [146](#)
BYOK, [146](#)

C

cocotb, [146](#)
CommandSpec, [146](#)
C±, [146](#)

D

Dagr, [146](#)
DLP, [146](#)

F

FFT, [146](#)
FPGA, [147](#)
FST, [147](#)

G

GTKWave, [147](#)

H

Harvard, [147](#)
HDL, [147](#)

I

Icarus Verilog, [147](#)
ISA, [147](#)

L

LSP, [147](#)

M

MCP, [147](#)
Monaco, [147](#)

N

NIPS-CERN, [147](#)

P

pipeline, [147](#)
PRISM, [147](#)

R

RTL, [147](#)

S

SAPHO, [147](#)
soft-processor, [147](#)
Surfer, [147](#)

T

testbench, [147](#)
top-level, [147](#)

U

ULA, [148](#)

V

VCD, [148](#)
Verilator, [148](#)
Verilog, [148](#)

Y

YANC, [148](#)
Yosys, [148](#)